Stochastic Numerics

Sebastian Lalvay

October 4, 2025

Contents

1 Stochastic Differential Equations

1.1 General Definitions

Let $\sigma : [0, \infty) \times \mathbb{R}^m \to \mathbb{R}^{m \times n}$ and $b : [0, \infty) \times \mathbb{R}^m \to \mathbb{R}^m$ be two Borel measurable functions and consider the Stochastic Differential Equation (SDE)

$$dX_t = b(t, X_t)dt + \sigma(t, X_t)dW_t, \quad X_0 \overset{\mathcal{L}}{\sim} \nu \tag{1}$$

where $W_t = \left(W_t^{(1)}, ..., W_t^{(n)}\right)$ is an n-dimensional Brownian Motion.

Definition 1 (Solution)**.** A solution to the SDE (1) is a pair (X, W) of processes defined on a filtered probability space $(\Omega, \mathscr{F}, \{\mathscr{F}_t\}_{t\geq 0}, \mathbb{P})$, where

(i) $X = \{X_t\}_{t\geq 0}$ is an $\mathbb{R}^m$-valued $\mathscr{F}_t$-continuous process.
$W = \{W_t\}_{t\geq 0}$ is a standard $\mathscr{F}_t$-Brownian motion.

(ii) For any $t \geq 0$, the integrals $\int_0^t b(s, X_s)ds$ and $\int_0^t \sigma(s, X_s)dW_s$ exists:

$$\int_0^t |b_i(s, X_s)|ds + \int_0^t \sigma_{ij}^2(s, X_s)ds < \infty \quad \mathbb{P} - a.s., \text{ for } 1 \leq i \leq m,\ 1 \leq j \leq n.$$

(iii) X satisfies (1), i.e., for $t \geq 0$,

$$X_t = X_0 + \int_0^t b(s, X_s)ds + \int_0^t \sigma(s, X_s)dW_s, \quad \mathbb{P} - a.s.$$

Remark 1. The probability measure $\nu(B) = \mathbb{P}(X_0 \in B)$, $B \in \mathscr{B}(\mathbb{R}^m)$ is called the initial distribution of the solution.

Definition 2 (Weak existence)**.** Weak existence holds for equation (1) if there exists a solution to (1) on some filtered probability space.

In Definition 1, the probability space, the filtration, and the Brownian motion are part of the solution. Usually, a solution in the sense of Definition 1 is referred as a **weak solution**. A different notion of a solution can be constructed on a given probability space with respect to a given filtration and a given Brownian motion.

We choose a probability space $(\Omega, \mathscr{F}, \mathbb{P})$ and a n-dimensional Brownian motion $W = \{W_t\}_{t\geq 0}$ defined on this space. Furthermore, we take a random vector ξ in the same probability space, which is independent of W. We now consider the filtration

$$\mathscr{H}_t := \sigma(\xi, W_s; 0 \leq s \leq t),$$

together with the collection of null sets

$$\mathscr{N} := \{N \subseteq \Omega : \exists H \in \mathscr{H}_\infty \text{ with } N \subseteq H \text{ and } \mathbb{P}(H) = 0\}, \quad \mathscr{H}_\infty := \sigma\left(\bigcup_{t\geq 0} \mathscr{H}_t\right),$$

and define the augmented filtration

$$\tilde{\mathscr{F}}_t^W := \sigma(\mathscr{H}_t \cup \mathscr{N}). \tag{2}$$

Definition 3 (Strong solution-Strong existence)**.**

(i) A solution (X, W) to (1) with initial condition ξ is said to be strong if X is adapted to the augmented filtration $\tilde{\mathscr{F}}_t^W$.

(ii) Strong existence holds for equation (1) if there is a strong solution to (1) on some probability space.

Remark 2. From the above definitions, it follows that the existence of a strong solution imply that of a weak solution, however, the converse is not true (see [**?**, Ex. 5.3.5, pg 301] or [**?**, Ex. 5.2, pg 183]).

There are two natural definition of uniqueness of solutions for the SDE (1).

Definition 4 (Pathwise Uniqueness)**.** We say that pathwise uniqueness of solutions for (1) holds if whenever (X, W) and $(\tilde{X}, W)$ are weak solutions with common Brownian motion W (with possibly different filtrations) on a common space $(\Omega, \mathscr{F}, \mathbb{P})$ and with common initial value, i.e., $X_0 = \tilde{X}_0$ $\mathbb{P}$-$a.s.$, then $X_t = \tilde{X}_t$ for all $t \geq 0$ $\mathbb{P}$-$a.s.$

Definition 5 (Uniqueness in probability)**.** We say that uniqueness in probability holds for equation (1) if, whenever (X, W) and $(\tilde{X}, \tilde{W})$ are two weak solutions whose initial distribution coincide, i.e.,

$$\mathbb{P}(X_0 \in B) = \tilde{\mathbb{P}}(\tilde{X}_0 \in B), \forall B \in \mathscr{B}(\mathbb{R}^m),$$

the processes $X, \tilde{X}$ have the same law.

The next results clarify the relationship between the different notions of existence and uniqueness. For $N \in \mathbb{N}$, we denote $\mathscr{C}_N = C([0, T]; \mathbb{R}^N)$ and for a metric space M, $\mathscr{B}(M)$ stands for its Borel σ algebra.

Proposition 1. *Let (X, W) be a strong solution to (1) on $(\Omega, \mathscr{F}, \{\mathscr{F}_t\}_{t\geq 0}, \mathbb{P})$. Then the following statements hold:*

(i) There exists a $\mathscr{B}(\mathbb{R}^m) \otimes \mathscr{B}(\mathscr{C}_n)$ - $\mathscr{B}(\mathscr{C}_m)$ measurable map

$$F : \mathbb{R}^m \times \mathscr{C}_n \longrightarrow \mathscr{C}_m$$

such that $F(X_0, W_.)$ is $\tilde{\mathscr{F}}_t^W$ adpated and

$$X_. = F(X_0, W_.) \quad \mathbb{P} - a.s. \tag{3}$$

(ii) If $\tilde{W}$ is a n-dimensional $\{\tilde{\mathscr{F}}\}_{t\geq 0}$ Brownian motion and ξ is a m dimensional random variable on a filtered probability space $(\tilde{\Omega}, \tilde{\mathscr{G}}, \{\tilde{\mathscr{G}}\}_{t\geq 0}, \tilde{\mathbb{Q}})$ such that $\tilde{X}_. = F(\xi, \tilde{W}_.)$, then $(\tilde{X}, \tilde{W})$ is a strong solition to (1).

Remark 3. The process $X : [0, T] \times \Omega \to \mathbb{R}^d$ can induce a random variable from Ω to $C([0, T]; \mathbb{R}^m) := C_m$. Thus, the Law of X is the image measure Λ in C_m. Also, from previous Proposition (i), by (3) and the transfer theorem, the law of X is a measure Λ on $\mathbb{R}^m \times C_n$ by F.

$$\mathbb{E}[X.] = E[F(X_0, W.)]$$

Now, let $e_t : C_m \to \mathbb{R}^m$ given by

$$e_t(X.) = X_t$$

We consider (C_m, Λ), then the Law of X_t, denoted by $\mathcal{L}(X_t)$ is the image of the law of X by e_t.

Theorem 1 (Itô: Existence and Uniqueness Results)**.** *Suppose that the functions $\sigma(t, x)$ and $b(t, x)$ satisfy the global Lipschitz and linear growth conditions*

$$\|b(t, x) - b(t, y)\| + \|\sigma(t, x) - \sigma(t, y)\| \leq K\|x - y\|, \quad \forall t \geq 0, \forall x, y \in \mathbb{R}^m, \tag{4}$$

$$\|b(t, x)\| + \|\sigma(t, x)\| \leq L(1 + \|x\|), \quad \forall t \geq 0, \forall x \in \mathbb{R}^m, \tag{5}$$

for some $K > 0$ and $L > 0$. On a probability space $(\Omega, \mathscr{F}, \mathbb{P})$, let ξ be an $\mathbb{R}^m$-valued random vector, independent of the n-dimensional Brownian motion $W = \{W_t\}_{t\geq 0}$. Then there exists a unique (pathwise) continuous, adapted process $X = \{X_t\}_{t\geq 0}$ that is a strong solution to (1) with respect to W and initial condition ξ. If in addition, for $p \geq 2$ we assume that $\mathbb{E}[\|\xi\|^p] < \infty$, then the process X is square integrable and

$$\mathbb{E}\|X_t\|^p \leq C(1 + \mathbb{E}\|\xi\|^p)e^{Ct}, \quad t \geq 0. \tag{6}$$

Remark 4. (a) The inequality (6) is used to determine the error rate of the Euler Maruyama aproximation.

1.2 Connections between SDEs and PDEs

For simplicity, in this section we will consider the following SDE

$$dX_t = a(X_t)dt + dW_t \tag{7}$$

We are interesting in computing

$$\mathbb{E}[\varphi(X_t)] = \int_{\mathbb{R}^d} \varphi d\mathcal{L}(X_t) \quad \text{(By Transfer Theroem)} \tag{8}$$

where $\varphi : \mathbb{R}^d \to \mathbb{R}$ is a test function. More generally, we consider a test function $\varphi : [0,\infty) \times \mathbb{R}^d \to \mathbb{R}$ with compact support in $[0,\infty) \times \mathbb{R}^d$ and with smooth derivatives of any order.

Proposition 2. *Let $(X_t)_{t\geq 0}$ to be the solution of the SDE*

$$dX_t = b(X_t)\,dt + dB_t \tag{9}$$

for b as before. We define

$$\mu_t = \mathcal{L}(X_t), \quad t \geq 0 \tag{10}$$

We then have:

$$\begin{aligned}\int_{\mathbb{R}^d} \varphi(t,x)\,d\mu_t(x) = &\int_{\mathbb{R}^d} \varphi(0,x)\,d\mu_0(x)\\ &+ \frac{1}{2}\int_0^t \int_{\mathbb{R}^d} \Delta\varphi(s,x)\,d\mu_s(x)\,ds\\ &+ \int_0^t \int_{\mathbb{R}^d} \nabla_x\varphi(s,x)\cdot b(x)\,d\mu_s(x)\,ds\\ &+ \int_0^t \int_{\mathbb{R}^d} \partial_s\varphi(s,x)\,d\mu_s(x)\,ds\end{aligned} \tag{11}$$

with the diffusive term defined by

$$\Delta\varphi = \sum_{i=1}^d \frac{\partial^2\varphi}{\partial x_i^2} \tag{12}$$

2 Nonlinear First-Order PDE

2.1 Characteristics

This part is based on [3] pp. 96. We consider the basic nonlinear first-order PDE

$$\begin{aligned} F(Du,u,x) = 0 \quad &\text{in } U \subseteq \mathbb{R}^n\\ u = g \quad &\text{on } \Gamma, \end{aligned} \tag{13}$$

where $u : \mathbb{R}^n \to \mathbb{R}$, $F : \mathbb{R}^n \times \mathbb{R} \times U \to \mathbb{R}$, $\Gamma \subseteq \partial U$ and $g : \Gamma \to \mathbb{R}$ are given. We assume that F, g are smooth enough functions. Note that

$$Du(x) = (u_{x_1}(x), ..., u_{x_n}(x)) \in \mathbb{R}^n, \quad \frac{\partial u}{\partial x_i}(x) = u_{x_i}(x)$$

and

$$\frac{\partial Du}{\partial x_i}(x) = (u_{x_1 x_i}, ..., u_{x_n x_i})$$

Method of characteristics: Solves (13) by converting the PDE into an appropriate system of ODE. Suppose u solves (13) and fix $x \in U$. We would like to calculate $u(x)$ by finding some curve lying within U, connecting x with $x^0 \in \Gamma$ and along which we can compute u. Since $u = g$ we know u at x^0. We hope then to calculate u all along the curve, and so in particular at x.

Finding the characteristics: Suppose the curve is described parametrically by

$$x(s) = (x^1(s), ..., x^n(s)), \quad s \in I \subseteq \mathbb{R}. \tag{14}$$

Assuming $u \in C^2 (u_{x_i x_j} = u_{x_j x_i})$ is a solution of (13), we define

$$z(s) := u(x(s)). \tag{15}$$

We also define $p(s) = (p^1(s), ..., p^n(s))$ by

$$p(s) := Du(x(s)), i.e., \tag{16}$$
$$p^i(s) = u_{x_i}(x(s)), \quad i = 1, ..., n. \tag{17}$$

We must choose $x(\cdot)$ in such a way that we can compute $z(\cdot)$ and $p(\cdot)$. For this, first we differentiate (17) with respect to s

$$\dot{p}^i(s) = \sum_{j=1}^{n} u_{x_i x_j}(x(s)) \dot{x}^j(s) \tag{18}$$

We can also differentiate the PDE (13) with respect to x_i, namely

$$\frac{\partial}{\partial x_i} F(Du(x), u(x), x) = D_p F \cdot \frac{\partial p}{\partial x_i} + \frac{\partial F}{\partial u} \cdot \frac{\partial u}{\partial x_i} + \frac{\partial F}{\partial x_i},$$

where

- $p(x) := Du(x)$
- $D_p F = \left(\frac{\partial F}{\partial p_1}, \ldots, \frac{\partial F}{\partial p_n} \right)$,
- $\frac{\partial p}{\partial x_i} = \left(\frac{\partial^2 u}{\partial x_1 \partial x_i}, \ldots, \frac{\partial^2 u}{\partial x_n \partial x_i} \right)^T = (u_{x_1 x_i}, ..., u_{x_n x_i})^T$.

Then

$$\frac{\partial}{\partial x_i}F(Du(x),u(x),x)=\sum_{j=1}^{n}F_{p_j}(Du,u,x)u_{x_jx_i}+F_z(Du,u,x)u_{x_i}+F_{x_i}(Du,u,x)=0 \quad (19)$$

We now set

$$\dot{x}^j(s)=F_{p_j}(p(s),z(s),x(s)),\quad j=1,...,n. \quad (20)$$

Assuming (20) holds, we evaluate (19) at $x=x(s)$, obtaining from (15),(16) the identity

$$\sum_{j=1}^{n}F_{p_j}(p(s),z(s),x(s))u_{x_ix_j}(x(s))+F_z(p(s),z(s),x(s))p^i(s)+F_{x_i}(p(s),z(s),x(s))=0 \quad (21)$$

Substitute this expression and (20) into (22):

$$\dot{p}^i(s)=-F_{x_i}(p(s),z(s),x(s))-F_z(p(s),z(s),x(s))p^i(s),\ \ i=1,...,n. \quad (22)$$

Finally, we differentiate (15):

$$\dot{z}(s)=\sum_{j=1}^{n}u_{x_j}(x(s))\dot{x}^j(s)=\sum_{j=1}^{n}p^j(s)F_{p_j}(p(s),z(s),x(s)), \quad (23)$$

the second equality holding by (17) and (20).

The characteristic equations: We sumarize by rewriting equations (20),(22),(23) in vector notation in the following theorem:

Theorem 2 (Characteristics ODE)**.** *Let $u\in C^2(U)$ solve the nonlinear, first-order PDE (13) in U. Assume $x(\cdot)$ solves the ODE (20), where $p(\cdot)=Du(x(\cdot))$, $z(\cdot)=u(x(\cdot))$. Then for s such that $x(s)\in U$*

$$\dot{p}(s)=-D_xF(p(s),z(s),x(s))-D_zF(p(s),z(s),x(s))p(s) \quad (24)$$
$$\dot{z}(s)=D_pF(p(s),z(s),x(s))\cdot p(s) \quad (25)$$
$$\dot{x}(s)=D_pF(p(s),z(s),x(s)) \quad (26)$$

Furthermore,

$$F(p(s),z(s),x(s))=0. \quad (27)$$

Example 1. Consider the fully nonlinear problem

$$\begin{aligned} u_{x_1}u_{x_2}&=u \quad \text{in} \quad U\\ u&=x_2^2 \quad \text{in} \quad \Gamma \end{aligned} \quad (28)$$

where $U=\{x_1>0\}$, $\Gamma=\{x_1=0\}=\partial U$.
Goal: Suppose u solves (28) and fix $x\in U$. We would like to calculate $u(x)$ by finding some curve lying within U, connecting x with $x^0\in\Gamma$ and along which we can compute u. Since

$u = g$ we know u at x^0. We hope then to calculate u all along the curve, and so in particular at x.

Here $F(p, z, x) = p_1 p_2 - z$, and hence the characteristics ODE become

$$\begin{aligned} \dot{p}^1 = p^1, \dot{p}^2 = p^2 \\ \dot{z} = 2p^1 p^2 \\ \dot{x}^1 = p^2, \dot{x}^2 = p^1 \end{aligned} \tag{29}$$

We integrate the equations, where (x_1^0, x_2^0) with $x_1^0 = 0$ and $x_0^2 = x^0 \in \mathbb{R}$ is a point in ∂U. Then we obtain

$$\begin{aligned} p^1(s) &= p_1^0 e^s,\ p^2(s) = p_2^0 e^s, \\ z(s) &= z^0 + p_1^0 p_2^0 (e^{2s} - 1) \\ x^1(s) &= \underbrace{x_1^0}_{=0} + p_2^0(e^s - 1),\ x^2(s) = x^0 + p_1^0(e^s - 1), \end{aligned}$$

where $s \in \mathbb{R}$ and $z^0 = (x^0)^2$. We must determine $p^0 = (p_1^0, p_2^0)$. Since $u = x_2^2$ on Γ, $p_2^0 = u_{x_2}(0, x^0) = 2x_0$. Furthermore the PDE $u_{x_1} u_{x_2} = u$ itself implies $p_1^0 p_2^0 = z^0 = (x^0)^2$, and so $p_1^0 = \frac{x^0}{2}$. Consequently the formulas above become

$$\begin{aligned} x^1(s) &= 2x^0(e^s - 1),\ x^2(s) = \frac{x^0}{2}(e^s + 1) \\ z(s) &= (x^0)^2 e^2 s \\ p^1(s) &= \frac{x^0}{2} e^s,\ p^2(s) = 2x^0 e^s \end{aligned}$$

Fix a point $(x_1, x_2) \in U$. Select s and x^0 so that $(x_1, x_2) = (x^1(s), x^2(s)) = (2x^0(e^s - 1), \frac{x^0}{2}(e^s + 1))$. This equality implies $x^0 = \frac{4x_2 - x_1}{4}$, $e^s = \frac{x_1 + 4x_2}{4x_2 - x_1}$; and so

$$u(x) = u(x^1(s), x^2(s)) = z(s) = (x^0)^2 e^{2s} = \frac{(x_1 + 4x_2)^2}{16}$$

Remark 5. In this example we observe that the goal of characteristics method is to find a curve $x(s)$ described by an ODE which initial condition is given from the boundary value of the PDE which is known. For an arbitrary point x in the domain U of the PDE, we find the adequate initial condition and final time describing the curve to go from x^0 in the boundary to x in U along $x(s)$. Then we retieve the value of the solution $u(x)$. The solutions of the ODEs by construction imply that $u(x(s)) = z(s)$

Example 2 (Characteristics for the Hamilton-Jacobi- equation (see [3] pp. 113))**.** We consider the the general Hamilton- Jacobi PDE

$$G(Du, u_t, u, x, t) = u_t + H(Du, x) = 0, \tag{30}$$

where $Du = D_x u = (u_{x_1}, ..., u_{x_n})$. Then writing $q = (p, p_{n+1}), y = (x, t)$, we have

$$G(q, z, y) = p_{n+1} + H(p, x);$$

and so

$$D_qG = (D_pH(p,x),1), D_yG = (D_xH(p,x),0), D_zG = 0$$

Thus equation (26) becomes

$$\dot{x}^i(s) = H_{p_i}(p(s),x(s)), \quad i = 1,...,n \tag{31}$$

$$\dot{x}^{n+1}(s) = 1. \tag{32}$$

In particular we can identify the parameter s with the time t. Equation (24) is

$$\dot{p}^i(s) = -H_{x_i}(p(s),x(s)), \quad i = 1,...,n \tag{33}$$

$$\dot{p}^{n+1}(s) = 0; \tag{34}$$

the equation (25) is

$$\dot{z}(s) = D_pH(p(s),x(s)) \cdot p(s) + p^{n+1}(s) \tag{35}$$

$$= D_pH(p(s),x(s)) \cdot p(s) - H(p(s),x(s)) \tag{36}$$

since $\dot{z}(s) = D_qG \cdot q$ and $u_t = -H(Du,x)$ by (30). In summary, the characteristic equations for the Hamilton Jacobi equation are

$$\dot{p}(s) = -D_xH(p(s),x(s)) \tag{37}$$

$$\dot{z}(s) = D_pH(p(s),x(s)) \cdot p(s) - H(p(s),x(s)) \tag{38}$$

$$\dot{x}(s) = D_pH(p(s),x(s)) \tag{39}$$

for $p(\cdot) = (p^1(\cdot),...,p^n(\cdot)), z(\cdot)$ and $x(\cdot) = (x^1(\cdot),...,x^n(\cdot))$. The first and third equalities,

$$\dot{x} = D_pH(p,x) \tag{40}$$

$$\dot{p} = -D_xH(p,x) \tag{41}$$

are called *Hamilton's equations.*

3 The Deterministic Optimal Control Setting

A mathematical setting for optimality controlling the solution to a deterministic ordinary differential equation

$$\begin{aligned} \dot{X}^s &= f(X^s,\alpha^s), \quad t < s < T \\ X^t &= x \end{aligned} \tag{42}$$

is to minimize

$$\int_t^T h(X^s,\alpha^s)ds + g(X^T) \tag{43}$$

for given *cost function* $h : \mathbb{R}^d \times [t,T] \to \mathbb{R}^d$ and $g : \mathbb{R}^d \to \mathbb{R}$ and a given set of *control functions* $\mathcal{A} = \{\alpha : [t,T] \to A\}$ and *flux* $f : \mathbb{R}^d \times A \to \mathbb{R}^d$. Here $A \subset \mathbb{R}^m$ is compact.

3.1 Approximation of Optimal Control

Optimal control problems can be solved by the following approaches:

Dynamic Programming: This approach uses the value function, defined by

$$u(x,t) := \inf_{\alpha\in\mathcal{A}} \left(\int_t^T h(X^s, \alpha^s) ds + g(X^T) \right) \tag{44}$$

for the ODE (42) with $X^t \in \mathbb{R}^d$, and leads to solution of a non linear *Hamilton-Jacobi-Bellman* partial differential equation

$$\begin{aligned} &\partial_t u(x,t) + \underbrace{\min_{\alpha\in A} \left(f(x,\alpha) \cdot \partial_x u(x,t) + h(x,\alpha) \right)}_{H(\partial_x u(x,t),x)} = 0, \quad t < T, \\ &u(\cdot, T) = g, \end{aligned} \tag{45}$$

in $(x,t) \in \mathbb{R}^d \times \mathbb{R}_+$.

Lagrange Principle: This method seeks for a minimum of the cost with the dynamics as a constrain and leads to the solution of a Hamiltonian system of ordinary differential equations, which are the characteristics of the Hamilton-Jacobi-Bellman equation

$$X'^t = f(X^t, \alpha^t), \quad X_0 \text{ given}, \tag{46}$$

$$-\lambda_i'^t = \partial_{x_i} f(X^t, \alpha^t) \cdot \lambda^t + \partial_{x_i} h(X^t, \alpha^t), \quad \lambda^T = g'(X^T), \tag{47}$$

$$\alpha^t \in \text{argmin}_{a\in A} \left(\lambda^t \cdot f(X^t, a) + h(X^t, a) \right) \tag{48}$$

based on the Pontryagin Principle.

3.2 Lagrange Formulation

Assume $F : \mathbb{R}^d \times \mathbb{R}^n \to \mathbb{R}$ is a differentiable function. Then our goal is to find the minimum $\min_{x\in A} F(x, g(x))$ for a given differentiable function $g : \mathbb{R}^d \to \mathbb{R}^n$ and a compact set $A \subset \mathbb{R}^d$. This problem leads to the usual necessary condition for an interior point

$$\frac{d}{dx} F(x, g(x)) = \partial_x F(x, g(x)) + \partial_y F(x, g(x)) \partial_x g(x) = 0. \tag{49}$$

An alternative method to find the solution is to introduce the *Lagrangian function*

$$\mathcal{L}(\lambda, y, x) := F(x, y) + \lambda \cdot (y - g(x)) \tag{50}$$

with the Lagrange multiplier $\lambda \in \mathbb{R}^n$ and choose λ appropiately to write the necessary conditions for an interior minimum

$$0 = \partial_\lambda \mathcal{L}(\lambda, y, x) = y - g(x) \tag{51}$$

$$0 = \partial_y \mathcal{L}(\lambda, y, x) = \partial_y F(x, y) + \lambda \tag{52}$$

$$0 = \partial_x \mathcal{L}(\lambda, y, x) = \partial_x F(x, y) - \lambda \cdot \partial_x g(x). \tag{53}$$

Equation (51) is the constraint. The equation (52) determines the multiplier $\lambda = -\partial_y F(x,y)$. The equation (53) yields for this multiplier

$$0 = \partial_x \mathcal{L}(-\partial_y F(x,y), y, x) = \partial_x F(x,y) + \partial_y F(x,y) \cdot \partial_x g(x) = \frac{d}{dx} F(x, g(x)),$$

that is, the multiplier is chosen precisely so that the partial derivative with respect to x of the Lagrangian is the total derivative of the objective function $F(x, g(x))$, which is equal to 0 to be minimized.

Remark 6. This Lagrange principle is often practical to use when the constraint is given implicity, e.g. as $G(x,y) = 0$, with a differentiable $G : \mathbb{R}^d \times \mathbb{R}^n \to \mathbb{R}^n$; then the condition $\det \partial_y G(x,y) \neq 0$ in the implicit function theorem implies that the function $y(x)$ is well defined and satisfies $G(x, y(x)) = 0$ and that $\partial_x y = -\partial_y G(x,y)^{-1} \partial_x G(x,y)$, so the Lagrange multiplier method works.

The Lagrange principle for the optimal control problem (42)-(156), to minimize the cost with the dynamics as a constraint, leads to the Lagrangian

$$\mathcal{L}(\lambda, X, \alpha) := g(X^T) + \int_0^T h(X^s, \alpha^s) ds + \int_0^T \lambda^s \cdot (f(X^s, \alpha^s) - \dot{X}) ds \tag{54}$$

with a Lagrange multiplier function $\lambda : [0,T] \to \mathbb{R}^d$. Differentiability of the Lagrangianm leads to the necessary conditions for a constrained interior point.

$$\partial_\lambda \mathcal{L}(X, \lambda, \alpha) = 0, \tag{55}$$
$$\partial_X \mathcal{L}(X, \lambda, \alpha) = 0, \tag{56}$$
$$\partial_\alpha \mathcal{L}(X, \lambda, \alpha) = 0. \tag{57}$$

Our next step is to verify that the two first equations (55),(56) are the same as (46),(47), and that the last equation is implied by the stronger Pontryagin principle in the equation (48). We will later use the Hamilton-Jacobi equation in the dynamic programming approach to verify the Pontryagin principle.

The first equation. Choose a real valued function $v : [0,T] \to \mathbb{R}^d$ and define the function $L : \mathbb{R} \to \mathbb{R}$ by

$$L(\varepsilon) := \mathcal{L}(X, \lambda + \varepsilon v, \alpha).$$

Then equation (55) means that $L'(0) = \frac{d}{d\varepsilon} \mathcal{L}(X, \lambda + \varepsilon v, \alpha)|_{\varepsilon=0} = 0$, which implies that

$$0 = \int_0^T v^s \cdot (f(X^s, \alpha^s) - \dot{X}^s) ds \tag{58}$$

for any continuous function v. If we assume that $f(X^s, \alpha^s) - \dot{X}^s$ is continuous we obtain $f(X^s, \alpha^s) - \dot{X}^s = 0$: By contradiction, suppose that $\beta^s := f(X^s, \alpha^s) - \dot{X}^s \neq 0$ for some s_0, then by the continuity of β, there is interval $[a,b]$ where β is positive (negative also works); by choosing

$$v^s = \begin{cases} \beta^{s_0}, & s \in [a,b], \\ 0, & s \notin [a,b], \end{cases}$$

we get

$$\int_0^T v^s \cdot \beta^s \, ds \;=\; \int_a^b \beta^{s_0} \cdot \beta^s \, ds \;>\; 0,$$

then we conclude that β is zero everywhere and derive equation (46).

The second equation. The next equation $\frac{d}{d\epsilon}\mathcal{L}(X + \epsilon v, \lambda, \alpha)\Big|_{\epsilon=0} = 0$ needs $v^0 = 0$ by the initial condition on X^0 and leads by integration by parts to

$$\int_0^T \lambda^s \cdot \dot{v}^s ds = \lambda^T v^T - \lambda^0 v^0 - \int_0^T \dot{\lambda}^s \cdot v^s ds$$

$$\begin{aligned}
0 &= \int_0^T \lambda^s \cdot \big(\partial_{X_i} f(X^s, \alpha^s) v_i^s - \dot{v}^s\big) + \partial_{X_i} h(X^s, \alpha^s) v_i^s \,\mathrm{d}s + \partial_{X_i} g(X^T) v_i^T \\
&= \int_0^T \lambda^s \cdot \partial_{X_i} f(X^s, \alpha^s) v_i^s + \dot{\lambda}^s \cdot v^s + \partial_{X_i} h(X^s, \alpha^s) v_i^s \,\mathrm{d}s \\
&\quad + \lambda^0 \cdot \underbrace{v^0}_{=0} - (\lambda^T - \partial_X g(X^T)) \cdot v^T \\
&= \int_0^T \big(\partial_X f^*(X^s, \alpha^s)\lambda^s + \dot{\lambda}^s + \partial_X h(X^s, \alpha^s)\big) \cdot v^s \,\mathrm{d}s \\
&\quad - \big(\lambda^T - \partial_X g(X^T)\big) \cdot v^T,
\end{aligned}$$

Choose now the function v to be zero outside an interior interval where possibly $\partial_X f^*(X^s, \alpha^s)\lambda^s + \dot{\lambda}^s + \partial_X h(X^s, \alpha^s)$ is non zero, so that in particular $v^T = 0$. We see then that in fact $\partial_X f^*(X^s, \alpha^s)\lambda^s + \dot{\lambda}^s + \partial_X h(X^s, \alpha^s)$ must be zero (as for the first equation) and we obtain equation (47). Since the integral in the right hand side vanishes, varying v^T shows that the final condition for the Lagrange multiplier $\lambda^T - \partial_X g(X^T) = 0$ also holds.

The third equation. Equation (57) implies as above that for any function $v(t)$ compactly supported in A

$$0 = \int_0^T \lambda^s \cdot \partial_\alpha f(X^s, \alpha^s) v + \partial_\alpha h(X^s, \alpha^s) v \,\mathrm{d}s$$

which yields

$$\lambda^s \cdot \partial_\alpha f(X^s, \alpha^s) + \partial_\alpha h(X^s, \alpha^s) = 0 \tag{59}$$

in the interior $\alpha \in A - \partial A$ minimum point (X, λ, α). The last equation (48) is a stronger condition: it says that α is a minimizer of $\lambda^s \cdot f(X^s, a) + h(X^s, a) = 0$ with respect to $a \in A$, which clearly implies (59) for interior points $\alpha \in A - \partial A$. To derive the Pontryagin principle we will use dynamic programming and the Hamilton-Jacobi-Bellman equation which is the subject of the next section.

3.3 Dynamic Programming and the Hamilton-Jacobi-Bellman Equation

We first state important considerations that allows to have a good understanding of the Dynamic Programming method:

$$\begin{aligned} &\dot{X}^s = f(X^s, \alpha^s), \quad t < s < T \\ &X^t = x \end{aligned} \tag{60}$$

$$\int_t^T h(X^s, \alpha^s) ds + g(X^T) \tag{61}$$

1. The initial condition in our system (60) is given in our problem, it is a fixed parameter.
2. The trajectory $X(\cdot) = X^{\alpha(\cdot)}(\cdot)$ of the solution X depends on the control $\alpha(\cdot)$. That is, for different controls α_1, α_2 we get different paths X_1, X_2 as solutions of (60). The goal is to find an optimal α^* such that minimize the cost (61).
3. The final state X^T is not prescribed and it depends on the initial condition x and control α.

The dynamic programming is based on the idea to track the optimal solution backwards: at the final time the value function in (44) is given by

$$u(X^T = x, T) = g(x) \tag{62}$$

We then observe that

$$u(x,t) = \inf_{\alpha \in \mathcal{A}} \left(\int_t^T h(X^s, \alpha^s) ds + g(X^T) \right) = \inf_{\alpha \in \mathcal{A}} \left(\int_t^T h(X^s, \alpha^s) ds + u(X^T, T) \right)$$

Assume for simplicity first that $h = 0$, then any path $X : [t, t+\Delta t] \to \mathbb{R}^d$ starting in $X^t = x$ will satisfy

$$u(x,t) = \inf_{\alpha:[t,t+\Delta t]\to A} u(X^{t+\Delta t}, t + \Delta t) \tag{63}$$

(a) As we mention in 2 and 3 above, $X^{t+\Delta t}$, the state at time $t+\Delta$, depends on the control, and is different for different controls $\alpha : [t, t+\Delta] \to A$.

(b) In the value

$$u(X^{t+\Delta t}, t+\Delta t) = \inf_{\alpha:[t+\Delta t,T]\to A} g(X^T), \quad (h=0)$$

X^T depends on controls $\alpha : [t+\Delta t, T] \to A$ and initial condition $X^{t+\Delta t}$.

(c) In words, the minimum cost starting at $X^t = x$ is equal to the minimum cost from x to $t + \Delta t$ of the minimum cost from $t + \Delta t$ to T.

(d) Observe that the right hand side in (63) only takes the infimum among optimal costs from $t + \Delta t$ to T. This is true because we are assuming $h = 0$, so we only need to optimize in $g(X^T)$ as stated in item (b).

so that, for a path $X^\cdot$ with $X^t = x$

$$\begin{aligned} u(X^{t+\Delta t}, t+\Delta t) &\geq u(x,t) \\ u(X^{t+\Delta t}, t+\Delta t) - u(X^t, t) &\geq 0. \end{aligned} \tag{64}$$

If u is differentiable, dividing (64) by Δt and taking the limit we can get

$$du(X^t, t) = (\partial_t u(X^t, t) + \partial_x u(X^t, t) \cdot f(X^t, \alpha^t))dt \geq 0 \tag{65}$$

If the minimun is attained, then an optimal path X_*^t exists, with control α_*^t. Thus (64) is equal to zero and

$$du(X_*^t, t) = (\partial_t u(X_*^t, t) + \partial_x u(X_*^t, t) \cdot f(X_*^t, \alpha_*^t))dt = 0 \tag{66}$$

The combination of (65) and (66) implies that

$$\partial_t u(x,t) + \min_{\alpha \in A} (\partial_x u(x,t) \cdot f(x,\alpha)) = 0, \quad t < T \tag{67}$$

$$u(\cdot, T) = g, \tag{68}$$

which is the Hamilton-Jacobi-Bellam equation in the special case $h = 0$.

The case with h nonzero follows similarly by nothing that now

$$u(x,t) = \inf_{\alpha:[t,t+\Delta t] \to A} \left(\int_t^{t+\Delta t} h(X^s, \alpha^s) ds + u(X^{t+\Delta t}, t+\Delta t) \right) \tag{69}$$

which for differentiable u implies the Hamiltonian-Jacobi-Bellman equation

$$\begin{aligned} &\partial_t u(x,t) + \underbrace{\min_{\alpha \in A} (f(x,\alpha) \cdot \partial_x u(x,t) + h(x,\alpha))}_{:= H(\partial_x u(x,t), x)} = 0, \quad t < T, \\ &u(\cdot, T) = g, \end{aligned} \tag{70}$$

3.4 Characteristics and the Pontryagin Principle

We want to relate the Lagrange multiplier method with the Pontryagin principle to the Hamilton-Jacobi-Bellman equation using characteristics. The next theorem shows that the characteristics of the Hamilton-Jacobi equation is a Hamiltonian system.

Theorem 3. *Assume $u \in \mathcal{C}^2$, $H \in \mathcal{C}^1$ and*

$$\dot{X}^t = \partial_\lambda H(\lambda^t, X^t) \quad (\textit{see Theorem 2 and Example 2})$$

with $\lambda^t := \partial_x u(X^t, t)$. Then the characteristics (X^t, λ^t) satisfy the Hamiltonian system

$$\begin{aligned} \dot{X}^t &= \partial_\lambda H(\lambda^t, X^t) \\ \dot{\lambda}^t &= -\partial_X H(\lambda^t, X^t) \end{aligned} \tag{71}$$

Proof. The goal is to verify that the assumption on $\dot{X}^t$ implies that λ has the dynamics (71). We first denote $\lambda_k^t = \partial_{x_k} u(X^t, t)$, then

$$\dot{\lambda}_k^t = \frac{d}{dt}\partial_{x_k} u(X^t,t) = \partial_t \partial_{x_k} u(X^t,t) + \sum_j \partial_{x_j}\partial_{x_k} u(X^t,t)\cdot \dot{X}_j^t \tag{72}$$

Now, we differentiate the Hamilton-Jacobi-Bellman equation with respect to x_k, obtaining

$$\partial_{x_k}\partial_t u(x,t) + \sum_j \partial_{\lambda_j} H(\partial_x u(x,t),x)\underbrace{\partial_{x_k}\partial_{x_j} u(x,t)}_{=\partial_{x_j}\partial_{x_k} u} + \partial_{x_k} H(\partial_x u(x,t),x) = 0 \tag{73}$$

The expression (73) along the path (X^t,t) yields

$$\partial_{x_k}\partial_t u(X^t,t) + \sum_j \partial_{\lambda_j} H(\partial_x u(X^t,t),X^t)\underbrace{\partial_{x_k}\partial_{x_j} u(X^t,t)}_{=\partial_{x_j}\partial_{x_k} u} + \partial_{x_k} H(\partial_x u(X^t,t),X^t) = 0 \tag{74}$$

From $\dot{X}^t = \partial_\lambda H(\lambda^t, X^t)$ in (72), we get from (74)

$$\dot{\lambda}_{x_k}^t + \partial_{x_k} H(\partial_x u(X^t,t),X^t) = 0, \tag{75}$$

which is precisely $\dot{\lambda}^t + \partial_x u(\lambda^t, X^t) = 0$. □

The next step is to relate the characteristics X^t, λ^t to the solution of the Lagrange principle (46). But note first that the Hamiltonian H in general is not differentiable even if f and h are very regular.

Example 3. For the admisible set of controls $A = [-1,1]$, we consider $\dot{X}^t = f(X^t)$ and $h(x,\alpha) = x\alpha$, then

$$\begin{aligned} H(\lambda,x) &= \min_{\alpha\in[-1,1]}(\lambda f(x) + x\alpha) && (76)\\ &= \lambda f(x) + \min_{\alpha\in[-1,1]} x\alpha && (77)\\ &= \lambda f(x) - |x|, && (78)\end{aligned}$$

which is only Lipschitz continuous, namely

$$|H(\lambda,x) - H(\lambda,y)| \leq K|x-y|, \quad \text{with} \quad K = 1 + ||\lambda\cdot\partial_x f(\cdot)||_\infty.$$

In fact, if f and h are bounded differentiable functions the Hamiltonian will always be Lipschitz continuous satisfying

$$|H(\lambda,x) - H(\nu,y)| \leq K(|\lambda-\nu| + |x-y|).$$

Theorem 4. *Assume that f, h are x-differentiable in (x,α_*) and a control α_* is optimal for a point (x,λ), i.e.,*

$$\lambda\cdot f(x,\alpha_*) + h(x,\alpha_*) = H(\lambda,x), \tag{79}$$

and suppose also that H is differentiable in the point or that α_* is unique. Then

$$\begin{aligned} f(x,\alpha_*) &= \partial_\lambda H(\lambda,x),\\ \lambda\cdot\partial_{x_i} f(x,\alpha_*) + \partial_{x_i} h(x,\alpha_*) &= \partial_{x_i} H(\lambda,x). \end{aligned} \tag{80}$$

Proof. We have for any $w, v \in \mathbb{R}^d$

$$H(\lambda+w, x+v) = \min_{\alpha}\{(\lambda+w)\cdot f(x+v,\alpha)+h(x+v,\alpha)\} \le (\lambda+w)\cdot f(x+v,\alpha_*)+h(x+v,\alpha_*),$$

then

$$\begin{aligned} H(\lambda+w, x+v) - H(\lambda, x) \le &(\lambda+w)\cdot f(x+v,\alpha_*) + h(x+v,\alpha_*) \\ &- \lambda\cdot f(x,\alpha_*) - h(x,\alpha_*). \end{aligned}$$

For f and h we consider the Taylor expansions

$$\begin{aligned} f(x+v,\alpha_*) &= f(x,\alpha_*) + \partial_x f(x,\alpha_*)\cdot v + o(|v|) \\ &= f(x,\alpha_*) + \sum_{i=1}^{d} \partial_{x_i} f(x,\alpha_*) v_i + o(|v|) \\ h(x+v,\alpha_*) &= h(x,\alpha_*) + \partial_x h(x,\alpha_*)\cdot v + o(|v|) \\ &= h(x,\alpha_*) + \sum_{i=1}^{d} \partial_{x_i} h(x,\alpha_*) v_i + o(|v|). \end{aligned}$$

Then we obtain

$$H(\lambda+w, x+v) - H(\lambda, x) \le w\cdot f(x,\alpha_*) + \sum_{i=1}^{d} (\lambda\cdot\partial_{x_i} f + \partial_{x_i} h) v_i + o(|v|+|w|)$$

Using the last inequality, we obtain the first identity in (80) by taking $v = 0$ and $w \in \mathbb{R}^d$. The second identity can be obtained similarly by letting $w = 0$ and $v \in \mathbb{R}^d$. □

This Theorem shows that the Hamiltonian system (71) is the same as the system (46), given by the Lagrange principle using the optimqal control α_* with the Pontryagin principle

$$\lambda\cdot f(x,\alpha_*) + h(x,\alpha_*) = \inf_{\alpha\in A}(\lambda\cdot f(x,\alpha) + h(x,\alpha)) =: H(\lambda, x)$$

Remark 7. If α_* is not unique (i.e not a single popint) the proof shows that (80) still holds for the optimal controls, so that $\partial_\lambda H, \partial_x H$ become set valued. We conclude that non unique local controls α_* is the phenomenom that makes the Hamiltonian non differentiable in certain points. In particular a differentiable Hamiltonian gives unique optimal control fluxes $\partial_\lambda H, \partial_x H$, even if α_* is not a single point. If the Hamiltonian can be explicity formulated, it is therefore often practical to use the Hamiltonian system formulation with the variables X and λ, avoiding the control variate.

Remark 8. Clearly, the Hamiltonian needs to be differentiable for the Hamiltonian system to make sense; in fact its flux $(\partial_\lambda H, -\partial_x H)$ must be Lipschitz continuous to give well posedness (System (71) requires Lipschitz condition for well posedness (ODE)). On the other hand, we shall see that the Hamilton-Jacobi-Bellman formulation based on dynamic programming, leads to non differentiable value functions u, so that, classical solutions lack well posedness. This is problematic for both aproaches, HJB equation and the Hamilton

ODE system (Well posedness of the Hamiltonian ODE system is described by the variations of the Hamiltonian at optimal controls). Candall-Lions-Evans formulated a complete well posedness theory for generalized so called viscosity solutions to Hamilton-Jacobi PDE, allowing Lipschitz continuous Hamiltonians. This theory proivides good theoretical foundation also for non smooth controls. In particular this theory removes one of the Pontryagin's two reasons, but no the other to favor the approach (46) and (71): the mathematical theory of viscosity solutions handles elegantly the non smoothness in control problems. We will use an alternative ODE method to solve the optimal control problems numerically based on regularized Hamiltonians, where we approximate the Hamiltonian with a two times differentiable Hamiltonian.

Example 4 (Classical solutions exists for short time in general)**.** The Hamilton-Jacobi equation

$$\partial_t u - \frac{1}{2}(\partial_x u)^2 = 0 \tag{81}$$

with

$$H(x, \lambda) = -\frac{1}{2}\lambda^2$$

has the characteristics

$$\begin{aligned} \dot{X}^t &= -\lambda^t \\ \dot{\lambda}^t &= 0, \end{aligned}$$

If the initial data $u(\cdot, T)$ is a concave function (e.g. a smooth version of $-|x|$) then the characteristics X will collide. We can understand this precisely by studying blow-up of the derivative w of $\partial_x u =: v$; since differentiating w.r.t x in (81), v satisfies

$$\partial_t v - \frac{1}{2}\partial_x(v^2) = \partial_t v - v\partial_x v = 0 \tag{82}$$

We have for $w = \partial_x v$ by x–differentiation in (82)

$$\underbrace{\partial_t w - v\partial_x w}_{\frac{d}{dt}w(X^t,t)} - w^2 = 0, \quad \dot{X}^t = -\lambda^t = -\partial_x u(X^t, t) \tag{83}$$

which reduces to the ordinary differential equation for $z^t := w(X^t, t)$

$$\frac{d}{dt}z(t) = z^2(t) \tag{84}$$

Its separation of variables solution $dz/z^2 = dt$ yields $-1/z^t = t + C$. The constant becomes $C = -T - 1/z^T$, so that

$$z^t = \frac{1}{t - T - 1/z^T}$$

blows up to infinity at time $T - t = 1/z^T$. For instance if $z^T = -10$, the time to blow-up time is $1/10$.

(a) [Concavity $\Rightarrow$ finite-time collision of characteristics] We label characteristics by their terminal point y:

$$X(t;y) = y - \lambda(y)(t-T), \qquad \lambda(y) = v(T,y) = u_x^T(y).$$

Then

$$\partial_y X(t;y) = 1 - (t-T)\lambda'(y) = 1 - (t-T)u_{xx}^T(y).$$

If u^T is concave, $u_{xx}^T \le 0$ and at some y_0 we have $u_{xx}^T(y_0) < 0$. Since $t < T$ implies $(t-T) < 0$, there is a finite time

$$t_* \;=\; T + \frac{1}{u_{xx}^T(y_0)} \;<\; T$$

at which $\partial_y X(t_*;y_0) = 0$. Hence the map $y \mapsto X(t_*;y)$ loses invertibility, so two distinct labels $y_1 \neq y_2$ satisfy $X(t_*;y_1) = X(t_*;y_2)$: characteristics collide. Equivalently, with $w := u_{xx}$, along characteristics $z(t) := w(X(t;y_0),t)$ solves $\dot z = z^2$, $z(T) = u_{xx}^T(y_0) < 0$, and blows up at the same t_*. Therefore a classical solution exists only for the short time interval $(t_*,\, T]$.

(b) A classical solution requires $u_x = v$ to be continuous (and $u_{xx} = w$ finite). But at t_* we have both: w becomes unbounded and v becomes discontinuous. Either failure already prevents u from being classical.

3.5 Genralized Viscosity Solutions of Hamilton-JUacobi Equations

Example 4 shows that Hamilton-Jacobi equations do in general not have global classical solutions – after finite time the derivative can become infinitely large even with smooth initial data and a smooth Hamiltonian. Therefore a more general solution concept is needed. We shall describe the so called viscosity solutions introduced by Crandall and Lions in [9], which can be characterised by the limit of viscous approximations u^ϵ satisfying for $\epsilon > 0$

$$\partial_t u^\epsilon(x,t) + H\big(\partial_x u^\epsilon(x,t), x\big) + \epsilon \partial_{xx} u^\epsilon(x,t) = 0 \quad t < T \tag{85}$$

$$u^\epsilon(\cdot, T) = g. \tag{86}$$

The function u^ϵ is also a value function, now for the stochastic optimal control problem

$$dX^t = f(X^t, \alpha^t)dt + \sqrt{2\epsilon}\, dW^t \quad t > 0 \tag{87}$$

with the objective to minimize

$$\min_\alpha \mathbb{E}\left[g(X^T) + \int_0^T h(X^t, \alpha^t)dt \,\middle|\, X^0 \text{ given} \right]. \tag{88}$$

over adapted controls $\alpha : [0,T] \to A$, where $W : [0,\infty) \to \mathbb{R}^d$ is the d-dimensional Wiener process with independent components. Here adapted controls means that α_t does not use values of W^s for $s > t$. Section 8.3 shows that the value function for this optimal control problem solves the second order Hamilton-Jacobi equation, that is

$$u^\epsilon(x,t) = \min_\alpha \mathbb{E}\left[g(X^T) + \int_0^T h(X^t, \alpha^t)\, dt \,\middle|\, X^t = x \right]. \tag{89}$$

Theorem 5 (Crandall-Lions)**.** *Assume f, h and g are Lipschitz continuous and bounded, then the limit $\lim_{\epsilon\to 0+} u^\epsilon$ exists. This limit is called the viscosity solution of the Hamilton-Jacobi equation*

$$\begin{aligned} \partial_t u(x,t) + H(\partial_x u(x,t), x) = 0 \quad t < T \\ u(\cdot, T) = g. \end{aligned} \tag{90}$$

There are several equivalent ways to describe the viscosity solution directly without using viscous or stochastic approximations. We shall use the one based on sub and super differentials presented first in . To simplify the notation introduce first the space-time coordinate $y = (x,t)$, the space-time gradient $p = (p_x, p_t) \in \mathbb{R}^{d+1}$ (related to $(\partial_x u(y), \partial_t u(y))$) and write the Hamilton-Jacobi operator $F(p,y) := p_t + H(p_x, x)$.

Definition 6. For a bounded uniformly continuous function $v : \mathbb{R}^d \times [0,T] \to \mathbb{R}$ define for each space-time point y its sub differential set

$$D^- v(y) = \{p \in \mathbb{R}^{d+1} : \liminf_{z\to 0} |z|^{-1}\big(v(y+z) - v(y) - p\cdot z\big) \geq 0\} \tag{91}$$

and its super differential set

$$D^+ v(y) = \{p \in \mathbb{R}^{d+1} : \limsup_{z\to 0} |z|^{-1}\big(v(y+z) - v(y) - p\cdot z\big) \leq 0\}. \tag{92}$$

Remark 9. The above deffinitions corresponds to viscosity Frechet sub/superdifferentials. They are local notions, unlike the usual sub/superdifferential sets that are global notions. In specific cases they coincide. The two sets above always exist (one may be empty). If v is differentiable, that is when

$$D^- v(y) = D^+ v(y) = \{p\} \iff v(y+z) - v(y) - p\cdot z = o(z). \tag{93}$$

Example 5. Let $u(x) = -|x|$, then

$$D^+ u(x) = D^- u(x) = \{-\operatorname{sgn}(x)\}, \qquad x \neq 0 \tag{94}$$

$$D^- u(0) = \varnothing, \qquad x = 0 \tag{95}$$

$$D^+ u(0) = [-1, 1], \qquad x = 0 \tag{96}$$

Definition 7 (Viscosity solution)**.** A bounded uniformly continuous function u is a viscosity solution to (90) if $u(\cdot, T) = g$ and for each point $y = (x,t)$

$$F(p,y) \geq 0 \quad \text{for all } p \in D^+ u(y), \tag{97}$$

$$F(p,y) \leq 0 \quad \text{for all } p \in D^- u(y). \tag{98}$$

Theorem 6. *The first variation of the value function is in the superdifferential.*

Theorem 7. *The value function is semi-concave, that is for any point (x,t) either the value function is differentiable or the sub differential is empty (i.e. $D^- u(x,t) = \varnothing$ and $D^+ u(x,t)$ is non empty).*

Theorem 8. *The value function is a viscosity solution.*

Theorem 9. *Bounded uniformly conitnuous viscosity solutions are unique.*

Example 6. Consider the function $u(x,t) = -|x|$. We have from Example 5

$$D^+u(x,t) = \begin{cases} (-\operatorname{sgn}(x), 0), & x \neq 0, \\ ([-1,1], 0), & x = 0 \end{cases} \tag{99}$$

and

$$D^-u(x,t) = \begin{cases} (-\operatorname{sgn}(x), 0), & x \neq 0, \\ \varnothing, & x = 0. \end{cases} \tag{100}$$

Consequently for $H(\lambda, x) := (1 - |\lambda|^2)/2$ we obtain

$$p_t + H(p_x, x) \geq 0 \qquad q \in D^+u(x,t), \tag{101}$$

$$p_t + H(p_x, x) = 0 \qquad q \in D^-u(x,t), \tag{102}$$

so that $-|x|$ is a viscosity solution to $\partial_t u + H(\partial_x u, x) = 0$.

Similarly the function $u(x,t) = |x|$ satisfies

$$D^-u(x,t) = \begin{cases} (\operatorname{sgn}(x), 0), & x \neq 0, \\ ([-1,1], 0), & x = 0 \end{cases} \tag{103}$$

and therefore

$$p_t + H(p_x, 0) > 0 \quad \text{for } q \in (-1,1) \subset D^-u(0,t), \tag{104}$$

so that $|x|$ is not a viscosity solution to $\partial_t u + H(\partial_x u, x) = 0$.

3.5.1 The Pontryagin Principle for Generalized Solutions

Assume that X_* and α_* is an optimal control solution. Let

$$-\dot{\lambda}_*^t = \partial_X f(X_*^t, \alpha_*^t)\lambda_*^t + \partial_X h(X_*^t, \alpha_*^t), \quad t < T, \tag{105}$$

$$\lambda_*^T = g'(X_*^T). \tag{106}$$

The proof of Theorem 6 shows first that

$$\big(\lambda_*^t, -\big(\lambda_*^t \cdot f(X_*^t, \alpha_*^t) + h(X_*^t, \alpha_*^t)\big)\big)$$

is the first variation in x and t of the value function at the point (X^t, t) and concludes then that the first variation is in the superdifferential, that is

$$\big(\lambda_*^t, -\big(\lambda_*^t \cdot f(X_*^t, \alpha_*^t) + h(X_*^t, \alpha_*^t)\big)\big) \in D^+u(X_*^t, t). \tag{107}$$

Since the value function is a viscosity solution we conclude that

$$-\big(\lambda_*^t \cdot f(X_*^t, \alpha_*^t) + h(X_*^t, \alpha_*^t)\big) + \underbrace{H\big(\lambda_*^t, x\big)}_{\min_{\alpha \in A}(\lambda_*^t \cdot f(X_*^t, \alpha_*^t) + h(X_*^t, \alpha_*^t))} \geq 0 \tag{108}$$

which means that α_* satisfies the Pontryagin principle also in the case of non differentiable solutions to Hamilton-Jacobi equations.

3.6 Maximum Norm Stability of Viscosity Solutions

An important aspect of the viscosity solution u of the Hamilton-Jacobi-Bellman equation is its maximum norm stability with respect to maximum norm perturbations of the data (Hamiltonian and the initial data); that is, the value function is stable with respect to perturbations of the flux f and cost functions h and g.

Assume first for simplicity that the optimal control is attained and that the value function u is differentiable for two different optimal control problems with data: f, h, g and the Hamiltonian H, respectively $\tilde{f}, \tilde{h}, \tilde{g}$ and Hamiltonian $\tilde{H}$. The general case with only superdifferentiable value functions is studied afterwards. We have for the special case with the same initial data $\bar{X}^0 = X^0$ and $\bar{g} = g$

$$\underbrace{\int_0^T \bar{h}(\bar{X}^t, \bar{\alpha}^t)\, dt + \bar{g}(\bar{X}^T)}_{\bar{u}(\bar{X}^0, 0)} - \underbrace{\int_0^T h(X^t, \alpha^t)\, dt + g(X^T)}_{u(X^0, 0)} \tag{109}$$

$$= \int_0^T \bar{h}(\bar{X}^t, \bar{\alpha}^t)\, dt + \underbrace{u(\bar{X}^T, T)}_{\bar{g}=g} - \underbrace{u(\bar{X}^0, 0)}_{X^0 = \bar{X}^0} \tag{110}$$

$$= \int_0^T \bar{h}(\bar{X}^t, \bar{\alpha}^t)\, dt + \int_0^T du(\bar{X}^t, t) \tag{111}$$

$$= \int_0^T \underbrace{\partial_t u(\bar{X}^t, t)}_{= -H(\partial_x u(\bar{X}^t, t), \bar{X}^t)} + \underbrace{\partial_x u(\bar{X}^t, t) \cdot \bar{f}(\bar{X}^t, \bar{\alpha}^t) + \bar{h}(\bar{X}^t, \bar{\alpha}^t)}_{\geq \bar{H}(\partial_x u(\bar{X}^t, t), \bar{X}^t)}\, dt \tag{112}$$

$$\geq \int_0^T (\bar{H} - H)(\partial_x u(\bar{X}^t, t), \bar{X}^t)\, dt. \tag{113}$$

The more general case with $\bar{g} \neq g$ yields the additional error term

$$(g - \bar{g})(\bar{X}^T) \tag{114}$$

to the right hand side.

To find an upper bound, repeat the derivation above, replacing u along $\bar{X}^t$ with $\bar{u}$ along

X^t, to obtain

$$\underbrace{\int_0^T h(X^t,\alpha^t)\,dt + g(X^T)}_{u(X^0,0)} - \underbrace{\int_0^T \bar h(\bar X^t,\bar\alpha^t)\,dt + \bar g(\bar X^T)}_{\bar u(\bar X^0,0)} \tag{115}$$

$$= \int_0^T h(X^t,\alpha^t)\,dt + \bar u(X^T,T) - \underbrace{\bar u(X^0,0)}_{\bar u(\bar X^0,0)} \tag{116}$$

$$= \int_0^T h(X^t,\alpha^t)\,dt + \int_0^T d\bar u(X^t,t) \tag{117}$$

$$= \int_0^T \underbrace{\partial_t \bar u(X^t,t)}_{=-\bar H(\partial_x \bar u(X^t,t),X^t)} + \underbrace{\partial_x \bar u(X^t,t)\cdot f(X^t,\alpha^t) + h(X^t,\alpha^t)}_{\geq H(\partial_x \bar u(X^t,t),X^t)}\,dt \tag{118}$$

$$\geq \int_0^T (H-\bar H)(\partial_x \bar u(X^t,t),X^t)\,dt. \tag{119}$$

The two estimates above yield both an upper and a lower bound

$$\begin{aligned}\int_0^T (H-\bar H)(\partial_x \bar u(X^t,t),X^t)\,dt &\leq u(X_0,0) - \bar u(X_0,0)\\ &\leq \int_0^T (H-\bar H)(\partial_x u(\bar X^t,t),\bar X^t)\,dt.\end{aligned} \tag{120}$$

Remark 10. (No minimizers). If there are no minimizers (α, X) and $(\bar\alpha,\bar X)$, then for every $\varepsilon > 0$, we can choose controls $\alpha,\bar\alpha$ with corresponding states $X,\bar X$ such that

$$E_{lhs} - \varepsilon \leq u(X_0,0) - \bar u(X_0,0) \leq E_{rhs} + \varepsilon \tag{121}$$

with E_{lhs}, E_{rhs} being the left and right hand sides of (120).

Solutions to Hamilton–Jacobi equations are in general not differentiable (Example 4). If u is a non differentiable semiconcave solution to a Hamilton–Jacobi equation, from Definition 7,

$$p_t + H(p_x,x) = 0 \qquad \text{for all } (p_t,p_x)\in Du(x,t) \text{ and all } t<T,\ x\in\mathbb{R}^d, \tag{122}$$

$$p_t + H(p_x,x) \geq 0 \qquad \text{for all } (p_t,p_x)\in D^+u(x,t) \text{ and all } t<T,\ x\in\mathbb{R}^d, \tag{123}$$

$$u(\cdot,T) = g. \tag{124}$$

Consider now a point (x,t) where the value function is not differentiable. This means that in (118) we can for each t choose a point $(p_t,p_x)\in D^+u(X^t,t)$ so that

$$\int_0^T du(\bar X^t,t) + \int_0^T \bar h(\bar X^t,\alpha^t)\,dt = \int_0^T \big(p_t + p_x\cdot \bar f(\bar X^t,\alpha^t) + \bar h(\bar X^t,\alpha^t)\big)\,dt \tag{125}$$

$$\geq \int_0^T \big(p_t + \bar H(p_x,\bar X^t)\big)\,dt \geq \int_0^T \big(\bar H - H\big)(p_x,\bar X^t)\,dt. \tag{126}$$

Note that the only difference compared to the differentiable case is the inequality instead of equality in the last step, which uses that optimal control problems have semi-concave viscosity solutions. The analogous formulation holds for $\bar{u}$. Consequently (120) holds for some $(p_t, p_x) \in D^+u(\bar{X}^t, t)$ replacing $(\partial_t u(\bar{X}^t, t), \partial_x u(\bar{X}^t, t))$ and some $(\bar{p}_t, \bar{p}_x) \in D^+\bar{u}(\bar{X}^t, t)$ replacing $(\partial_t \bar{u}(\bar{X}^t, t), \partial_x \bar{u}(\bar{X}^t, t))$, namely

$$
\begin{aligned}
\int_0^T (H - \bar{H})(\bar{p}_x, X^t)\, dt &\leq u(X_0, 0) - \bar{u}(X_0, 0) \\
&\leq \int_0^T (H - \bar{H})(p_x, \bar{X}^t)\, dt.
\end{aligned}
\tag{127}
$$

3.7 Numerical Approximation of ODE Constrained Minimization

We consider numerical approximations with the time steps

$$
t_n = \frac{n}{N}T, \quad n = 0, 1, 2, ..., N. \tag{128}
$$

The most basic approximation is based on the minimization

$$
\min_{\bar{\alpha} \in B^N} \left(g(\bar{X}_N) + \sum_{n=0}^{N-1} h(\bar{X}_n, \bar{\alpha}_n)\Delta t \right) \tag{129}
$$

where $\Delta t = t_{n+1} - t_n$, $\bar{X}_0 = X_0$ and $\bar{X}_n = \bar{X}(t_n)$, for $1 \leq n \leq N$, satisfy the *forward Euler* constraint

$$
\bar{X}_{n+1} = \bar{X}_n + \Delta t f(\bar{X}_n, \bar{\alpha}_n) \tag{130}
$$

The existence of at least one minimum of (129) is clear since it is a minimization of a continuous function in the compact set B^N. The Lagrange principle can be used to solve such a constrained minimization problem. We will focus on a variant of this method based on the *discrete Pontryagin principle* where the control is eliminated

$$
\begin{aligned}
\bar{X}_{n+1} &= \bar{X}_n + \Delta t H_\lambda(\bar{\lambda}_{n+1}, \bar{X}_n), \quad \bar{X}_0 = X_0, \\
\bar{\lambda}_n &= \bar{\lambda}_{n+1} + \Delta t H_x(\bar{\lambda}_{n+1}, \bar{X}_n), \quad \bar{\lambda}_N = g_x(\bar{X}_N)
\end{aligned}
\tag{131}
$$

called the symplectic Euler method for the Hamiltonian system (71). From the Hamiltonian system (71) we derive the condition $\bar{\lambda}_N$.

A natural question is in what sense the discrete problem (131) is an approximation to the continuous optimal control problem (71). In this section we show that the value function of the discrete problem approximates the continuous value function, using the theory of viscosity solutions to Hmilton-Jacobi equations to construct and analyse regularized Hamiltonians.

Our analysis is a kind of backward error analysis more related to the standard finite element analysis. We first extend the approximate Euler solution to a continuous piecewise linear function in time and define a *discrete value function*, $\bar{u} : \mathbb{R}^d \times [0, T] \to \mathbb{R}$. This value function satisfies a perturbed Hamilton-Jacobi partial differential equation, with a small residual error.

Special case:

(a) If the optimal control α in (46) is a differentiable function of x and λ

Equation (46) defines α as

$$\alpha = \text{argmin}_{a \in A} \{\lambda \cdot f(x, a) + h(x, a)\}$$

If the minimun is reached at $\alpha = \alpha(x, \lambda)$, then α can be considered as an implicit function of x and λ. Under regularity conditions on f and h, assumin the optimum α is unique, we could see that $\alpha^*(x, \lambda)$ is differentiable w.r.t x and λ.

(b) If the optimal backward paths, $\bar{X}(s)$ for $s < T$, do not collide

Under (a) and (b), the discrete value function, $\bar{u}$, for the Pontryagin method (131) satisfy the Hamilton-Jacobi equation:

$$\bar{u}_t + H(\bar{u}_x, \cdot) = O(\Delta t), \quad \text{as } \Delta t \to 0^+, \tag{132}$$

where

$$\bar{u}(x, t_m) = \min_{\bar{\alpha} \in B^N} \left(g(\bar{X}_N) + \sum_{n=m}^{N-1} h(\bar{X}_n, \bar{\alpha}_n)\Delta t \right) \tag{133}$$

for solutions $\bar{X}$ to with $\bar{X}(t_m) = \bar{X}_m = x$. The minimum in (133) is taken over the solutions of the discrete Pontryagin principle (131). The maximum norm stability of the Hamilton-Jacobi PDE solutions and a comparison between the two equations (70) and (132) show that

$$O\|u - \bar{u}\|_{\mathcal{C}} = O(\Delta t). \tag{134}$$

General Case: However, in general the optimal controls $\bar{\alpha}$ and α in (130) and (42) are discontinuous functions of x, and $\bar{\lambda}$ or u_x, respectively, and the backward paths do collide. There are two different reasons for discontinuous controls:

- The Hamiltonian is in general only Lipschitz continuous, even if f and h are smooth.
- The optimal backward paths may collide.

The standard error analysis for ODEs is directly applicable to control problems when the time derivative of the control is integrable. General control problems with discontinuous controls require alternative analysis, which will be in two steps:

Step 1

- Construct Regularizations of the functions f and h, based on (80) applied to a $\mathcal{C}^2(\mathbb{R}^d \times \mathbb{R}^d)$ approximate Hamiltonian H^δ which is λ-concave and satisfies

$$\|H^\delta - H\|_{\mathcal{C}} = O(\delta), \quad \text{as} \quad \delta \to 0^+ \tag{135}$$

and to introduce the regularized paths

$$\begin{aligned} \bar{X}_{n+1} &= \bar{X}_n + \Delta t H_\lambda^\delta(\bar{\lambda}_{n+1}, \bar{X}_n), \quad \bar{X}_0 = X_0, \\ \bar{\lambda}_n &= \bar{\lambda}_{n+1} + \Delta t H_x^\delta(\bar{\lambda}_{n+1}, \bar{X}_n), \quad \bar{\lambda}_N = g_x(\bar{X}_N) \end{aligned} \tag{136}$$

We will sometimes use the bnotation $f^\delta = H_\lambda^\delta$ and $h^\delta = H^\delta - \lambda H_\lambda^\delta$. Observe that the solutions of (136) corresponds to the optimal trajectories by Theorem 4.

Step 2

- Estimate the residual of the discrete value function in the HJB equation (70). That is, since we are approximating the value function, we want to know the rhs of the HJB equation. The maximum norm stability of viscosity solutions and the residual estimate imply then an estimate for the error in the value function (similar idea to (134)). An approximation of the form (136) may be viewed as a general symplectic one step method for the Hamiltonian system (71)
- Advantages: There is a second reason to use Hamiltonians with smooth flux: in practice the nonlinear boundary value problem (136) has to be solved by iterations. If the flux is not continuous it seems difficult to construct a convergent iterative method, in any case iterations perform better with smoother solutions. When the Hamiltonian can be formed explicitly, the Pontryagin based method has the advantage that the Newton method can be applied to solve the discrete nonlinear Hamiltonian system with a sparse Jacobian.

If the optimal discrete backward paths $\bar{X}(t)$ in (136) collide on a codimension one surface Γ in $\mathbb{R}^d \times [0,T]$, the dual variable $\bar{\lambda} = \bar{u}_x$ may have a discontinuity at Γ, as a function of x. Theorem 8.28 proves, for $\bar{u}$ based on the Pontryagin method, that in the viscosity solution sense

$$\bar{u}_t + H(\bar{u}_x, \cdot) = \mathcal{O}(\Delta t + \delta + \frac{(\Delta t)^2}{\delta}), \tag{137}$$

where the discrete value function, $\bar{u}$, in (133) has been modified to

$$\bar{u}(x, t_m) = \min_{\bar{X}_m = x} \Big(g(\bar{X}_N) + \sum_{n=m}^{N-1} h^\delta(\bar{X}_n, \bar{\lambda}_{n+1})\Delta t \Big). \tag{8.31}$$

The regularizations make the right hand side in (137) a Lipschitz continuous function of $(\bar{\lambda}(t), \bar{X}(t), t)$, bounded by $C(\Delta t + \delta + \frac{(\Delta t)^2}{\delta})$ where C depends only on the Lipschitz constants of f, h and $\bar{\lambda}$. Therefore the maximum norm stability can be used to prove

$$\|u - \bar{u}\|_C = \mathcal{O}(\Delta t), \quad \text{for } \delta = \Delta t. \tag{138}$$

3.8 Optimization Examples

We give some examples when the Hamiltonian, H, is not a differentiable function, and difficulties associated with this.

Example 7. Let $B = [-1, 1]$, $f = \alpha$, $h = x^2/2$ and $g = 0$, then our optimal control problem becomes

$$\begin{aligned} X'(t) &= \alpha(t), \quad 0 < t < T \\ X(0) &= X_0 \end{aligned} \tag{139}$$

$$u(X_0, 0) = \inf_{\alpha \in [-1,1]} \int_0^T \frac{X^2(s)}{2} ds \tag{140}$$

Note that for any X_0, $u(X_0, 0) \geq 0$. In the simple case $X_0 = 0$, the infimum is obtained with optimal control $\alpha(t) = 0$. Suppose that $X_0 \neq 0$, then we consider the following cases

(a) $X_0 > 0$: We want that $X(t)$ approaches to 0 as fast as possible to avoid accumulating cost $(X^2(t)/2)$. Since $X'(t) = \alpha(t)$ and $\alpha(t) \in [-1, 1]$, the best control to decrease $X(t)$ as fast as allowed is

$$\alpha(t) = -1$$

With this optimal control we obtain

$$X'(t) = -1 \implies X(t) = X_0 - t$$

Observe that $X(t) = 0$ at $t = X_0$ assuming that $X_0 < T$. So for $t > X_0$ we define $\alpha(t) = 0$. Therefore, the optimal control is discontinuous and

$$X(t) = \begin{cases} X_0 - t & \text{if } 0 \le t \le X_0 \\ 0 & \text{if } X_0 < t \le T \end{cases} \tag{141}$$

$$\alpha(t) = \begin{cases} -1 & \text{if } 0 \le t \le X_0 \\ 0 & \text{if } X_0 < t \le T \end{cases} \tag{142}$$

$$\lambda(t) = \begin{cases} \dfrac{(X_0 - t)^2}{2}, & 0 \le t \le X_0, \\ 0, & X_0 \le t \le T. \end{cases} \tag{143}$$

(b) $X_0 < 0$: Similarly as in item (a), the optimal control is given by

$$\alpha(t) = \begin{cases} 1 & \text{if } 0 \le t \le X_0 \\ 0 & \text{if } X_0 < t \le T \end{cases} \tag{144}$$

The Hamiltonian for this problem is

$$H(\lambda, x) = \min_{\alpha \in [-1,1]} \left(\lambda \alpha + \frac{x^2}{2} \right) = \frac{x^2}{2} + \min_{\alpha \in [-1,1]} \lambda \alpha \tag{145}$$

$$= -|\lambda| + \frac{x^2}{2} \tag{146}$$

So, the Hamiltonian is not differentiable w.r.t λ. We construct an approximate Hamiltonian H^δ which is λ-concave, namely

$$H^\delta(\lambda, x) = -\delta \log \left(\cosh \left(\frac{\lambda}{\delta} \right) \right) + \frac{x^2}{2} \tag{147}$$

For the smooth approximation of the λ-derivative, $H^\delta_\lambda(\lambda, x) = -\tanh(\lambda/\delta)$, the symplectic one step method for the Hamiltonian system becomes

$$\begin{aligned} \bar{X}_{n+1} &= \bar{X}_n - \Delta t \tanh(\bar{\lambda}_{n+1}/\delta), \quad \bar{X}_0 = X_0, \\ \bar{\lambda}_n &= \bar{\lambda}_{n+1} + \Delta t \bar{X}_n, \quad \bar{\lambda}_N = g_x(\bar{X}_N) \end{aligned} \tag{148}$$

We use the Newton iterations for the nonlinear system $F(\bar{X},\bar{\lambda})=0$, where

$$
\begin{aligned}
F(\bar{X},\bar{\lambda}) &= \bar{X}_{n+1}-\bar{X}_n+\Delta t\tanh(\bar{\lambda}_{n+1}/\delta),\\
F(\bar{X},\bar{\lambda}) &= \bar{\lambda}_n-\bar{\lambda}_{n+1}-\Delta t\bar{X}_n.
\end{aligned}
\tag{149}
$$

Newton's method starts with an initial guess $\left(\bar{X}^0_{n+1},\bar{\lambda}^0_n\right)$, for all times $n=0,\dots,N-1$, and updates the solution, for some damping factor $\gamma\in(0,1]$, according to

$$
\bar{X}^{i+1}_{n+1}=\bar{X}^i_{n+1}-\gamma\Delta\bar{X}^i_{n+1}, \tag{150}
$$

$$
\bar{\lambda}^{i+1}_n=\bar{\lambda}^i_n-\gamma\Delta\bar{\lambda}^i_n. \tag{151}
$$

We consider the case $N=3$. The values X_0 and $\bar{\lambda}_3=g_x(\bar{X}_3)=0$ are given and our system has the variables

$$
z=\left(\bar{\lambda}_0,\,\bar{X}_1,\,\bar{\lambda}_1,\,\bar{X}_2,\,\bar{\lambda}_2,\,\bar{X}_3\right)^\top\in\mathbb{R}^6. \tag{152}
$$

with

$$
F(z)=\begin{pmatrix}
\bar{\lambda}_0-\bar{\lambda}_1-\Delta t\,\bar{X}_0\\
\bar{X}_1-\bar{X}_0+\Delta t\,\tanh(\bar{\lambda}_1/\delta)\\
\bar{\lambda}_1-\bar{\lambda}_2-\Delta t\,\bar{X}_1\\
\bar{X}_2-\bar{X}_1+\Delta t\,\tanh(\bar{\lambda}_2/\delta)\\
\bar{\lambda}_2-\bar{\lambda}_3-\Delta t\,\bar{X}_2\\
\bar{X}_3-\bar{X}_2+\Delta t\,\tanh(\bar{\lambda}_3/\delta)
\end{pmatrix}. \tag{153}
$$

Thus we obtain the following Newton Iteration system

$$
\begin{pmatrix}
1 & -1 & & & & \\
 & d_1^i\Delta t & 1 & & & \\
 & 1 & -\Delta t & -1 & & \\
 & & -1 & d_2^i\Delta t & 1 & \\
 & & & 1 & -\Delta t & \\
 & & & & -1 & 1
\end{pmatrix}
\begin{pmatrix}
\Delta\bar{\lambda}^i_0\\
\Delta\bar{\lambda}^i_1\\
\Delta\bar{X}^i_1\\
\Delta\bar{\lambda}^i_2\\
\Delta\bar{X}^i_2\\
\Delta\bar{X}^i_3
\end{pmatrix}
=
\begin{pmatrix}
\bar{\lambda}^i_0-\bar{\lambda}^i_1-\Delta t\,\bar{X}^i_0\\
\bar{X}^i_1-\bar{X}^i_0+\Delta t\,\tanh(\bar{\lambda}^i_1/\delta)\\
\bar{\lambda}^i_1-\bar{\lambda}^i_2-\Delta t\,\bar{X}^i_1\\
\bar{X}^i_2-\bar{X}^i_1+\Delta t\,\tanh(\bar{\lambda}^i_2/\delta)\\
\bar{\lambda}^i_2-\bar{\lambda}^i_3-\Delta t\,\bar{X}^i_2\\
\bar{X}^i_3-\bar{X}^i_2+\Delta t\,\tanh(\bar{\lambda}^i_3/\delta)
\end{pmatrix} \tag{154}
$$

and $d^i_j:=\partial_\lambda\tanh(\bar{\lambda}^i_j/\delta)=\delta^{-1}\cosh^{-2}(\bar{\lambda}^i_j/\delta)$, where $d^i_j\Delta t=\cosh^{-2}(\bar{\lambda}^i_j/\delta)$ since we are taking $\delta=\Delta t$.

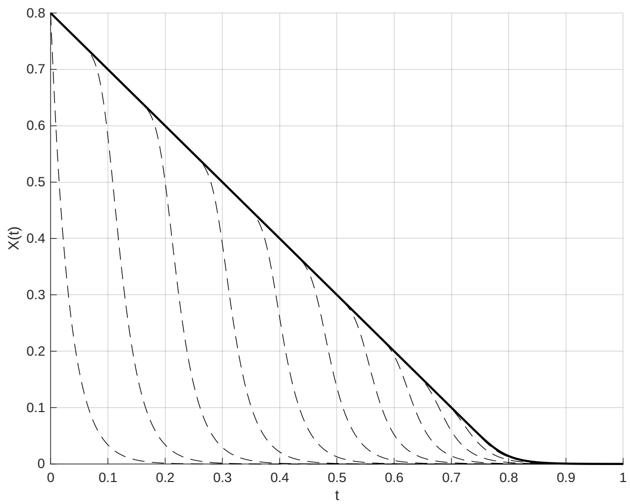

Figure 1: Solution $\bar{X}$ of the discrete optimization problem (148) for $\delta = \Delta t = 1/N$, $X_0 = 0.8$ and $H^\delta(\lambda, x) = -\tanh(\lambda/\delta)$, using the Newton method for $N = 1000$. The dashed lines shows the solution after each Newton iteration.

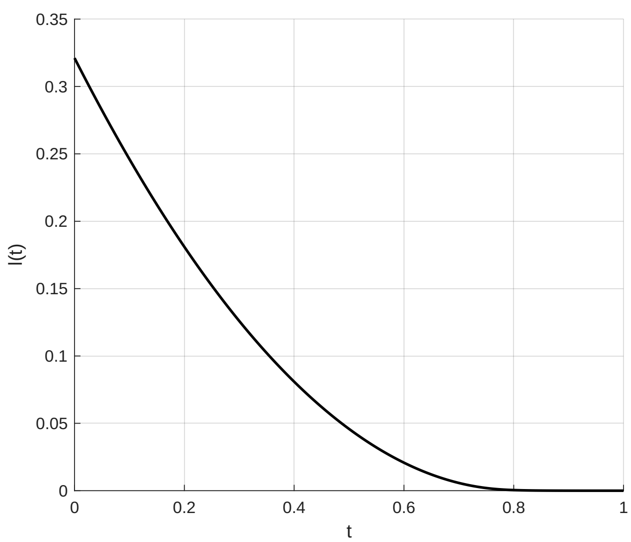

Figure 2: Solution $\bar{\lambda}$ of the discrete optimization problem (148) for $\delta = \Delta t = 1/N$, $X_0 = 0.8$ and $H^\delta(\lambda, x) = -\tanh(\lambda/\delta)$, using the Newton method for $N = 1000$.

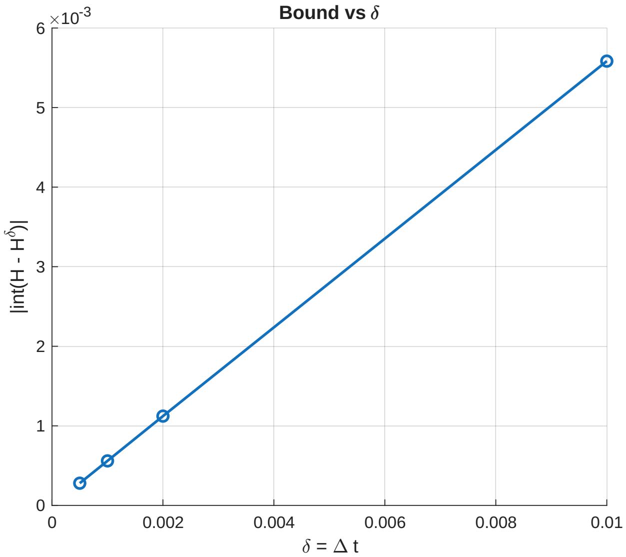

Figure 3: Solution $\bar{\lambda}$ of the discrete optimization problem (136) for $\delta = \Delta t = 1/N$, $X_0 = 0.8$ and $H^\delta(\lambda, x) = -\tanh(\lambda/\delta)$, using the Newton method for $N = 1000$.

Remark 11. In the last example we have an explicit formula for the Hamiltonian. This in general is not true and approximation techniques should be applied as well as regularization. One option to approximate the Hamiltonian is to use affine envelopes using the concavity property in the dual variable of the Hamiltonian. In the previous example this approximation is exactly the originbal hamiltonian since it is already define as an affine function in the dual variable.

Example 8. Let $B = \{u \in \mathbb{R}^2 : \|u\| \leq 1\}$, $X(t) \in \mathbb{R}^2$, $\alpha(t) \in \mathbb{R}^2$, $f(x, \alpha) = \alpha$, $h(x, \alpha) = \frac{1}{2}||x||^2$ and $g(x) = 0$, then our optimal control problem becomes

$$\begin{aligned} X'(t) &= \alpha(t), \quad 0 < t < T \\ X(0) &= X_0 \end{aligned} \tag{155}$$

with $X_0 = (1, 0)$ and

$$u(X_0, 0) = \inf_{\alpha \in B} \int_0^T \frac{1}{2}\|X(t)\|^2 dt \tag{156}$$

The Hamiltonian $H : \mathbb{R}^2 \times \mathbb{R}^2 \to \mathbb{R}$ for this problem is

$$H(\lambda, x) = \min_{\alpha \in B} \left(\frac{1}{2}||x||^2 + \lambda^T \alpha \right) \tag{157}$$

$$\tag{158}$$

By Cauchy-Schwarz inequality

$$\lambda^T \alpha \geq -\|\lambda\| ||\alpha|| \geq -\|\lambda\|$$

For $\lambda = 0$, the equality is trivial in the above expression. For λ with $\|\lambda\| > 0$, the equality is attained by chosing $\alpha = -\frac{\lambda}{\|\lambda\|}$, thus $\min_{\alpha \in B} \lambda^T \alpha = -\|\lambda\|$, and consequently

$$H(\lambda, x) = \frac{1}{2}\|x\|^2 - \|\lambda\| \tag{159}$$

We observe that

(a) H is concave in λ.

(b) Also

$$\nabla_x H(\lambda, x) = x \tag{160}$$

$$\nabla_\lambda H(\lambda, x) = -\frac{\lambda}{\|\lambda\|}, \quad \|\lambda\| \neq 0 \tag{161}$$

Since $H(\lambda, x)$ is concave in the dual variable, for $\hat{\lambda}$ with $\|\hat{\lambda}\| > 0$, we have that

$$\begin{aligned}
H(\lambda, x) &\leq H(\hat{\lambda}, x) + \nabla_\lambda H(\hat{\lambda}, x)(\lambda - \hat{\lambda}) && (162)\\
&= \frac{1}{2}\|x\|^2 - \|\hat{\lambda}\| - \frac{\hat{\lambda}}{\|\lambda\|}(\lambda - \hat{\lambda}) && (163)\\
&= \frac{1}{2}\|x\|^2 - v^T\lambda, \quad v = \hat{\lambda}/\|\hat{\lambda}\| && (164)
\end{aligned}$$

For $\hat{\lambda} = 0$, any v with $\|v\| \leq 1$ works, and it belongs to the super differential. Hence we can approximate H by

$$\begin{aligned}
\bar{H}(\lambda, x) &= \min_i \left\{ \frac{1}{2}\|x\|^2 - v_i^T\lambda \right\} && (165)\\
&= \frac{1}{2}\|x\|^2 - \max_i\{v_i^T\lambda\} && (166)
\end{aligned}$$

We again regularize the approximated hamiltonian using the softmax smoothing, namely for $\delta > 0$,

$$\bar{H}_\delta(\lambda, x) = \frac{1}{2}\|x\|^2 - \delta \log \sum_{i=1}^{K} e^{v_i^T\lambda/\delta} \tag{167}$$

We first observe that

$$\max_i\{v_i^T\lambda\} = v^*\lambda = \delta \log e^{v^*\lambda/\delta} \leq \delta \log \sum_{i=1}^{K} e^{v_i^T\lambda/\delta} \tag{168}$$

which yields

$$\bar{H}_\delta(\lambda, x) \leq \bar{H}(\lambda, x) \tag{169}$$

One can prove the following useful inequality (see Remark 32)

$$\delta \log \sum_{i=1}^{K} e^{v_i^T\lambda/\delta} - \max_i\{v_i^T\lambda\} \leq \delta \log K, \tag{170}$$

Hence, we obtain the following error bound

$$0 \leq \bar{H}(\lambda, x) - \bar{H}_\delta(\lambda, x) \leq \delta \log K \tag{171}$$

From (164) we have that $H(\lambda, x) \le \bar{H}(\lambda, x)$, then

$$H - \bar{H}_\delta \le \bar{H} - \bar{H}_\delta \le \delta \log K \tag{172}$$

Similarly, from (169), we get

$$\bar{H}_\delta - H \le \bar{H} - H = \|\lambda\| - \max_i \{v_i^T \lambda\} = \gamma(\lambda) \tag{173}$$

observe that $v_i^T \lambda \le \|\lambda\|$, so $\gamma(\lambda) \ge 0$. From (172) and (173) we obtain

$$-\gamma(\lambda) \le H - \bar{H}_\delta \le \delta \log K, \tag{174}$$

equivalently

$$|H - \bar{H}_\delta| \le \max\{\gamma(\lambda), \delta \log K\} \le \gamma(\lambda) + \delta \log K \tag{175}$$

The Hamiltonian system is the following

$$\dot{X}(t) = \nabla_\lambda \bar{H}_\delta(X(t), \lambda(t)) = -\sum_{i=1}^{K} w_i(\lambda(t); \delta)\, v_i, \qquad X(0) = X_0, \tag{176}$$

$$\dot{\lambda}(t) = -\nabla_X \bar{H}_\delta(X(t), \lambda(t)) = -X(t), \qquad \lambda(T) = 0. \tag{177}$$

where

$$w_i(\lambda; \delta) = \frac{e^{v_i^\top \lambda/\delta}}{\sum_{j=1}^{K} e^{v_j^\top \lambda/\delta}}. \tag{178}$$

The symplectic one step method for the Hamiltonian system becomes

$$\bar{X}_{n+1} = \bar{X}_n - \Delta t \sum_{i=1}^{K} w_i(\lambda_{n+1}; \delta) v_i, \quad \bar{X}_0 = X_0 \tag{179}$$

$$\bar{\lambda}_n = \bar{\lambda}_{n+1} + \Delta t \bar{X}_n, \quad \bar{\lambda}_N = 0 \tag{180}$$

One can use Newton iterations for the nonlinear system $F(\bar{X}, \bar{\lambda}) = 0$ where $F : \mathbb{R}^{2N} \times \mathbb{R}^{2N} \to \mathbb{R}^{2Nd}$ and

$$F(\bar{X}, \bar{\lambda})_{2n} = \bar{X}_{n+1} - \bar{X}_n + \Delta t \sum_{i=1}^{K} w_i(\lambda_{n+1}; \delta) v_i, \tag{181}$$

$$F(\bar{X}, \bar{\lambda})_{2n+1} = \bar{\lambda}_n - \bar{\lambda}_{n+1} - \Delta t \bar{X}_n. \tag{182}$$

We consider the case $N = 3$. The values X_0 and $\bar{\lambda}_3 = g_x(\bar{X}_3) = 0$ are given and our system has the variables

$$z = \left(\bar{\lambda}_0,\, \bar{X}_1,\, \bar{\lambda}_1,\, \bar{X}_2,\, \bar{\lambda}_2,\, \bar{X}_3\right)^\top \in \mathbb{R}^{12}. \tag{183}$$

with

$$F(z) = \begin{pmatrix} \bar{\lambda}_0 - \bar{\lambda}_1 - \Delta t\, \bar{X}_0 \\ \bar{X}_1 - \bar{X}_0 + \Delta t \sum_{i=1}^{K} w_i(\lambda_1; \delta) v_i \\ \bar{\lambda}_1 - \bar{\lambda}_2 - \Delta t\, \bar{X}_1 \\ \bar{X}_2 - \bar{X}_1 + \Delta t \sum_{i=1}^{K} w_i(\lambda_2; \delta) v_i \\ \bar{\lambda}_2 - \bar{\lambda}_3 - \Delta t\, \bar{X}_2 \\ \bar{X}_3 - \bar{X}_2 + \Delta t \sum_{i=1}^{K} w_i(\lambda_3; \delta) v_i \end{pmatrix}. \tag{184}$$

We compute the Jacobian matrix for F. For the first component we have

$$F_0^{(\bar{\lambda})}(\bar{\lambda}_0, \bar{X}_1, \bar{\lambda}_1) = \bar{\lambda}_0 - \bar{\lambda}_1 - \Delta t \bar{X}_0, \tag{185}$$

with $\bar{X}_0$ fixed (not an unknown). Write the components:

$$F_0^{(\bar{\lambda})} = \begin{bmatrix} r_1 \\ r_2 \end{bmatrix} = \begin{bmatrix} \bar{\lambda}_{0,1} - \bar{\lambda}_{1,1} - \Delta t \bar{X}_{0,1} \\ \bar{\lambda}_{0,2} - \bar{\lambda}_{1,2} - \Delta t \bar{X}_{0,2} \end{bmatrix}. \tag{186}$$

Partial derivatives (block form)

- W.r.t. $\bar{\lambda}_0 \in \mathbb{R}^2$:
$$\frac{\partial F_0^{(\bar{\lambda})}}{\partial \bar{\lambda}_0} = \begin{bmatrix} \frac{\partial r_1}{\partial \bar{\lambda}_{0,1}} & \frac{\partial r_1}{\partial \bar{\lambda}_{0,2}} \\ \frac{\partial r_2}{\partial \bar{\lambda}_{0,1}} & \frac{\partial r_2}{\partial \bar{\lambda}_{0,2}} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I_2. \tag{187}$$

- W.r.t. $\bar{X}_1 \in \mathbb{R}^2$:
$$\frac{\partial F_0^{(\bar{\lambda})}}{\partial \bar{X}_1} = 0_{2\times 2} \quad \text{(porque } \bar{X}_1 \text{ no aparece).} \tag{188}$$

- W.r.t. $\bar{\lambda}_1 \in \mathbb{R}^2$:
$$\frac{\partial F_0^{(\bar{\lambda})}}{\partial \bar{\lambda}_1} = \begin{bmatrix} -1 & 0 \\ 0 & -1 \end{bmatrix} = -I_2. \tag{189}$$

All other partials (w.r.t. $\bar{X}_2, \bar{\lambda}_2, \bar{X}_3$) are $0_{2\times 2}$.
Similarly,

$$F_0^{(X)}(\bar{\lambda}_1, \bar{X}_1) = \bar{X}_1 - \bar{X}_0 + \Delta t\, u(\bar{\lambda}_1), \qquad u(\lambda) = \sum_{i=1}^{K} w_i(\lambda; \delta) v_i, \tag{190}$$

with

$$w_i(\lambda; \delta) = \frac{e^{v_i^\top \lambda/\delta}}{\sum_{j=1}^{K} e^{v_j^\top \lambda/\delta}}, \qquad \|v_i\| = 1, \qquad \bar{X}_0 \text{ fixed.} \tag{191}$$

Write

$$F_0^{(X)} = \begin{bmatrix} r_1 \\ r_2 \end{bmatrix} \quad \text{with} \quad r_j = (\bar{X}_1)_j - (\bar{X}_0)_j + \Delta t\, u_j(\bar{\lambda}_1), \qquad j = 1, 2. \tag{192}$$

Partial derivatives (blocks 2×2)

- With respect to $\bar{X}_1$:
$$\frac{\partial F_0^{(X)}}{\partial \bar{X}_1} = \begin{bmatrix} \frac{\partial r_1}{\partial (\bar{X}_1)_1} & \frac{\partial r_1}{\partial (\bar{X}_1)_2} \\ \frac{\partial r_2}{\partial (\bar{X}_1)_1} & \frac{\partial r_2}{\partial (\bar{X}_1)_2} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix} = I_2. \tag{193}$$

- With respect to $\bar{\lambda}_1$: you need $\frac{\partial u}{\partial \lambda}$.
  First, the gradient of the softmax weights:
$$\nabla_\lambda w_i(\lambda) = \frac{1}{\delta} w_i(\lambda)\big(v_i - u(\lambda)\big). \tag{194}$$

Therefore,

$$\frac{\partial u}{\partial \lambda} = \sum_i v_i (\nabla_\lambda w_i)^\top = \frac{1}{\delta}\Big(\sum_i w_i v_i v_i^\top - u u^\top\Big) =: G(\lambda) \in \mathbb{R}^{2\times 2}. \tag{195}$$

(It is symmetric, negative semidefinite, and $\|G(\lambda)\| \le 1/\delta$). Hence,

$$\frac{\partial F_0^{(X)}}{\partial \bar{\lambda}_1} = \Delta t\, G(\lambda_1). \tag{196}$$

- With respect to the other variables $(\lambda_0, \bar{\lambda}_2, \bar{X}_2, \bar{X}_3)$:

$$\frac{\partial F_0^{(X)}}{\partial \lambda_0} = \frac{\partial F_0^{(X)}}{\partial \bar{\lambda}_2} = \frac{\partial F_0^{(X)}}{\partial \bar{X}_2} = \frac{\partial F_0^{(X)}}{\partial \bar{X}_3} = 0_{2\times 2}. \tag{197}$$

For implementation, we construct the following system

$$\Delta z = (\Delta\bar{\lambda}_0,\, \Delta\bar{\lambda}_1,\, \Delta\bar{X}_1,\, \Delta\bar{\lambda}_2,\, \Delta\bar{X}_2,\, \Delta\bar{X}_3), \tag{198}$$

(each entry is a 2-vector). Residuals are ordered

$$F = \big(F_0^{(\bar{\lambda})},\, F_0^{(X)},\, F_1^{(\bar{\lambda})},\, F_1^{(X)},\, F_2^{(\bar{\lambda})},\, F_2^{(X)}\big). \tag{199}$$

Define, at the current iterate,

$$u(\lambda) = \sum_{i=1}^{K} w_i(\lambda;\delta) v_i, \qquad G(\lambda) = \frac{\partial u}{\partial \lambda} = \frac{1}{\delta}\left(\sum_{i=1}^{K} w_i v_i v_i^\top - u u^\top\right) \in \mathbb{R}^{2\times 2}. \tag{200}$$

Block Newton system

$$J\,\Delta z = -F \quad \text{(each boxed entry is } 2\times 2\text{)}$$

$$\underbrace{\begin{bmatrix} I & -I & 0 & 0 & 0 & 0 \\ 0 & \Delta t G(\bar{\lambda}_1) & I & 0 & 0 & 0 \\ 0 & I & -\Delta t I & -I & 0 & 0 \\ 0 & 0 & -I & \Delta t G(\bar{\lambda}_2) & I & 0 \\ 0 & 0 & 0 & I & -\Delta t I & 0 \\ 0 & 0 & 0 & 0 & -I & I \end{bmatrix}}_{J} \begin{bmatrix} \Delta\bar{\lambda}_0 \\ \Delta\bar{\lambda}_1 \\ \Delta\bar{X}_1 \\ \Delta\bar{\lambda}_2 \\ \Delta\bar{X}_2 \\ \Delta\bar{X}_3 \end{bmatrix} = - \begin{bmatrix} F_0^{(\bar{\lambda})} \\ F_0^{(X)} \\ F_1^{(\bar{\lambda})} \\ F_1^{(X)} \\ F_2^{(\bar{\lambda})} \\ F_2^{(X)} \end{bmatrix}. \tag{201}$$

Analytical Solution : The problem is **radially symmetric**: the dynamics $X'(t) = \alpha(t)$ with $\|\alpha\| \le 1$, and the running cost $\frac{1}{2}\|X(t)\|^2$, depend only on the distance to the origin. So the optimal control is point straight towards the origin at maximum speed until you arrive, then stop. Let

$$r_0 := \|X_0\|, \quad e := \frac{X_0}{\|X_0\|} \quad (r_0 > 0). \tag{202}$$

The optimal controls is

$$\alpha^*(t) = \begin{cases} -e, & t < \min\{r_0, T\}, \\ 0, & t \geq r_0 \text{ and } r_0 \leq T. \end{cases} \tag{203}$$

The optimal state is

$$X^*(t) = \begin{cases} (r_0 - t)e, & 0 \leq t \leq \min\{r_0, T\}, \\ 0, & r_0 \leq t \leq T \text{ (if } r_0 \leq T). \end{cases} \tag{204}$$

The Value function (cost) is

$$u(X_0, 0) = \frac{1}{2} \int_0^T \|X^*(t)\|^2 \, dt = \begin{cases} \dfrac{r_0^3}{6}, & r_0 \leq T, \\ \dfrac{r_0^3 - (r_0 - T)^3}{6}, & r_0 > T. \end{cases} \tag{205}$$

Pontryagin gives $\lambda'(t) = -X(t)$, $\lambda(T) = 0$ (consistent with $\nabla_x H = x$). Thus:

- If $r_0 \leq T$:

$$\lambda^*(t) = \begin{cases} \dfrac{1}{2}(r_0 - t)^2 \, e, & 0 \leq t \leq r_0, \\ 0, & r_0 \leq t \leq T. \end{cases} \tag{206}$$

- If $r_0 > T$:

$$\lambda^*(t) = \frac{1}{2} \left((r_0 - t)^2 - (r_0 - T)^2 \right) e, \quad 0 \leq t \leq T. \tag{207}$$

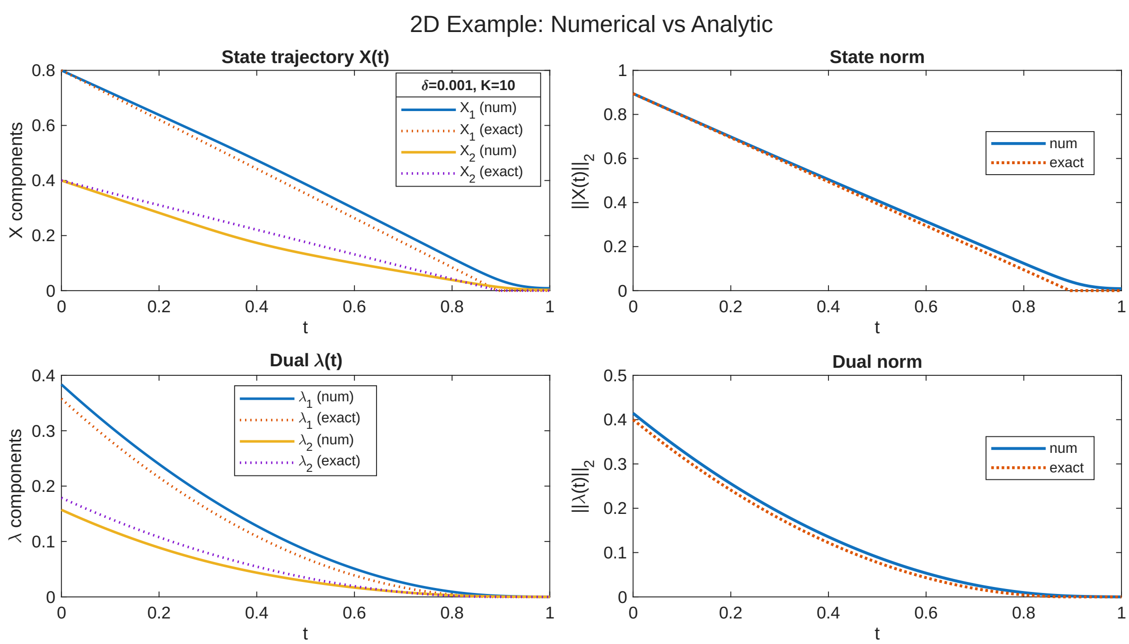

Figure 4: Solution $\bar{X}$, $\bar{\lambda}$ of the discrete optimization problem (179)-(180) for $\delta = \Delta t = 1/N$, $X_0 = [0.8, 0.4]$, using the Newton method. Plots on the right shows the norm of the trajectories.

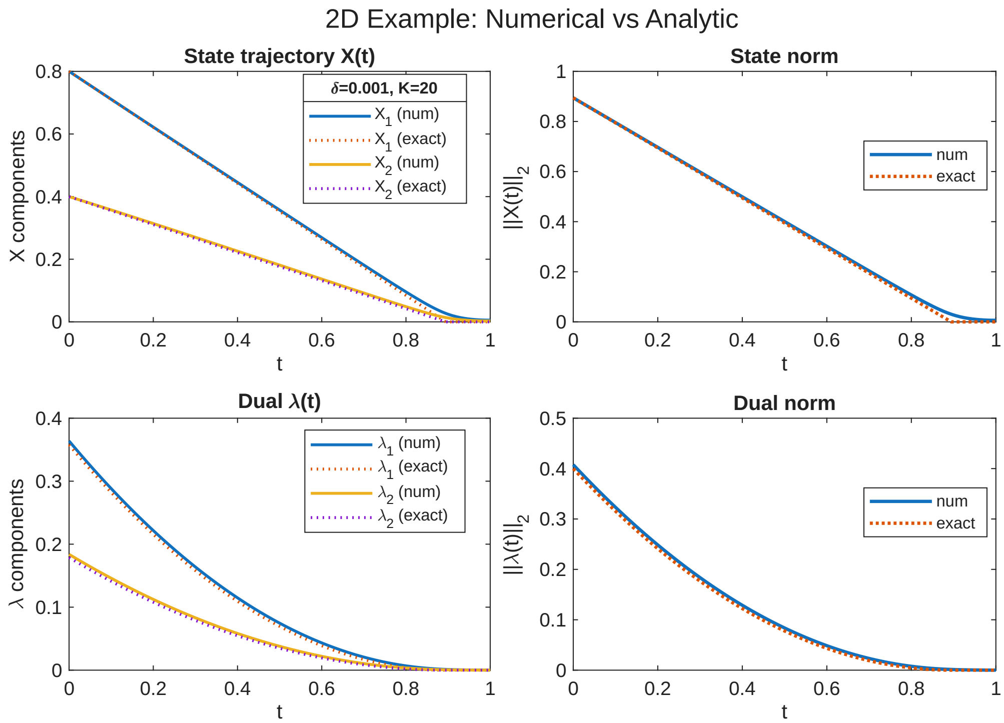

Figure 5: Solution $\bar{X}$, $\bar{\lambda}$ of the discrete optimization problem (179)-(180) for $\delta = \Delta t = 1/N$, $X_0 = [0.8, 0.4]$, using the Newton method. Plots on the right shows the norm of the trajectories.

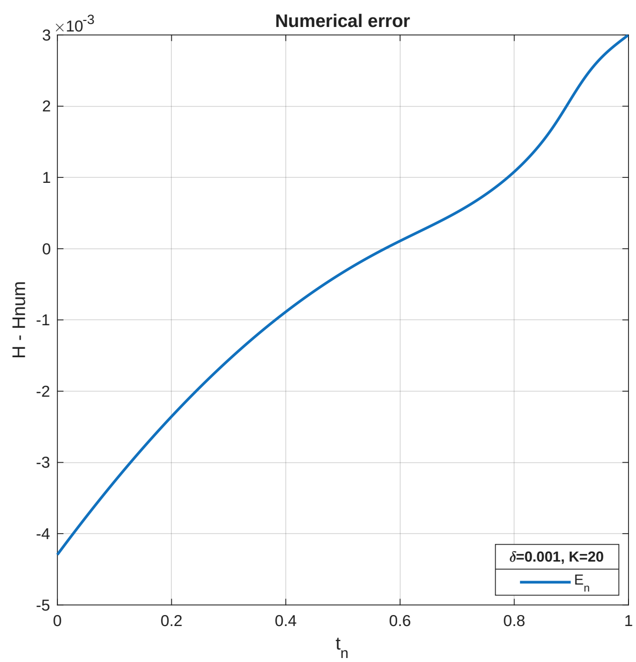

Figure 6: $H - \bar{H}_\delta$ along the solutions $\bar{X}$, and $\bar{\lambda}$ for $\delta = \Delta t = 1/N$, $X_0 = [0.8, 0.4]$.

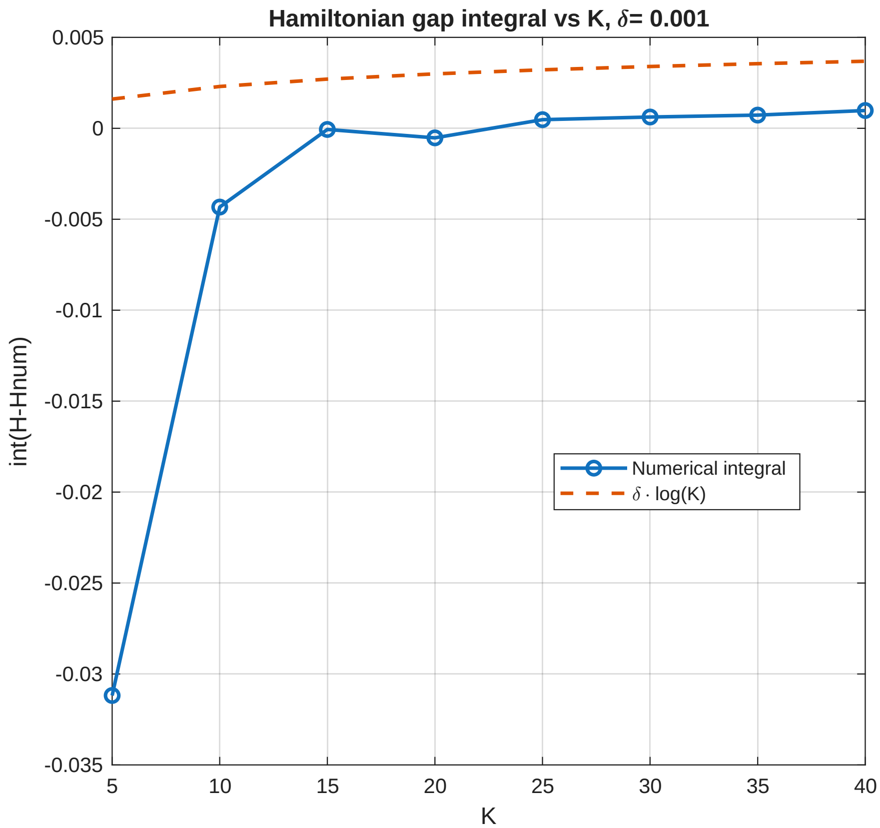

Figure 7: $\int_0^T (H - \bar{H}_\delta)dt$ along the solutions $\bar{X}$, and $\bar{\lambda}$ for $K = [5, 10, 15, 20, 25, 30, 35, 40]$ and $\delta = \Delta t = 1/N$, $X_0 = [0.8, 0.4]$.

4 Hamiltonian Regularization

4.1 Concave and Semiconcave functions

Definition 8. A function $f : C \to \mathbb{R}$, with C a convex subset of $\mathbb{R}^n$ is concave if

$$f(\alpha x + (1-\alpha)y) \geq \alpha f(x) + (1-\alpha) f(y), \quad \forall x, y \in C, \forall \alpha \in [0,1],$$

Proposition 3. *Let $f_i : \mathbb{R}^n \to (-\infty, \infty]$ be given functions for $i \in I$, where I is an arbitrary index set, and let $f : \mathbb{R}^n \to (-\infty, \infty]$ be the function given by*

$$f(x) = \inf_{i \in I} f_i(x).$$

If $f_i, i \in I$, are concave, then f is also concave, while if hypo$(f_i), i \in I$, are closed, then hypo$(f)f$ is also closed.

The next result characterizes concavity under differentiability condition.

Proposition 4. *Let C be a nonempty convex subset of $\mathbb{R}^n$ and let $f : \mathbb{R}^n \to \mathbb{R}$ be differentiable over an open set that contains C.*

(a) f is concave over C if and only if

$$f(z) \leq f(x) + \nabla f(x)^T (z - x), \quad \forall x, z \in C. \tag{208}$$

(b) f is strictly concave over C if and only if the above inequality is strict whenever $x \neq z$.

Remark 12. If $f : \mathbb{R}^n \to \mathbb{R}$ ia a differentiable concave function and $\nabla f(x^*) = 0$, then from inequality (208), x^* maximizes f over $\mathbb{R}^n$.

Exercise 1. The Hamiltonian

$$H(\lambda, x) := \min_{\alpha \in A} (\lambda \cdot f(x, \alpha) + h(x, \alpha)) \tag{209}$$

is concave in the λ-variable, that is, for each λ_1 and λ_2 in $\mathbb{R}^d$ and for all $s \in [0, 1]$ there holds

$$H(s\lambda_1 + (1 - s)\lambda_2, x) \geq sH(\lambda_1, x) + (1 - s)H(\lambda_2, x). \tag{210}$$

5 Research Statement on Optimal Control

Consider the folowing problem

$$\min_{\alpha \in \mathcal{A}} \int_t^T h(s, X_s, \alpha_s) ds + g(X_T), \tag{211}$$

where

$$\dot{X}_s = f(s, X_s, \alpha_s), \quad s > t, \tag{212}$$
$$X_t = x. \tag{213}$$

The Hamiltonian is given by

$$H(t, x, p) = \min_{\alpha \in \mathcal{A}} (h + pf)(t, x, \alpha). \tag{214}$$

We want to compute the subgradient of H with respect to p at p_0. To this end, we introduce the constrain $p = p_0$ in the definition of the Hamiltonian

$$H(t, x, p_0) = \min_{\substack{\alpha \in \mathcal{A} \\ p = p_0}} (h + pf)(t, x, \alpha),$$

and now we compute the Lagrange multiplier, μ, corresponding to the constraint $p = p_0$. For this, we introduce the Lagrangian

$$\mathcal{L}(t, x, p, \mu) = \min_{\alpha \in \mathcal{A}} (h(t, x, \alpha) + pf(t, x, \alpha) - \mu(p - p_0)) \tag{215}$$
$$= h(t, x, \alpha^*) + pf(t, x, \alpha^*) - \mu(p - p_0) \tag{216}$$

and take the variation $\partial_p \mathcal{L} = 0$, yielding

$$\mu = f(t, x, \alpha^*).$$

Thanks to the duality theorem we have

$$\partial_p H = \mu$$

and we conclude

$$\partial_p H = \mu = f(t, x, \alpha^*)$$

Now let us use the property that the Hamiltonian is concave with respect to p to write

$$\begin{aligned} H(x, p) &= \min_{\alpha \in \mathcal{A}} h(\alpha, x) + p f(\alpha, x) \\ &\leq \min_{i \in I} g_i(x) p + d_i(x) \end{aligned}$$

with

$$g_i(x) p + d_i(x) \geq H(x, p), \quad \forall (x, p).$$

We can simply choose (g_i, d_i) by evaluating the Hamiltonian and its derivative at properly chosen points, namely

$$H(x, p) \leq H(x, \hat{p}) + \partial_p H(x, \hat{p})(p - \hat{p}), \quad \forall (x, p). \tag{217}$$

By defining

$$g_i(x) := \partial_p H(x, \hat{p}_i), \tag{218}$$

$$d_i(x) := H(x, \hat{p}_i) - g_i(x) \hat{p}_i \tag{219}$$

In general, our approximation (upper bound)

$$\bar{H} = \min_{i \in I} \{ g_i(x) p + d_i(x) \}$$

will not be exact. Still, thanks to its structure we can consider the Pontryagin system

$$\begin{aligned} \dot{X} &= \partial_p \bar{H} \\ \dot{p} &= -\partial_x \bar{H} \end{aligned}$$

and its regularization. The regularization of $\bar{H}$ with $\delta > 0$ produces $\bar{H}_\delta$ and it goes like

$$H(x, p) \leq \bar{H}(x, p) = \min_{i \in I} \{ g_i(x) p + d_i(x) \}$$

$$\bar{H}_\delta(x, p) = \min\nolimits_{i \in I}^{\delta} \{ g_i(x) p + d_i(x) \} \leq \bar{H}(x, p)$$

As discussed, we can construct a function $\min_\delta \in C^2$ and such that

$$\| \min_\delta - \min \|_C \leq C \delta.$$

Task: Solve the Pontryagin system corresponding to $\bar{H}_\delta$ using Newton iterations.

Example 9. Consider the problem

$$\min_{\alpha\in[-1,1]} \int_0^T h(s, X_s, \alpha_s)ds + g(X_T) = \min_{\alpha\in[-1,1]} \int_0^T X_s^2 ds$$

where,

$$\begin{aligned} \dot{X}_s &= f(s, X_s, \alpha_s) = \alpha \\ X_0 &= 0. \end{aligned}$$

Then

$$\begin{aligned} H(t, x, p) &= \min_{\alpha\in[-1,1]} \{h(t, x, \alpha) + pf(t, x, \alpha)\} \\ &= \min_{\alpha\in[-1,1]} (x^2 + p\alpha) \\ &= x^2 - |p|, \end{aligned}$$

and

$$f(t, x, \alpha^*) = \alpha^* = \text{sign}(p) = \partial_p H.$$

Observe that at $p = 0$ we only have subgradients and any number in $[-1, 1]$ does the job. Take the evaluations $\tilde{p}_1 > 0$ and $\tilde{p}_2 < 0$, to produce an exact representation of H. Namely from (217), we have

$$\begin{aligned} H(x, p) &\leq H(x, \tilde{p}_1) + \partial_p H(x, \tilde{p}_1)(p - \tilde{p}_1), \quad \forall(x, p) \\ &= x^2 - \tilde{p}_1 - p + \tilde{p}_1 \\ &= x^2 - p, \end{aligned}$$

$$\begin{aligned} H(x, p) &\leq H(x, \tilde{p}_2) + \partial_p H(x, \tilde{p}_2)(p - \tilde{p}_2), \quad \forall(x, p) \\ &= x^2 + \tilde{p}_2 + p - \tilde{p}_2 \\ &= x^2 + p. \end{aligned}$$

Then we get

$$\bar{H}(x, p) = x^2 + \min\{-p, p\} = H(x, p), \quad \forall(x, p).$$

6 Optimal Control of Stochastic Differential Equations

7 Discretizations of Stochastic Differential Equations

Consider a general n-dimensional SDE driven by a d-dimensional Brownian motion B, i.e.,

7.1 Research Statement

- Entender el metodo para el caso deterministico y generalizarlo para el caso estocastico.
-

$$dX_t = V(X_t)dt + \sum_{i=1}^{d} V_i(X_t)\, dB_t^i, \tag{220}$$

for some vector fields $V, V_1, \ldots, V_d : \mathbb{R}^n \to \mathbb{R}^n$, which satisfy:

- **Lipschitz (global, uniform)**
  There exists $L > 0$ such that for all $x, y \in \mathbb{R}^n$,

$$\|V(x) - V(y)\| \le L\|x - y\|, \tag{221}$$
$$\|V_i(x) - V_i(y)\| \le L\|x - y\| \quad \text{for } i = 1, \ldots, d. \tag{222}$$

  Equivalently, with $\sigma(x) = [V_1(x) \ \cdots \ V_d(x)] \in \mathbb{R}^{n\times d}$,

$$\|\sigma(x) - \sigma(y)\|_{\mathrm{F}} \le L\|x - y\|. \tag{223}$$

- **Linear growth (linearly bounded, uniform)**
  There exists $K > 0$ such that for all $x \in \mathbb{R}^n$,

$$\|V(x)\| \le K(1 + \|x\|), \tag{224}$$
$$\|V_i(x)\| \le K(1 + \|x\|) \quad \text{for } i = 1, \ldots, d. \tag{225}$$

  Equivalently,

$$\|V(x)\|^2 + \|\sigma(x)\|_{\mathrm{F}}^2 \le K^2(1 + \|x\|^2). \tag{226}$$

If the coefficients are time-dependent, $V(t,x), V_i(t,x)$, the same inequalities hold *uniformly* in $t \ge 0$ (i.e., the same constants L, K work for all t).

Remark 13. Equation (220) can be generalized to the **non-autonomous** form

$$dX_t = V(t, X_t)\, dt + \sum_{i=1}^{d} V_i(t, X_t)\, dB_t^i, \tag{227}$$

for existence and uniqueness, the regularity in t is usually less restrictive (e.g. measurability and uniform boundedness in t) than in x (global Lipschitz and linear growth). However, the autonomous formulation used here is chosen for sharper error bounds; a direct extension of these results to the non-autonomous case generally produces weaker estimates.

Remark 14. We assume that $X_0 = x_0 \in \mathbb{R}^n$. The theory is not more difficult as long as X_0 is independent of the noise.

We fix the time interval $[0, T]$, and consider a grid $\mathcal{D} = \{0 = t_0 < t_1 < \cdots < t_N = T\}$ with size N. We denote

$$|\mathcal{D}| := \max_{1 \le i \le N} |t_i - t_{i-1}| \tag{228}$$

the *mesh* of the grid, and we define the increments of time and of any process Y (which will usually be either X, B, or Z) along the grid by

$$\Delta t_i := t_i - t_{i-1}, \quad \Delta Y_i := Y_{t_i} - Y_{t_{i-1}}, \quad 1 \le i \le N. \tag{229}$$

Moreover, for $t \in [0, T]$ we set $\lfloor t \rfloor = \sup\{t_i \mid 0 \le i \le N, \ t_i \le t\}$. We will define the approximation along the grid, i.e., we will define the random variables $\overline{X}_i = \overline{X}_{t_i}$, $0 \le i \le N$. We will write $\overline{X}^{\mathcal{D}}$ if we want to emphasise the dependence on the grid. The first natural question arising from this program is in which sense $\overline{X}$ should be an approximation to X. The two most important concepts are *strong* and *weak approximation*.

Definition 9. The scheme $\overline{X}^{\mathcal{D}}$ *converges strongly* to X if

$$\lim_{|\mathcal{D}| \to 0} \mathbb{E}\left[\left|X_T - \overline{X}_T^{\mathcal{D}}\right|\right] = 0. \tag{230}$$

Moreover, we say that the scheme $\overline{X}^{\mathcal{D}}$ has *strong order* γ if (for $|\mathcal{D}|$ small enough)

$$\mathbb{E}\left[\left|X_T - \overline{X}_T^{\mathcal{D}}\right|\right] \le C\,|\mathcal{D}|^{\gamma} \tag{231}$$

for some constant $C > 0$, which does not depend on $\gamma > 0$.

Definition 10. Given a suitable class $\mathcal{G}$ of functions $f : \mathbb{R}^n \to \mathbb{R}$, we say that the scheme $\overline{X}^{\mathcal{D}}$ *converges weakly* (with respect to $\mathcal{G}$) if

$$\forall f \in \mathcal{G}: \quad \lim_{|\mathcal{D}| \to 0} \mathbb{E}\left[f\left(\overline{X}_T^{\mathcal{D}}\right)\right] = \mathbb{E}\left[f(X_T)\right]. \tag{232}$$

Moreover, we say that $\overline{X}^{\mathcal{D}}$ has *weak order* $\gamma > 0$ if for every $f \in \mathcal{G}$ there is a constant C (not depending on $|\mathcal{D}|$) such that

$$\left|\mathbb{E}\left[f\left(\overline{X}_T^{\mathcal{D}}\right)\right] - \mathbb{E}\left[f(X_T)\right]\right| \le C\,|\mathcal{D}|^{\gamma} \tag{233}$$

provided that $|\mathcal{D}|$ is small enough.

Remark 15. If a scheme converges strongly with order γ, then we can immediately conclude that it will also converge weakly with order γ, provided that all the functions in $\mathcal{G}$ are uniformly Lipschitz. There is a big difference between these concepts: for instance, a strong scheme must be defined on the same probability space as the true solution X, which is clearly not necessary in the weak case.

7.2 The Euler-Maruyama Method

7.2.1 Strong Error Rate

The Euler scheme for SDEs (also known as *Euler–Maruyama scheme*) is defined by

$$\overline{X}_i = \overline{X}_{i-1} + V(\overline{X}_{i-1})\,\Delta t_i + \sum_{j=1}^{d} V_j(\overline{X}_{i-1})\,\Delta B_i^j, \qquad 1 \le i \le N \tag{234}$$

$$\overline{X}_0 = x \tag{235}$$

Moreover, we extend the definition of $\overline{X}_i = \overline{X}_{t_i}$ for all times $t \in [0,T]$ by some kind of stochastic interpolation between the grid points, more precisely by

$$\overline{X}_t = \overline{X}_{\lfloor t \rfloor} + V(\overline{X}_{\lfloor t \rfloor})(t - \lfloor t \rfloor) + \sum_{i=1}^{d} V_i(\overline{X}_{\lfloor t \rfloor})(B_t^i - B_{\lfloor t \rfloor}^i). \tag{236}$$

Before presenting the main theorem, we recall two fundamental lemmas from analysis and probability theory essential for the proof.

Lemma 1 (Grönwall's inequality)**.** *Let u be a real-valued, continuous function and let $\alpha, \beta \ge 0$. Assuming that for all $0 \le t \le T$ we have*

$$u(t) \le \alpha + \beta \int_0^t u(s)\,ds, \tag{237}$$

then we get

$$u(t) \le \alpha e^{\beta t}, \qquad 0 \le t \le T. \tag{238}$$

Lemma 2 (Doob's inequality, L^2 case)**.** *Let $(M_t)_{t\in[0,T]}$ be a square integrable martingale. Then we have*

$$\mathbb{E}\left[\sup_{0\le t\le T} |M_t|^2\right] \le 4\mathbb{E}\left[|M_T|^2\right]. \tag{239}$$

Theorem 10. *Suppose that the coefficients of the SDE* (220) *have a uniform Lipschitz constant $K > 0$ and satisfy the linear growth condition with the same constant. Then the Euler–Maruyama approximation $\overline{X}$ satisfies*

$$\mathbb{E}\left[\sup_{0\le t\le T} \left|X_t - \overline{X}_t\right|\right] \le C\sqrt{|\mathcal{D}|} \tag{240}$$

for some constant C only depending on the coefficients, the initial value and the time horizon T. In particular, the Euler–Maruyama method has strong order $1/2$.

Remark 16. (a) In Theorem 10 the bound is written in L^1 form:

$$\mathbb{E}\left[\sup_{0\le t\le T} \left|X_t - \overline{X}_t\right|\right] \le C|\mathcal{D}|^{1/2}. \tag{241}$$

If we consider the L^2 norm (RMS) form, we get

$$\left(\mathbb{E}\left[\sup_{0\le t\le T}\left|X_t-\overline{X}_t\right|^2\right]\right)^{1/2}\le C|\mathcal{D}|^{1/2}. \tag{242}$$

In the proof of the theorem, the author actually computes the MSE (243), but from above equation and below item, we can obtain the other bounds.

$$\mathbb{E}\left[\sup_{0\le t\le T}\left|X_t-\overline{X}_t\right|^2\right]\le C|\mathcal{D}|. \tag{243}$$

(b) Observe that particurly, by Cauchy-Swartz[1]

$$\mathbb{E}\left[\sup_{0\le t\le T}\left|X_t-\overline{X}_t\right|\right]\le\left(\mathbb{E}\left[\sup_{0\le t\le T}\left|X_t-\overline{X}_t\right|^2\right]\right)^{1/2}. \tag{244}$$

Thus, an L^2 error bound automatically yields an L^1 bound of the same order.

7.2.2 Weak Convergence of the Euler Method

Next we discuss the weak convergence of the Euler method. Weak approximation of SDEs can be used as a numerical method for solving linear parabolic PDEs. Indeed,

$$u(t,x):=\mathbb{E}\left[f(X_T)\mid X_t=x\right] \tag{245}$$

satisfies the *Kolmogorov backward equation* associated to the generator $L=V_0+\frac{1}{2}\sum_{i=1}^d V_i^2$, i.e., the Cauchy problem

$$\begin{cases}\dfrac{\partial}{\partial t}u(t,x)+Lu(t,x)=0,\\ \qquad\qquad u(T,x)=f(x),\end{cases} \tag{246}$$

a PDE known as Black–Scholes PDE in finance.

Remark 17. On the other hand, strong convergence implies weak convergence. Indeed, assume that f is Lipschitz, with Lipschitz constant denoted by $\|\nabla f\|_\infty$. Then we have

$$\left|\mathbb{E}\left[f\left(\overline{X}_T^{\mathcal{D}}\right)\right]-\mathbb{E}\left[f(X_T)\right]\right|\le\|\nabla f\|_\infty\,\mathbb{E}\left[\left|X_T-\overline{X}_T^{\mathcal{D}}\right|\right]\le C\|\nabla f\|_\infty\sqrt{|\mathcal{D}|}, \tag{247}$$

by Theorem 10. Thus, the Euler scheme has (at least) weak order 1/2 for all Lipschitz functions f. However, in many situations the weak order is actually better than the strong order.

In the following, we shall first present (and prove) "the typical situation" under unnecessarily restrictive regularity assumptions, before we state sharper results (without proofs). Our presentation is mainly based on Talay and Tubaro [?]. For our discussion we assume that the grids $\mathcal{D}$ are homogeneous, i.e., $\Delta t_i=h:=T/N$ for every i.

[1] $Z:=|X_t|\ge 0$. Then $\mathbb{E}|X_t|=\mathbb{E}[Z\cdot 1]\overset{\text{C–S}}{\le}\left(\mathbb{E}[Z^2]\right)^{1/2}\left(\mathbb{E}[1^2]\right)^{1/2}=\left(\mathbb{E}|X_t|^2\right)^{1/2}$.

Theorem 11. *Assume that the vector fields $V, V_1, \dots, V_d$ are C^∞-bounded, i.e., they are smooth and the vector fields together with all their derivatives are bounded functions. Moreover, assume that $\mathcal{G}$ consists of smooth, polynomially bounded functions. Then the Euler method has weak order one. Moreover, the error*

$$e(T,h,f) := \mathbb{E}\left[f\left(\overline{X}_T^{\mathcal{D}}\right)\right] - u(0,x) \tag{248}$$

for the weak approximation problem started at $t = 0$ at $X_0 = \overline{X}_0 = x \in \mathbb{R}^n$ has the representation

$$e(T,h,f) = h\int_0^T \mathbb{E}[\psi_1(s,X_s)]\,ds + h^2 e_2(T,f) + \mathcal{O}(h^3), \tag{249}$$

where ψ_1 is given by

$$\begin{aligned}\psi_1(t,x) &= \frac{1}{2}\sum_{i,j=1}^n V^i(x)V^j(x)\partial_{(i,j)}u(t,x) + \frac{1}{2}\sum_{i,j,k=1}^n V^i(x)a_k^j(x)\partial_{(i,j,k)}u(t,x)\\ &+ \frac{1}{8}\sum_{i,j,k,l=1}^n a_j^i(x)a_k^l(x)\partial_{(i,j,k,l)}u(t,x) + \frac{1}{2}\frac{\partial^2}{\partial t^2}u(t,x)\\ &+ \sum_{i=1}^n V^i(x)\frac{\partial}{\partial t}u(t,x)\partial_i u(t,x) + \frac{1}{2}\sum_{i,j=1}^d a_j^i(x)\frac{\partial}{\partial t}u(t,x)\partial_{(i,j)}u(t,x),\end{aligned}$$

where $\partial_I = \frac{\partial^k}{\partial x^{i_1}\cdots\partial x^{i_k}}$ for a multi-index $I = (i_1,\dots,i_k)$ and

$$a_j^i(x) = \sum_{k=1}^d V_k^i(x)V_k^j(x), \quad 1 \le i,j \le n.$$

Remark 18. The result also holds for the non-autonomous case, i.e., for $f = f(t,x)$ and the vector fields also depending on time.

7.2.3 The Euler Monte Carlo Method

The Euler method only solves half the problem in determining the quantity $\mathbb{E}[f(X_T)]$, when X_T is given as the solution of an SDE. Indeed, it replaces the unknown random variable X_T by a known random variable $\overline{X}_N$. Therefore, we want to approximate $\mathbb{E}[f(X_T)]$ by $\mathbb{E}[f(\overline{X}_N)]$.

In the end, we approximate our quantity of interest $\mathbb{E}[f(X_T)]$ by a weighted average of copies of $f(\overline{X}_N)$, which are either chosen to be random, independent of each other in the case of Monte Carlo simulation, or deterministic according to a sequence of low discrepancy in the case of Quasi Monte Carlo. Of course, this gives us a natural decomposition of the (absolute) computational error into two parts:

$$\hat{\mu}_M = \frac{1}{M}\sum_{i=1}^M f\left(\overline{X}_N^{(i)}\right), \qquad \overline{X}_N = \text{Euler on grid of size } N. \tag{250}$$

Then by adding and substracting $\mathbb{E}[f(\overline{X}_N)]$, we get

$$|\mathbb{E}[f(X_T)] - \hat{\mu}_M| \leq \underbrace{\left|\mathbb{E}[f(X_T)] - \mathbb{E}[f(\overline{X}_N)]\right|}_{\text{discretization (bias)}} + \underbrace{\left|\mathbb{E}[f(\overline{X}_N)] - \hat{\mu}_M\right|}_{\text{integration/statistical error}}, \tag{251}$$

If we instead consider the MSE:

$$MSE = \mathbb{E}\left[\left(\frac{1}{M}\sum_{i=1}^{M} f\left(\overline{X}_N^{(i)}\right) - \mathbb{E}[f(X_T)]\right)^2\right]. \tag{252}$$

Again by adding and substracting $\mathbb{E}[f(\overline{X}_N)]$, we obtain

$$\text{MSE} = \underbrace{\left(\mathbb{E}[f(\overline{X}_N)] - \mathbb{E}[f(X_T)]\right)^2}_{\text{bias}^2} + \underbrace{\text{Var}\left(\frac{1}{M}\sum_{i=1}^{M} f\left(\overline{X}_N^{(i)}\right)\right)}_{\text{variance term}}. \tag{253}$$

8 Monte Carlo and Multilevel Monte Carlo

8.1 Monte Carlo Method

Monte Carlo Method estimates $\mathbb{E}[X]$ as a mean of M independent samples values $X(\omega)$ in a probability space $(\Omega, \mathscr{F}, \mathbb{P})$,

$$\bar{X}_M = \frac{1}{M}\sum_{i=1}^{M} X(\omega^{(i)}) \tag{254}$$

where $\mathbb{E}[\bar{X}_M] = \mathbb{E}[X] := m_X$. The variance of $\bar{X}_M$ (or the Mean Square Error of $\epsilon_M = m_X - \bar{X}_M$, see Proposition 5) is

$$\text{Var}(\bar{X}_M) = \mathbb{E}[(m_X - \bar{X}_M)^2] = \frac{\sigma_X^2}{M} \tag{255}$$

where $\sigma_X^2 = \text{Var}(X)$. So, the Root Mean Square error is

$$\mathbb{E}[(m_X - \bar{X}_M)^2]^{1/2} = \frac{\sigma_X}{\sqrt{M}} \tag{256}$$

An accuracy of ε requires (see Remark 26)

$$M = O\left(\frac{1}{\varepsilon^2}\right) \tag{257}$$

samples.

- Weakness of MC comes from (257), where the computational cost of MC can be high when each sample $X(\omega)$ comes from an approximate solution of a PDE or a discretization scheme.

Variance Reduction

We would like to improve the constant factor in the error (256) by reducing σ_X^2. Suppose there is a r.v Y such that $\mathbb{E}[X] = \mathbb{E}[Y]$, but with smaller variance, i.e.,

$$\sigma_Y^2 < \sigma_X^2$$

Then inserting σ_Y^2 in (256) will decrease the computational work to achieve a target error, provided that simulating Y is not more expensive that generating samples from X. With this approach, the same error can be achieved with fewer samples.

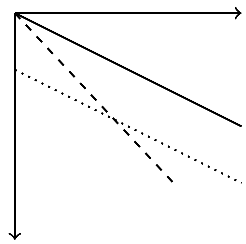

Figure 8: Improved convergence rate vs. improved constant.

- If we take log in both sides of (256) and plot $\log \sigma_X - \frac{1}{2} \log M$ in the x-axis against $\log \text{error}$ in the y-axis, we obtain the convergence rate of MC as a solid line with slope $-\frac{1}{2}$ in Figure 8.
- An improved constant will lead to a parallel shift of the line with slope $-\frac{1}{2}$, as the dotted line in Figure 8.
- An improved convergence rate would lead to a line with different slope, as in the dashed line with slope -1 in Figure 8.

Control Variates

Assume there exists a r.v. Y such that we know

$$\mathbb{E}[Y] \tag{258}$$

Then

$$\mathbb{E}[X] = \mathbb{E}\left[X - \lambda\left(Y - \mathbb{E}[Y]\right)\right] \tag{259}$$

$$= \mathbb{E}[X - \lambda Y] + \lambda \mathbb{E}[Y] \tag{260}$$

for any $\lambda \in \mathbb{R}$. Thus a Monte Carlo estimate for $\mathbb{E}[X]$ is given by

$$\bar{X}_M^\lambda = \frac{1}{M} \sum_{i=1}^{M} (X_i - \lambda Y_i) + \lambda \mathbb{E}[Y] \tag{261}$$

Assume that the simulation of $\bar{X}_M^\lambda$ requires at most c times the computing time of $\bar{X}_M$, where $c > 1$. We are going to chose λ such that

$$\text{Var}\,(X - \lambda Y) \;\text{ is minimized} \tag{262}$$

We have that

$$\text{Var}(X - \lambda Y) = \text{Var}(X) - 2\lambda\,\text{Cov}(X, Y) + \lambda^2\,\text{Var}(Y), \tag{263}$$

is minimized by choosing

$$\lambda^* = \frac{\text{Cov}(f(X), g(Y))}{\text{Var}(g(Y))} = \rho_{XY}\frac{\sigma_X}{\sigma_Y}, \quad \rho_{XY} = \frac{\text{Cov}(X, Y)}{\sigma_X \sigma_Y} \tag{264}$$

For λ^* and from (255), the MSE of $\bar{X}_M^{\lambda^*}$ is given by

$$\text{Var}(\bar{X}_M^{\lambda^*}) = \frac{\sigma_X^2}{M}(1 - \rho^2) \le \frac{\sigma_X^2}{M} = \text{Var}(\bar{X}_M) \tag{265}$$

9 Multilevel Monte Carlo

Two Level Monte Carlo

Similarly as in variates control, We want to estimate $\mathbb{E}[P_1]$ but it is much cheaper to simulate P_0 which approximates P_1, then since

$$\mathbb{E}[P_1] = \mathbb{E}[P_0] + \mathbb{E}[P_1 - P_0] \tag{266}$$

we can use the unbiased two-level estimator

$$\frac{1}{N_0}\sum_{n=1}^{N_0} P_0^{(n)} + \frac{1}{N_1}\sum_{n=1}^{N_1}\left(P_1^{(n)} - P_0^{(n)}\right) \tag{267}$$

(a) We consider $P_1^{(n)} - P_0^{(n)}$ for the same underlying stochastic sample $\omega^{(n)}$, so that $P_1^{(n)}$ and $P_0^{(n)}$ are correlated and $P_1^{(n)} - P_0^{(n)}$ is small with small variance (see (263))

(b) The two main differences from the control variate approach is that $\mathbb{E}[P_0]$ is not known and $\lambda = 1$.

Let C_0 and C_1 be the cost of computing a single sample of P_0 and $P_1 - P_0$, respectively, then the total cost is

$$N_0 C_0 + N_1 C_1 \tag{268}$$

If V_0 and V_1 are the variance of P_0 and $P_1 - P_0$, then the overall variance is

$$\frac{1}{N_0}V_0 + \frac{1}{N_1}V_1 \tag{269}$$

Assuming that

$$\sum_{n=1}^{N_0} P_0^{(n)} \quad \text{and} \quad \sum_{n=1}^{N_1} \left(P_1^{(n)} - P_0^{(n)} \right) \tag{270}$$

use independent samples, we consider the optimization problem:

- **Objective:** Minimize V_{total} where

$$V_{\text{total}} = \frac{V_0}{N_0} + \frac{V_1}{N_1}$$

- **Constraint:**

$$N_0 C_0 + N_1 C_1 = C_{\text{Total}} \quad \text{(Total Cost, fixed)}$$

Hence, the variance is minimized for a fixed cost by choosing

$$\frac{N_1}{N_0} = \frac{\sqrt{\frac{V_1}{C_1}}}{\sqrt{\frac{V_0}{C_0}}} \tag{271}$$

9.1 Multilevel Monte Carlo

Given a sequence $P_0, \ldots, P_{L-1}$ which approximates P_L with increasing accuracy but also increasing cost, we have the simple identity

$$\mathbb{E}[P_L] = \mathbb{E}[P_0] + \sum_{\ell=1}^{L} \mathbb{E}[P_\ell - P_{\ell-1}] \tag{272}$$

and therefore we can use the unbiased estimator for $\mathbb{E}[P_L]$:

$$\underbrace{\frac{1}{N_0} \sum_{n=1}^{N_0} P_0^{(0,n)}}_{\text{level 0 estimate}} + \underbrace{\sum_{\ell=1}^{L} \frac{1}{N_\ell} \sum_{n=1}^{N_\ell} \left(P_\ell^{(\ell,n)} - P_{\ell-1}^{(\ell,n)} \right)}_{\text{level } \ell \text{ ``correction''}} \tag{273}$$

- Within one level we reuse the *same* path to compute the pair $(P_\ell, P_{\ell-1})$ so that the *difference* benefits from strong coupling.
- Across levels you start afresh; level 2 does not reuse paths from level 1, etc. The superscript (ℓ, n) indicate that independent samples are used at each level.

Let C_0, V_0 to be the cost and variance of one sample of P_0, and C_ℓ, V_ℓ to be the cost and variance of one sample of $P_\ell - P_{\ell-1}$, then the overall cost and variance of the multilevel estimator are:

$$\text{Cost} = \sum_{\ell=0}^{L} N_\ell C_\ell, \quad \text{Variance} = \sum_{\ell=0}^{L} \frac{1}{N_\ell} V_\ell \tag{274}$$

For a fixed variance, we consider the optimization problem:

$$1.\ \text{Objective:}\quad \text{Minimize}\quad \sum_{\ell=0}^{L} N_\ell C_\ell \tag{275}$$

$$2.\ \text{Constraint:}\quad \sum_{\ell=0}^{L} \frac{1}{N_\ell} V_\ell = V_{\text{total}} = \varepsilon^2 \tag{276}$$

The cost is minimized by choosing N_ℓ to minimize

$$\sum_{\ell=0}^{L} \left(N_\ell C_\ell + \mu^2 N_\ell^{-1} V_\ell \right) \tag{277}$$

for some value of the Lagrange multiplier μ^2. This gives

$$N_\ell = \mu \sqrt{\frac{V_\ell}{C_\ell}} \tag{278}$$

To achieve an overall variance of ε^2, we require

$$\mu = \varepsilon^{-2} \sum_{\ell=0}^{L} \sqrt{V_\ell C_\ell} \tag{279}$$

and the total computational cost is therefore

$$C = \varepsilon^{-2} \left(\sum_{\ell=0}^{L} \sqrt{V_\ell C_\ell} \right)^2 \tag{280}$$

(a) It is important to know whether the product $V_\ell C_\ell$ increases or decreases with ℓ, i.e., whether C_ℓ increases or not with level faster than V_ℓ decreases

(b) If $V_\ell C_\ell$ increases with ℓ, the dominant contribution comes from $V_L C_L$, then $C_{MLMC} \approx \frac{1}{\varepsilon^2} V_L C_L$

(c) If $V_\ell C_\ell$ decreases with ℓ, the dominant contribution comes from $V_0 C_0$, then $C_{MLMC} \approx \frac{1}{\varepsilon^2} V_0 C_0$

(d) In Monte Carlo we estimate directly from P_L. Assume that the cost of computing P_L is similar to the cost of computing $P_L - P_{L-1}$, i.e., C_L and that $\mathbb{V}[P_L] \approx \mathbb{V}[P_0]$. then the cost of MC is $C_{MC} \approx \frac{1}{\varepsilon^2} V_0 C_L$ which constrast with Multilevel MC.

(e) Assuming the case (b) and (d), then $\frac{C_{MLMC}}{C_{MC}} = \frac{V_L}{V_0}$, which shows that the cost in MLMC is reduced by factor $\frac{V_L}{V_0}$.

(f) Assuming the case (c) and (d), then $\frac{C_{MLMC}}{C_{MC}} = \frac{C_0}{C_L}$, which shows that the cost in MLMC is reduced by factor $\frac{C_0}{C_L}$.

(g) Finally, if the product $V_\ell C_\ell$ does not vary with ℓ, then the total cost is $\varepsilon^{-2} L^2 V_0 C_0 = \varepsilon^{-2} L^2 V_L C_L$.

9.2 Geometric Multilevel Monte Carlo

Before we considered P_L the quantity of interest. In many infinite-dimensional applications, the output P_ℓ on level ℓ is an approximation to a random variable P which can not be simulated exactly.

If Y is an approximation of $\mathbb{E}[P]$, then the MSE reads

$$\mathrm{MSE} \equiv \mathbb{E}\left[(Y-\mathbb{E}[P])^2\right] = \underbrace{(\mathbb{E}[Y]-\mathbb{E}[P])^2}_{\text{bias}} + \underbrace{\mathbb{V}[Y]}_{\text{statistical error}} \tag{281}$$

(a) In Monte Carlo, we assume that P can be simulated and consider $Y = \bar{P}_M$, which has $\mathbb{E}[Y] = \mathbb{E}[P]$ and the first term in the right hand side of (281) vanish.

(b) In General Multilevel Monte Carlo, we assume that $P = P_L$ and P_L can be simulated. Then we consider the Multilevel approximation $Y = \sum_{\ell=0}^{L} Y_\ell$, which has $\mathbb{E}[Y] = \mathbb{E}[P_L]$ and the first term in the right hand side of (281) vanish.

If Y is now the multilevel estimator

$$Y = \sum_{\ell=0}^{L} Y_\ell, \quad Y_\ell = \frac{1}{N_\ell}\sum_{n=1}^{N_\ell}\left(P_\ell^{(\ell,n)} - P_{\ell-1}^{(\ell,n)}\right) \tag{282}$$

with $P_{-1} \equiv 0$, then

$$\mathbb{E}[Y] = \mathbb{E}[P_L], \quad \mathbb{V}[Y] = \sum_{\ell=0}^{L}\frac{1}{N_\ell}V_\ell, \quad V_\ell \equiv \mathbb{V}\left[P_\ell - P_{\ell-1}\right] \tag{283}$$

To get $\mathrm{MSE} \leq \varepsilon^2$ in (281), we ensure that

$$\mathbb{V}[Y] \leq \frac{\varepsilon^2}{2}, \quad (\mathbb{E}[P_L]-\mathbb{E}[P])^2 \leq \frac{\varepsilon^2}{2} \tag{284}$$

Idea: Geometric sequence of levels in which the cost increases exponentially with level, while both the weak error $\mathbb{E}[P_L - P]$ and the multilevel correction variance V_ℓ decrease exponentially, leads to the following theorem:

Theorem 12. *Let P denote a random variable, and let P_ℓ denote the corresponding level ℓ numerical approximation.*

If there exist independent estimators Y_ℓ based on N_ℓ Monte Carlo samples, each with expected cost C_ℓ and variance V_ℓ, and positive constants $\alpha, \beta, \gamma, c_1, c_2, c_3$ such that $\alpha \geq \frac{1}{2}\min(\beta,\gamma)$ and

(i) $\mathbb{E}[|P_\ell - P|] \leq c_1 2^{-\alpha\ell}$ *(Weak Error)*

(ii) $\mathbb{E}[Y_\ell] = \begin{cases} \mathbb{E}[P_0], & \ell = 0 \\ \mathbb{E}[P_\ell - P_{\ell-1}], & \ell > 0 \end{cases}$

(iii) $V_\ell \leq c_2 2^{-\beta\ell}$

(iv) $C_\ell \le c_3 2^{\gamma \ell}$

then there exists a positive constant c_4 *such that for any* $\varepsilon < e^{-1}$, *there are values* L *and* N_ℓ *for which the multilevel estimator*

$$Y = \sum_{\ell=0}^{L} Y_\ell \tag{285}$$

has a mean-square-error with bound

$$MSE \equiv \mathbb{E}\left[(Y - \mathbb{E}[P])^2 \right] < \varepsilon^2 \tag{286}$$

with a computational complexity C *with bound*

$$\mathbb{E}[C] \le \begin{cases} c_4 \varepsilon^{-2}, & \beta > \gamma \\ c_4 \varepsilon^{-2} (\log \varepsilon)^2, & \beta = \gamma \\ c_4 \varepsilon^{-2-(\gamma-\beta)/\alpha}, & \beta < \gamma \end{cases} \tag{287}$$

9.2.1 Multilevel Monte Carlo Implementation

Algorithm 1 MLMC algorithm

The geometric MLMC algorithm used for the numerical tests in this paper is:

```
start with L = 2, and initial target of N_0 samples on levels ℓ = 0, 1, 2
while extra samples need to be evaluated do
    evaluate extra samples on each level
    compute/update estimates for V_ℓ, ℓ = 0, ..., L
    define optimal N_ℓ, ℓ = 0, ..., L
    test for weak convergence
    if not converged then
        set L := L + 1, and initialise target N_L
    end if
end while
```

(a) In the above algorithm, the equation for the optimal N_ℓ comes from (278) and is

$$N_\ell = \left\lceil 2\, \varepsilon^{-2} \sqrt{V_\ell / C_\ell} \left(\sum_{\ell=0}^{L} \sqrt{V_\ell C_\ell} \right) \right\rceil, \tag{288}$$

where V_ℓ is the estimated variance, and C_ℓ is the cost of an individual sample on level ℓ. This ensures that the estimated variance of the combined multilevel estimator is less than $\frac{1}{2}\varepsilon^2$ (See (284))

(b) The test for weak convergence tries to ensure that $|\mathbb{E}[P - P_L]| < \varepsilon/\sqrt{2}$, to achieve an MSE which is less than ε^2 (See (284)), with ε being a user-specified r.m.s. accuracy. If $\mathbb{E}[P_\ell - P_{\ell-1}] \propto 2^{-\alpha \ell}$ then the remaining error is

$$\mathbb{E}[P - P_L] = \sum_{\ell=L+1}^{\infty} \mathbb{E}[P_\ell - P_{\ell-1}] = \mathbb{E}[P_L - P_{L-1}] \,/\, (2^\alpha - 1) \tag{289}$$

This leads to the convergence test $|\mathbb{E}[P_L - P_{L-1}]|/(2^\alpha - 1) < \varepsilon/\sqrt{2}$, but for robustness, we extend this check to extrapolate from the previous two data points $\mathbb{E}[P_{L-1} - P_{L-2}]$, $\mathbb{E}[P_{L-2} - P_{L-3}]$, and take the maximum over all three as the estimated remaining error.

(c) The conditions of the MLMC theorem assume *a priori* knowledge of the constants c_1 and c_2 governing the weak convergence and the variance convergence as $\ell \to \infty$. These two constants are in effect being estimated in the numerical algorithm described above. In addition, the accuracy of the variance estimate at each level depends on the size of the sample set. If it is small, then this variance estimate may be poor.

The results to be presented later all use the same two routines written in MATLAB:

- `mlmc.m`: driver code which performs the MLMC calculation using an application-specific routine to compute $\sum_n (P_\ell^{(n)} - P_{\ell-1}^{(n)})^p$ for $p = 1, 2$ and a specified number of independent samples;

- `mlmc_test.m`: a program which performs various tests and then calls `mlmc.m` to perform a number of MLMC calculations.

9.2.2 MLMC Driver Routine

The inputs and outputs of the `mlmc.m` function are:

Listing 1: My MATLAB algorithm

```
function [P, Nl, cost] = mlmc(mlmc_l,N0,eps,Lmin,Lmax,
                               alpha,beta,gamma, varargin)

multi-level Monte Carlo estimation
%Outputs
P      = value Y in the theory, approximation of E[P]
Nl     = number of samples at each level
cost   = total cost

%Inputs
N0     = initial number of samples          > 0
eps    = desired accuracy (rms error)       > 0
Lmin   = minimum level of refinement        >= 2
Lmax   = maximum level of refinement        >= Lmin

alpha -> weak error is  O(2^{-alpha*l})
beta  -> variance is    O(2^{-beta*l})
gamma -> sample cost is O(2^{gamma*l})

varargin = optional additional user variables to be passed to mlmc_l

if alpha, beta, gamma are not positive, then they will be estimated
```

The user must supply an application-specific function to perform the calculations to compute Y_ℓ on level ℓ.

$$Y_\ell = \frac{1}{N_\ell}\sum_{n=1}^{N_\ell}\left(P_\ell^{(\ell,n)} - P_{\ell-1}^{(\ell,n)}\right)$$

This function has to conform to the following template:

Listing 2: My MATLAB algorithm

```
 %mlmc_l = function for level l estimator

[sums, cost] = mlmc_fn(l,N, varargin)     low-level routine

inputs:  l = level
         N = number of samples
         varargin = optional additional user variables

output: sums(1) = sum(Y)
        sums(2) = sum(Y.^2)
        where Y are iid samples with expected value:
        E[P_0]           on level 0
        E[P_l - P_{l-1}] on level l>0
        cost = cost of N samples
```

The first part of the code initialises various parameters, and then uses `mlmc_l` to generate the initial N_0 samples on each of the first `L=Lmin` levels.

Listing 3: My MATLAB algorithm

```
    function [P, Nl, Cl] = mlmc(mlmc_l,N0,eps,Lmin,Lmax, ...
                            alpha0,beta0,gamma0, varargin)

% check input parameters

% initialisation
  alpha = max(0, alpha0);
  beta  = max(0, beta0);
  gamma = max(0, gamma0);
  theta = 0.25;

  L = Lmin;

  Nl(1:L+1)        = 0;
  suml(1:2,1:L+1)  = 0;
  costl(1:L+1)     = 0;
  dNl(1:L+1)       = N0;

  while sum(dNl) > 0
% update sample sums

```

```
22    for l=0:L
23      if dNl(l+1) > 0
24        [sums cost] = mlmc_l(l,dNl(l+1), varargin{:});
25        Nl(l+1)     = Nl(l+1)     + dNl(l+1);
26        suml(1,l+1) = suml(1,l+1) + sums(1);
27        suml(2,l+1) = suml(2,l+1) + sums(2);
28        costl(l+1)  = costl(l+1)  + cost;
29      end
30    end
```

(a) In line 14 `Nl` initialize an array of lenght $L+1$ which will store the N_l values for levels $\ell = 0, ..., L$, that is why of the lenght $L+1$.

(b) In line 15, `suml` is a matrix $2 \times L+1$, thaw will store $\text{sum1}(\ell) = \sum_{n=1}^{N_\ell} \left(P_\ell^{(\ell,n)} - P_{\ell-1}^{(\ell,n)} \right)$ (coming from mlmcl-see line 24) in the first row for the first levels $\ell = 0, ..., L\text{min}$. Similarly, for each level, the second row of `suml` stores $\text{sum2}(\ell) = \sum_{n=1}^{N_\ell} \left(P_\ell^{(\ell,n)} - P_{\ell-1}^{(\ell,n)} \right)^2$

(c) In line 16, `costl` will store the cost C_ℓ of N_ℓ samples produced by `mlmcl` at each level.

(d) In line 17, `dNl` is the prelimiray $N_\ell, \ell = 0, ..., Lmin$ set to N_0 initially. Observe that we use these N_0 samples for the first $Lmin$ levels in mlmcl (line 24), after that the actual N_ℓ will update and will be stored in `Nl`.

The next part, computes the estimates of interest

Listing 4: My MATLAB algorithm

```
1    % compute absolute average, variance and cost
2    ml = abs(   suml(1,:)./Nl); %ml stores Y_0,...,Y_L, so its
       lenght is initially Lmin+1
3    Vl = max(0, suml(2,:)./Nl - ml.^2); % Vl has lenght Lmin+1
4    Cl = costl./Nl;
```

(a) In line 2, `ml` $= |\mathbb{E}[Y_\ell]|$, which by theorem 12, is an unbiased estimator for $\mathbb{E}[P_\ell - P_{\ell-1}]$. So empirically, we are computing $\frac{1}{N_\ell} \sum_{n=1}^{N_\ell} \left(P_\ell^{(\ell,n)} - P_{\ell-1}^{(\ell,n)} \right)$

(b) In line 3, we are estimating $V_\ell = \mathbb{V}[P_\ell - P_{\ell-1}]$

(c) The routine mlmcl, returns cost=cost of N_ℓ samples. So, if we want to know the cost per sample C_ℓ, we compute `costl ./Nl`, as we do in line 4.

From the weak error in Theorem 12, we can see that

$$
\begin{aligned}
m_\ell = |\mathbb{E}[Y_\ell]| = |\mathbb{E}[P_\ell - P_{\ell-1}]| &= |\mathbb{E}[P_\ell - P] - \mathbb{E}[P_{\ell-1} - P]| \\
&\leq |\mathbb{E}[P_\ell - P]| + |\mathbb{E}[P_{\ell-1} - P]| \leq \mathbb{E}\left[|P_\ell - P|\right] + \mathbb{E}\left[|P_{\ell-1} - P|\right] \\
&\leq c_1 2^{-\alpha\ell} + c_1 2^{-\alpha(\ell-1)} = \left(c_1 + c_1 2^{\alpha}\right) 2^{-\alpha\ell}
\end{aligned} \tag{290}
$$

Similarly

$$m_{\ell-1} \leq (c_1 + c_1 2^{\alpha})\, 2^{-\alpha(\ell-1)} = 2^{\alpha}\,(c_1 + c_1 2^{\alpha})\, 2^{-\alpha\ell} \tag{291}$$

For the variance we have

$$V_\ell \leq c_2 2^{-\beta\ell} \quad \text{and} \quad V_{\ell-1} \leq 2^{\beta} c_2 2^{-\beta\ell} \tag{292}$$

The next part computes (or in later passes updates) the estimates for $m_\ell \equiv |\mathbb{E}[Y_\ell]|$, and $V_\ell \equiv \mathbb{V}[Y_\ell]$. If the mean and variance are decaying as expected, one would expect $m_\ell = 2^{-\alpha} m_{\ell-1}$, $V_\ell = 2^{-\beta} V_{\ell-1}$. To avoid possible problems from high kurtosis generating poor empirical estimates, the estimates for m_ℓ and V_ℓ are not allowed to decrease by more than factor $\frac{1}{2}$ relative to this anticipated value.

Listing 5: My MATLAB algorithm

```
1       % fix to cope with possible zero values for ml and Vl
2   % (can happen in some applications when there are few samples)
3       for l = 3:L+1
4         ml(l) = max(ml(l), 0.5*ml(l-1)/2^alpha);
5         Vl(l) = max(Vl(l), 0.5*Vl(l-1)/2^beta);
6       end
```

(a) In lines 4 and 5, if m_ℓ or V_ℓ are less than $0.5 * m_\ell(\ell-1)/2^{\alpha}$ or $0.5 * V_\ell(\ell-1)/2^{\beta}$, respectively, then take these values instead to preserve the decaying.

If the user has not supplied the values for α, β and/or γ. they are now estimated by linear regression.

Listing 6: My MATLAB algorithm

```
% use linear regression to estimate alpha, beta, gamma if not given
%
    A = repmat((1:L)',1,2).^repmat(1:-1:0,L,1);

    if alpha0 <= 0
      x     = A \ log2(ml(2:end))';
      alpha = max(0.5,-x(1));
    end

    if beta0 <= 0
      x     = A \ log2(Vl(2:end))';
      beta  = max(0.5,-x(1));
    end

    if gamma0 <= 0
      x     = A \ log2(Cl(2:end))';
      gamma = max(0.5,x(1));
    end
```

(a) From (290), $m_\ell = \tilde{c}2^{-\alpha\ell}$, taking $\log_2$ in both sides we get

$$\log_2 m_\ell = \log_2 \tilde{c} - \alpha\ell, \ell = 1, ..., L \tag{293}$$

We have $\log_2 m_\ell$ from the previous steps, so we estimate $\log_2 \tilde{c}$ and α using Linear regression. The solution x in line 4 is $x = [-\alpha, \log_2 \tilde{c}]$. Then we let $\alpha = \max(0.5, -x(1))$. Similarly we estimate β using linear regression and the vector `Vl` previously computed.

Next, the optimal N_ℓ each level is computed using (288), which in turn gives the number of additional samples which will need to be generated in the next pass.

Listing 7: My MATLAB algorithm

```
% set optimal number of additional samples
% ceil rounds each element of X following --> direction in the real line to the nearest integer.
Ns  = ceil( sqrt(Vl./Cl)*sum(sqrt(Vl.*Cl)) / ((1-theta)*eps^2) );
dNl = max(0, Ns-Nl);
```

In the final part of the loop, we test for weak convergence as in Algorithm 1. If a new level is added, its variance is estimated by extrapolation, and the optimal number of samples is updated.

Listing 8: My MATLAB algorithm

```
    % if (almost) converged, estimate remaining error and decide
% whether a new level is required
%
    if sum( dNl > 0.01*Nl ) == 0
      % 23/3/18 this change copes with cases with erratic ml values
      range = 0:min(2,L-1);
      rem = max(ml(L+1-range) ./ 2.^(range*alpha)) / (2^alpha - 1);
      %       rem = ml(L+1) / (2^alpha - 1);

      if rem > sqrt(theta)*eps
        if (L==Lmax)
          fprintf(1,'***failed␣to␣achieve␣weak␣convergence***␣\n');
        else
          L         = L+1;
          Vl(L+1) = Vl(L) / 2^beta;
          Cl(L+1) = Cl(L) * 2^gamma;
          Nl(L+1) = 0;
          suml(1:2,L+1) = 0;
          costl(L+1) = 0;

          Ns   = ceil( sqrt(Vl./Cl) * sum(sqrt(Vl.*Cl)) ...
                                    / ((1-theta)*eps^2) );
          dNl = max(0, Ns-Nl);
        end
      end
    end
```

(a) In line 4, `dNl > 0.01*Nl` yields an array with 0 (false) or 1(true) in each level. Obtaining 0 in all levels $\ell's$ means that `Ns-Nl` is less than `0.01*Nl`, so `Nl` is already enough close to the optimal `Ns` for each level. Getting 1 at some level ℓ means that `Ns-Nl` is significant and we proceed to compute the remaining `Ns-Nl` paths at the level were 1 was obtained. This procedure is controled by the condition `while sum(dNl) > 0` at the begining of the function.

(b) If condition in blue above happens, then we proceed to check the weak error. For this we check if the weak error achieves a desired accuracy as explained in 9.2.1[(b)]. If the accuracy is not achieved then we add a new level as in line 14. Since for this new level we want to know the number of sample paths, we use the rate for $V_{L+1} = 2^{-\beta} V_L$ and $C_{L+1} = 2^{\gamma} C_L$ to aproximate V_L and C_L. With these two values we then can cmpute the optimal N_{L+1} as in line 21. Then because of this new level we have `dNl>0`, so we return to the begining of the function to compute the actual values for $\mathbb{E}[P_{L+1} - P_L]$ and $\mathbb{V}[P_{L+1} - P_L]$ now using the N_{L+1} samples.

The final step after the completion of the main loop is to compute the combined multilevel estimator:

Listing 9: My MATLAB algorithm

```
% finally, evaluate multilevel estimator
P = sum(suml(1,:)./Nl);
end
```

9.2.3 MLMC Test routine

`mlmc_test.m` first performs a set of calculations using a fixed number of samples on each level of resolution, and produces four plots:

- $\log_2(V_\ell)$ versus level ℓ
- $\log_2(|\mathbb{E}[P_\ell - P_{\ell-1}]|)$ versus level ℓ
- consistency check versus level
- kurtosis versus level

(a) Consistency Check versus level: Suppose $a = \mathbb{E}[P^f_{\ell-1}]$, $b = \mathbb{E}[P^f_\ell]$, and $c = \mathbb{E}[Y_\ell]$ (This comes from our mlmc simulations), then it should be true that $a - b + c \approx 0$. The consistency check verifies that this is true, to within the accuracy one would expect due to Monte Carlo sampling error. This is particularly important when $P^f_\ell \neq P^c_\ell$ because one is using different approximations on the coarse and fine levels.

Since
$$\sqrt{\mathbb{V}[a-b+c]} \;\leq\; \sqrt{\mathbb{V}[a]} + \sqrt{\mathbb{V}[b]} + \sqrt{\mathbb{V}[c]} \tag{294}$$
it computes and plots the ratio
$$\frac{|a-b+c|}{3\left(\sqrt{V_a} + \sqrt{V_b} + \sqrt{V_c}\right)} \tag{295}$$

where V_a, V_b, V_c are empirical estimates for the variances of a, b, c. The probability of this ratio being greater than unity is less than 0.3%. Hence, if it is, it indicates a likely programming error, or a mathematical error in the formulation which violates the crucial identity (299).

(b) kurtosis: The MLMC approach needs a good estimate for $V_\ell = \mathbb{V}[Y_\ell]$. When the number of samples N is large, the standard deviation of the sample variance for a random variable X with zero mean is approximately $\sqrt{(\kappa-1)/N}\ \mathbb{E}[X^2]$ where the kurtosis κ is defined as $\kappa = \mathbb{E}[X^4]/\mathbb{E}[X^2]^2$.

Hence $O(\kappa)$ samples are required to obtain a reasonable estimate for the variance. As well as plotting the kurtosis κ_ℓ, `mlmc_test.m` will give a warning if κ_ℓ is very large, as this is an indication that the empirical variance estimate may be poor. See remark below.

Remark 19. Let X be mean-zero with variance $V = \mathbb{E}[X^2]$ and kurtosis $\kappa = \mathbb{E}[X^4]/\mathbb{E}[X^2]^2$. The standar deviation (sampling spread) of the variance estimator $\hat{V}$ from N samples is

$$\mathrm{sd}(\hat{V}) \approx V\sqrt{\frac{\kappa-1}{N}}, \quad \text{so} \quad \frac{\mathrm{sd}(\hat{V})}{V} \approx \sqrt{\frac{\kappa-1}{N}}. \tag{296}$$

To target a relative s.d. $\leq r$ you need

$$N \gtrsim \frac{\kappa-1}{r^2} \quad \Rightarrow \quad N = O(\kappa). \tag{297}$$

This is the "need $O(\kappa)$ samples" statement. **High kurtosis** $\Rightarrow$ heavy tails / rare large values $\Rightarrow$ the sample variance is noisy. You need $N = O(\kappa)$ samples for a *reliable* variance estimate. If κ is large and N modest, $\hat{V}$ can fluctuate a lot (often *underestimating* when rare events aren't seen), so your variance estimate is poor.

Remark 20. Equation (282) gives the natural choice for the multilevel correction estimator Y_ℓ. However, the multilevel theorem allows for the use of other estimators, provided they satisfy the condition ii) which ensures that $\mathbb{E}[Y] = \mathbb{E}[P_L]$. Some use slightly different numerical approximations for the coarse and fine paths in SDE simulations, resulting in the estimator

$$Y_\ell = N_\ell^{-1} \sum_{n=1}^{N_\ell} \left(P_\ell^f \left(\omega^{(n)}\right) - P_{\ell-1}^c \left(\omega^{(n)}\right)\right). \tag{298}$$

Provided we maintain the identity

$$\mathbb{E}\left[P_\ell^f\right] = \mathbb{E}\left[P_\ell^c\right] \tag{299}$$

so that the expectation on level ℓ is the same for the two approximations, then condition ii) is satisfied and no additional bias is introduced, namely,

$$\mathbb{E}[Y_\ell] = \mathbb{E}\left[P_\ell^f - P_{\ell-1}^c\right] \tag{300}$$

$$= \underbrace{\left(\mathbb{E}\left[P_\ell^f\right] - \mathbb{E}\left[P_\ell^c\right]\right)}_{=0 \text{ by } (299)} + \mathbb{E}\left[P_\ell^c - P_{\ell-1}^c\right] \tag{301}$$

$$= \mathbb{E}\left[P_\ell^c - P_{\ell-1}^c\right]. \tag{302}$$

The next fragment shows the information about the function mlmc_test, its inputs and outputs:

Listing 10: My MATLAB algorithm

```
function mlmc_test(mlmc_l, N,L, N0,Eps,Lmin,Lmax, fp, varargin)

multilevel Monte Carlo test routine

[sums cost] = mlmc_l(l,N, varargin)     low-level routine

    inputs:  l = level
         N = number of paths

    output: sums(1) = sum(Pf-Pc)
        sums(2) = sum((Pf-Pc).^2)
        sums(3) = sum((Pf-Pc).^3)
        sums(4) = sum((Pf-Pc).^4)
        sums(5) = sum(Pf)
        sums(6) = sum(Pf.^2)
        cost    = user-defined computational cost

N         = number of samples for convergence tests
L         = number of levels for convergence tests

N0        = initial number of samples for MLMC calcs
Eps       = desired accuracy array for MLMC calcs
Lmin      = minimum number of levels for MLMC calcs
Lmax      = maximum number of levels for MLMC calcs

fp        = file handle for printing to file
varargin = optional additional user variables to be passed to mlmc_l
```

(a) The mlmc_l used in the test routine has more outputs in comparison with the mlmc_l function in the mlmc routine.

Listing 11: My MATLAB algorithm

```
    % first, convergence tests
N = 100*ceil(N/100);   % make N a multiple of 100
del1 = [];
del2 = [];
var1 = [];
var2 = [];
kur1 = [];
chk1 = [];
cost = [];

```

```
11 for l = 0:L
12 %  disp(sprintf('l = %d',l))
13   sums = 0;
14   cst  = 0;
15   parfor j=1:100 %iterations in j will be executed in parallel
16     RandStream.setGlobalStream( ...
17     RandStream.create('mrg32k3a','NumStreams',100,'StreamIndices',j)
          );
18
19     [sums_j cst_j] = mlmc_l(l,N/100, varargin{:});
20     sums  = sums  + sums_j/N;
21     cst   = cst   + cst_j/N;
22   end
```

(a) Line 15 splits the N samples into **100 equal batches**, and since N is divisible by 100, for each j runs $N/100$ paths.

(b) The next mini example tries to explain what is in lines 16-17. Basically we create 100 sequences or incides that are independents and are used to to generate the random numbers by each worker in the parallel loop. This avoids that different workers use same random numers.

Listing 12: My MATLAB algorithm

```
        % Creamos 3 secuencias independientes del RNG mrg32k3a.
s1  = RandStream.create('mrg32k3a','NumStreams',3,'StreamIndices
    ',1);
s2  = RandStream.create('mrg32k3a','NumStreams',3,'StreamIndices
    ',2);
s3  = RandStream.create('mrg32k3a','NumStreams',3,'StreamIndices
    ',3);

% Usamos la secuencia 1 como fuente global y tomamos 2 numeros
RandStream.setGlobalStream(s1);  a = rand(1,2); % = (u1^1, u2
    ^1)

% Cambiamos a la secuencia 2 y tomamos 2 numeros
RandStream.setGlobalStream(s2);  b = rand(1,2); % = (u1^2, u2
    ^2)

% Volvemos a la secuencia 1 y seguimos donde quedo su estado
RandStream.setGlobalStream(s1);  c = rand(1,2);   % = (u3^1, u4
    ^1)

% Re-creamos la secuencia 1 desde el inicio y comprobamos
    reproducibilidad
s1b = RandStream.create('mrg32k3a','NumStreams',3,'
    StreamIndices',1);
```

```
RandStream.setGlobalStream(s1b); d = rand(1,2);  % = (u1^1, u2^1)  igual que 'a'
```

(c) `mlmc_l` returns *totals* over that batch:

$$\text{sums\_j} = \big[\Sigma(P^f - P^c),\ \Sigma(P^f - P^c)^2,\ \Sigma(P^f - P^c)^3,\ \Sigma(P^f - P^c)^4,\ \Sigma P^f,\ \Sigma(P^f)^2\big], \quad \text{cst\_j} = \text{total cost.}$$

The lines `sums += sums_j/N` and `cst += cst_j/N` accumulate *global averages*: after the `parfor`,

$$\text{sums} = \frac{1}{N}\sum_{n=1}^{N}\big[Y,\ Y^2,\ Y^3,\ Y^4,\ P^f,\ (P^f)^2\big],$$

$$\text{cst} = \frac{1}{N}\sum \text{cost\_per\_path}.$$

Listing 13: My MATLAB algorithm

```
1      if (l==0)
2      kurt = 0.0;
3    else
4      kurt = (      sums(4)             ...
5                - 4*sums(3)*sums(1)     ...
6                + 6*sums(2)*sums(1)^2   ...
7                - 3*sums(1)*sums(1)^3 ) ...
8              / (sums(2)-sums(1)^2)^2;
9    end
10
11   cost = [cost cst];
12 del1 = [del1 sums(1)]; % E[Pf-Pc] = m_l
13 del2 = [del2 sums(5)]; % E[Pf]
14 var1 = [var1 sums(2)-sums(1)^2 ]; % Var(Pf-Pc) = V_l
15 var2 = [var2 sums(6)-sums(5)^2 ]; % Var(Pf)
16 var2 = max(var2, 1e-10); % avoid zero
17 kur1 = [kur1 kurt];
```

(a) In line 4, computes

$$\kappa_\ell = \frac{\mathbb{E}[Y_\ell^4] - 4\mu\mathbb{E}[Y_\ell^3] + 6\mu^2\mathbb{E}[Y_\ell^2] - 3\mu^4}{\left(\mathbb{E}[Y_\ell^2] - \mu^2\right)^2}, \quad \mu = \mathbb{E}[Y_\ell], \tag{303}$$

i.e., the kurtosis of Y_ℓ.

Listing 14: My MATLAB algorithm

```
1     if l==0
2     check = 0;
3   else
4     check = abs(       del1(l+1)   +      del2(l)   -      del2(l+1)) ...
5       /    ( 3.0*(sqrt(var1(l+1)) + sqrt(var2(l)) + sqrt(var2(l+1)) )/sqrt(N));
6   end
7   chk1 = [chk1 check];
8 
9   PRINTF2(fp,'%2d␣␣%11.4e␣%11.4e␣␣%.3e␣␣%.3e␣␣%.2e␣␣%.2e␣␣%.2e␣\n', ...
10          l,del1(l+1),del2(l+1),var1(l+1),var2(l+1),kur1(l+1),chk1(l+1),cst);
11 end (end of loop for l=0:L)
12 % print out a warning if kurtosis or consistency check looks bad
13 if ( kur1(end) > 100.0 )
14   PRINTF2(fp,'\n␣WARNING:␣kurtosis␣on␣finest␣level␣=␣%f␣\n',kur1(end));
15   PRINTF2(fp,'␣indicates␣MLMC␣correction␣dominated␣by␣a␣few␣rare␣paths;␣\n');
16   PRINTF2(fp,'␣for␣information␣on␣the␣connection␣to␣variance␣of␣sample␣variances,\n');
17   PRINTF2(fp,'␣see␣http://mathworld.wolfram.com/SampleVarianceDistribution.html\n\n');
18 end
19 
20 if ( max(chk1) > 1.0 )
21   PRINTF2(fp,'\n␣WARNING:␣maximum␣consistency␣error␣=␣%f␣\n',max(chk1));
22   PRINTF2(fp,'␣indicates␣identity␣E[Pf-Pc]␣=␣E[Pf]␣-␣E[Pc]␣not␣satisfied;␣\n');
23   PRINTF2(fp,'␣to␣be␣more␣certain,␣re-run␣mlmc_test␣with␣larger␣N␣\n\n');
24 end
```

(a) Line 4 computes the ration (295). For each level in the loop, if this ration is greater than 1, then there is a possible problem in the coupling or implementation of the estimator al level ℓ. At the end this causes a problem with condition ii) of the theorem, and consequently, the telecopic sum.

(b) In line 13, we check kurtosis on the finest level, a warning is preinted out if kurtosis is high.

The next part estimates α, β, γ, applying a linear regression using the values for each level computed before.

Listing 15: My MATLAB algorithm

```
 % use linear regression to estimate alpha, beta and gamma
L1 = 2;
L2 = L+1;
pa     = polyfit(L1:L2,log2(abs(del1(L1:L2))),1);  alpha = -pa(1);
pb     = polyfit(L1:L2,log2(abs(var1(L1:L2))),1);  beta  = -pb(1);
pg     = polyfit(L1:L2,log2(abs(cost(L1:L2))),1);  gamma =  pg(1);

PRINTF2(fp,'\n******************************************************\n');
PRINTF2(fp,'***␣Linear␣regression␣estimates␣of␣MLMC␣parameters␣***\n');
PRINTF2(fp,'******************************************************\n');
PRINTF2(fp,'\n␣alpha␣=␣%f␣␣(exponent␣for␣MLMC␣weak␣convergence)\n',alpha);
PRINTF2(fp,'␣beta␣␣=␣%f␣␣(exponent␣for␣MLMC␣variance)␣\n',beta);
PRINTF2(fp,'␣gamma␣=␣%f␣␣(exponent␣for␣MLMC␣cost)␣\n',gamma);
```

Next, we make a complexity test, for that we reset the random numbers generators and set the values for α, β, and γ.

Listing 16: My MATLAB algorithm

```
    % reset random number generators

reset(RandStream.getGlobalStream);
spmd
  RandStream.setGlobalStream( ...
  RandStream.create('mrg32k3a','NumStreams',numlabs,'StreamIndices',labindex));
end

alpha = max(alpha,0.5);
beta  = max(beta,0.5);
theta = 0.25;
```

In the final part, we computes our estimates using mlmc routine:

Listing 17: My MATLAB algorithm

```
    for i = 1:length(Eps)
  eps = Eps(i);
  [P, Nl, Cl] = mlmc(mlmc_l,N0,eps,Lmin,Lmax, alpha,beta,gamma, ...
                     varargin{:});
  mlmc_cost = sum(Nl.*Cl);
  std_cost  = var2(min(end,length(Nl)))*Cl(end) / ((1.0-theta)*eps^2);

  PRINTF2(fp,'%.3e␣%10.3e␣␣%.3e␣␣%.3e␣␣%7.2f␣', ...
```

```
            eps, P, mlmc_cost, std_cost, std_cost/mlmc_cost);
    PRINTF2(fp,'%9d',Nl);
    PRINTF2(fp,'\n');
end

PRINTF2(fp,'\n');

end
```

(a) `eps`: target accuracy ε for MSE.

`value`: MLMC estimate $P \approx \mathbb{E}[\text{payoff}]$ returned by `mlmc(...)`.

`mlmc_cost`: total MLMC work

$$\text{mlmc\_cost} = \sum_{\ell} N_\ell C_\ell \tag{304}$$

where N_ℓ = samples per level, C_ℓ = cost per sample on level ℓ.

`std_cost`: **single-level Monte Carlo cost on the finest level used**, to reach variance target $(1-\theta)\varepsilon^2$ (See remark below)

$$\text{std\_cost} = \frac{\mathrm{Var}(P^f_{L^*})\, C_{L^*}}{(1-\theta)\varepsilon^2} \tag{305}$$

coded as `var2(min(end,length(Nl)))*Cl(end)/((1.0-theta)*eps2̂)`.

`savings`: ratio std_cost/mlmc_cost (> 1 means MLMC is cheaper than standard MC).

`N_l`: the vector N_ℓ (optimal MLMC sample allocation across levels).

Remark 21. Crude MC on the finest level used by MLMC (call it L^*).

$$\bar{P}_N = \frac{1}{N}\sum_{i=1}^{N} P^{(i)}_{L^*}. \tag{306}$$

Its variance is

$$\mathrm{Var}(\bar{P}_N) = \frac{\mathrm{Var}(P_{L^*})}{N}. \tag{307}$$

In the test they use an **MSE split**:

$$\text{MSE} = \text{bias}^2 + \mathrm{Var}(\bar{P}_N) \le \varepsilon^2, \tag{308}$$

allocating

$$\text{bias}^2 \le \theta\varepsilon^2, \quad \mathrm{Var}(\bar{P}_N) \le (1-\theta)\varepsilon^2,$$

with $\theta = 0.25$ in the code. From $\mathrm{Var}(\bar{P}_N) \le (1-\theta)\varepsilon^2$ we get the **required sample size** for crude MC:

$$N_{\text{std}} \ge \frac{\mathrm{Var}(P_{L^*})}{(1-\theta)\varepsilon^2}. \tag{309}$$

If the *cost per sample* on level L^* is C_{L^*}, the total crude-MC cost is

$$\text{std\_cost} = N_{\text{std}} C_{L^*} \;\approx\; \frac{\text{Var}(P_{L^*})\, C_{L^*}}{(1-\theta)\varepsilon^2}. \tag{310}$$

9.3 Multilevel Monte Carlo Path Simulations

Consider the following multidimensional SDE

$$\mathrm{d}S(t) = a(S,t)\,\mathrm{d}t + b(S,t)\,\mathrm{d}W(t), \quad 0 < t < T, \tag{311}$$

with given initial data S_0. We want to compute $\mathbb{E}f(S(T))$, where $f(S)$ is a scalar function with a uniform Lipschitz bound, i.e.,

$$|f(U) - f(V)| \le c\|U - V\| \quad \forall\, U, V. \tag{312}$$

A simple Euler discretisation of this SDE with time-step h is

$$\hat{S}_{n+1} = \hat{S}_n + a(\hat{S}_n, t_n)h + b(\hat{S}_n, t_n)\Delta W_n, \tag{313}$$

and the simplest estimate for $\mathbb{E}[f(S_T)]$ is the mean of the payoff values $f(\hat{S}^{(i)}_{T/h})$, from N independent path simulations (Crude Monte Carlo):

$$\hat{Y} = N^{-1}\sum_{i=1}^{N} f(\hat{S}^{(i)}_{T/h}). \tag{314}$$

It is well established that, provided $a(S,t)$ and $b(S,t)$ satisfy certain conditions the expected mean-square error (MSE) in the estimate $\hat{Y}$ is asymptotically of the form

$$\text{MSE} \approx \underbrace{c_2 h^2}_{\text{bias}} + \underbrace{c_1 N^{-1}}_{\text{variance}}. \tag{315}$$

where c_1, c_2 are positive constants.

(a) See (253), the first terms is the square of the $O(h)$ bias introduced by the Euler discretisation (weak error) (249), and the second term is the variance in $\hat{Y}$ due to the Monte Carlo sampling (255)

(b) To make the MSE $O(\varepsilon^2)$ in (315), so that the RSME is $O(\varepsilon)$, requires that $N = O(\varepsilon^{-2})$ and $h = O(\varepsilon)$. Since the paths are independent, the total cost is $C \approx n \times N$, where $n = T/h = O(\varepsilon^{-1})$,hence the computational complexity (cost) is $O(\varepsilon^{-3})$

(c) The computational complexity $O(\varepsilon^{-3})$ can be reduced to $O(\varepsilon^{-2}(\log\varepsilon)^2)$ through the use of a multilevel MC method that reduces the variance, leaving unchanged the bias due to the Euler discretisation.

9.3.1 MLMC Method

- Consider Monte Carlo path simulations with different timesteps $h_\ell = M^{-\ell}T$, $\ell = 0, 1, \ldots, L$.
- For a given Brownian path $W(t)$, let
$$P = f(S(T)), \quad \text{and} \quad \hat{S}_{\ell,M^\ell} \approx S(T),\ \hat{P}_\ell \approx f(S(T)) = P \tag{316}$$
using a numerical discretisation with timestep h_ℓ.

It is clearly true that
$$\mathbb{E}[\hat{P}_L] = \mathbb{E}[\hat{P}_0] + \sum_{\ell=1}^{L} \mathbb{E}[\hat{P}_\ell - \hat{P}_{\ell-1}]. \tag{317}$$

(a) The multilevel method independently estimates each of the expectations on the right-hand of (317) side in a way that minimises the computational complexity.

Let $\hat{Y}_0$ be an estimator for $\mathbb{E}[\hat{P}_0]$ using N_0 samples, and let $\hat{Y}_\ell$ for $\ell > 0$ be an estimator for $\mathbb{E}[\hat{P}_\ell - \hat{P}_{\ell-1}]$ using N_ℓ paths. Consider the estimator as the mean of N_ℓ independent samples, which for $\ell > 0$ is
$$\hat{Y}_\ell = N_\ell^{-1} \sum_{i=1}^{N_\ell} \left(\hat{P}_\ell^{(i)} - \hat{P}_{\ell-1}^{(i)} \right). \tag{318}$$

The key point here is that the quantity $\hat{P}_\ell^{(i)} - \hat{P}_{\ell-1}^{(i)}$ comes from two discrete approximations with different timesteps but the same Brownian path. This is easily implemented by first constructing the Brownian increments for the simulation of the discrete path leading to the evaluation of $\hat{P}_\ell^{(i)}$, and then summing them in groups of size M to give the discrete Brownian increments for the evaluation of $\hat{P}_{\ell-1}^{(i)}$. For a more detailed explanation see Remark 27.

The variance of the estimator $\hat{Y}_\ell$ of $\mathbb{E}[P_\ell - P_{\ell-1}]$ is
$$\mathbb{V}[\hat{Y}_\ell] = \frac{1}{N_\ell} V_\ell, \quad V_\ell = \mathbb{V}[P_\ell - P_{\ell-1}] \tag{319}$$

The variance of the combined estimator $\hat{Y} = \sum_{\ell=0}^{L} \hat{Y}_\ell$ is
$$\mathbb{V}[\hat{Y}] = \sum_{\ell=0}^{L} \frac{1}{N_\ell} V_\ell \tag{320}$$

The computational cost, if one ignores the asymptotically negligible cost of the final payoff evaluation, is proportional to (for details see Remark 28)
$$\sum_{\ell=0}^{L} N_\ell\, h_\ell^{-1}. \tag{321}$$

Treating the N_ℓ as continuous variables, the variance is minimized for a fixed computational cost by choosing

$$N_\ell \propto \sqrt{V_\ell h_\ell} \tag{322}$$

The above analysis holds for any value of L. We now assume that $L \gg 1$, and consider the behaviour of V_ℓ as $\ell \to \infty$. In the particular case of the Euler discretisation and the Lipschitz payoff function, provided $a(S,t)$ and $b(S,t)$ satisfy certain conditions there is $O(h)$ weak convergence and $O(h^{1/2})$ strong convergence. Hence, as $\ell \to \infty$,

$$\mathbb{E}[\hat{P}_\ell - P] = O(h_\ell) \tag{323}$$

$$\mathbb{E}[\|\hat{S}_{\ell,M^\ell} - S(T)\|^2] = O(h_\ell) \tag{324}$$

From the Lipschitz property (312), it follows that

$$\mathbb{V}[\hat{P}_\ell - P] \le {}^{2}\mathbb{E}[(\hat{P}_\ell - P)^2] \le c^2\mathbb{E}[\|\hat{S}_{\ell,M^\ell} - S(T)\|^2] \tag{325}$$

Combining this with (324) gives $\mathbb{V}[\hat{P}_\ell - P] = O(h_\ell)$. Furthermore,

$$(\hat{P}_\ell - \hat{P}_{\ell-1}) = (\hat{P}_\ell - P) - (\hat{P}_{\ell-1} - P) \tag{326}$$

$$\Rightarrow \quad \mathbb{V}[\hat{P}_\ell - \hat{P}_{\ell-1}] \le \left(\mathbb{V}[\hat{P}_\ell - P]^{1/2} + \mathbb{V}[\hat{P}_{\ell-1} - P]^{1/2}\right)^2 \text{ (see Remark 29)} \tag{327}$$

Hence, for the simple estimator (318), the single sample variance

$$V_\ell = O(h_\ell) \tag{328}$$

and from (322), $N_\ell \propto h_\ell$. Setting $N_\ell = O(\varepsilon^{-2} L h_\ell)$, the variance of $\hat{Y}$ in (320) is $O(\varepsilon^2)$.

If L is now chosen such that

$$L = \frac{\log \varepsilon^{-1}}{\log M} + O(1), \tag{329}$$

as $\varepsilon \to 0$, then $h_L = M^{-L} = O(\varepsilon)$ (see Remar 30), and so the bias error $\mathbb{E}[\hat{P}_L - P]$ is $O(\varepsilon)$, due to (323). We recall that for any unbiased/biased Monte-Carlo estimator

$$\mathrm{MSE}(\hat{Y}) = \underbrace{\left(\mathbb{E}[\hat{Y}] - \mathbb{E}[P]\right)^2}_{\text{bias}} + \underbrace{\mathbb{V}[\hat{Y}]}_{\text{statistical error}} \tag{330}$$

Consequently, since $\mathbb{E}[\hat{Y}] = \mathbb{E}[P_L]$, we obtain an MSE that is $O(\varepsilon^2)$, with a computational complexity that is

$$O(\varepsilon^{-2}L) = O(\varepsilon^{-2}(\log \varepsilon)^2). \tag{331}$$

[2] $\mathrm{Var}(Z) = \mathbb{E}[Z^2] - \left(\mathbb{E}[Z]\right)^2 \le \mathbb{E}[Z^2]$ since $\left(\mathbb{E}[Z]\right)^2 \ge 0$.

9.3.2 Geometric Brownian Motion

We consider the GBM SDE under the risk-neutral measure

$$dS_t = rS_t\,dt + \sigma S_t\,dW_t, \qquad S_0 = K = 100, \tag{332}$$

with $r = 0.05$, $\sigma = 0.2$, and maturity $T = 1$. We want to estimate the price of an European call option, more precisely, we want to estimate P below

$$f(S_T) = (S_T - K)^+, \qquad P = e^{-rT}\,\mathbb{E}[f(S_T)]\,. \tag{333}$$

We want to approximate P using MLMC. To this end, we first approximate S_T using Euler Maruyama discretization as follows:

$$M = 4, \quad n_f = 4^\ell, \quad h_f = 4^{-\ell}T = \frac{T}{nf}, \quad t_i^f = ih_f, \quad i = 0, 1, \ldots, n_f. \tag{334}$$

Similraly, for the course level we have

$$M = 4, \quad n_c = 4^{\ell-1} = \frac{n_f}{4}, \quad h_c = 4^{-(\ell-1)}T = \frac{T}{n_c}, \quad t_j^c = jh_{\ell-1} = 4jh_c, \quad j = 0, 1, \ldots, n_c \tag{335}$$

Hence the Euler scheme is expressed as

$$S_{n+1}^{(\ell)} = S_n^{(\ell)} + rS_n^{(\ell)}h_\ell + \sigma S_n^{(\ell)}\Delta W_n^{(\ell)}, \quad \Delta W_n^{(\ell)} \sim N(0, h_\ell), n = 0, ..., 4^\ell - 1 \tag{336}$$

For the level $\ell - 1$, we have

$$S_{k+1}^{(\ell-1)} = S_k^{(\ell-1)} + rS_k^{(\ell-1)}h_{\ell-1} + \sigma S_k^{(\ell-1)}\Delta W_k^{(\ell-1)}, \quad \Delta W_k^{(\ell-1)} \sim \mathcal{N}(0, h_{\ell-1}), \quad k = 0, \ldots, 4^{\ell-1} - 1. \tag{337}$$

where the Brownian increments are obtained usuming the Brownina increments of the fine level ℓ

$$\Delta W_k^{(\ell-1)} = \sum_{m=1}^{4} \Delta W_{4(k-1)+m}^{(\ell)}. \tag{338}$$

Listing 18: My MATLAB algorithm

```
    function [sums, cost] = opre_l(l,N, option)

M = 4;

T   = 1;
r   = 0.05;
sig = 0.2;
K   = 100;

nf = M^l;
nc = nf/M;

hf = T/nf;
hc = T/nc;

sums(1:6) = 0;
```

Next we continue defining the estimator at level ℓ

Listing 19: My MATLAB algorithm

```
for N1 = 1:10000:N %N1 = [1,10001,20001,30001,..,(1 + 10000*k)leq N].
  N2 = min(10000,N-N1+1); %N-N1+1: Cuantos paths quedandesde N1 hasta N
    % N2 es: 10000 si quedan >= 10000 paths, el resto si quedan menos

    X0 = K;    % S0 = K = 100

    Xf = X0*ones(1,N2); % fine  paths at t=0 (N2 paths in this batch
    Xc = Xf;             % coarse paths at t=0
```

Listing 20: My MATLAB algorithm

```
    if l==0
      dWf = sqrt(hf)*randn(1,N2);
      Xf  = Xf + r*Xf*hf + sig*Xf.*dWf;
```

(a) For $\ell = 0$,$h_f = T$ (since $n_f = 1$), so you take **one** Euler–Maruyama step to reach $S_T^{(\ell=0)}$. For European, only `Xf` at the end is needed.

Listing 21: My MATLAB algorithm

```
1  for n = 1:nc   % nc coarse steps, each spans M fine steps
2    dWc = zeros(1,N2);
3    for m = 1:M                    % M = 4 fine substeps per coarse step
4      dWf = sqrt(hf)*randn(1,N2);    % dW_f -> N(0, hf)
5      dWc = dWc + dWf;    % aggregate: dW_c = sum dW_f -> N(0, hc)
6      Xf  = Xf + r*Xf*hf + sig*Xf.*dWf;    % fine Euler update
7    end
8    Xc = Xc + r*Xc*hc + sig*Xc.*dWc;       % one coarse Euler update
9  end
10 Pf = max(0,Xf-K);
11 Pc = max(0,Xc-K);
```

(a) For the case $\ell \geq 1$, line 5 implements the MLMC coupling:

$$\Delta W_c = \sum_{m=1}^{M} \Delta W_f \sim \mathcal{N}(0, h_c), \tag{339}$$

so fine and coarse share the same Brownian motion when viewed at coarse times. After the loop you have terminal values $Xf = S_T^{(\ell)}$ and $Xc = S_T^{(\ell-1)}$.

Listing 22: My MATLAB algorithm

```
    Pf = exp(-r*T)*Pf;
  Pc = exp(-r*T)*Pc;

  if l==0
    Pc=0;
  end

  sums(1) = sums(1) + sum(Pf-Pc);
  sums(2) = sums(2) + sum((Pf-Pc).^2);
  sums(3) = sums(3) + sum((Pf-Pc).^3);
  sums(4) = sums(4) + sum((Pf-Pc).^4);
  sums(5) = sums(5) + sum(Pf);
  sums(6) = sums(6) + sum(Pf.^2);
end
```

The cost at level ℓ is computed as follows

Listing 23: My MATLAB algorithm

```
    cost = N*nf;   % cost defined as number of fine timesteps times
       the number of samples
```

Remark 22. Here $h_\ell = 4^{-\ell} h_0$, then this gives $\alpha = 2$, $\beta = 2$ and $\gamma = 2$. Alternatively, if $h_\ell = 2^{-\ell} h_0$, then $\alpha = 1$, $\beta = 1$ and $\gamma = 1$. In either case, Theorem 1 gives the complexity to achieve a root-mean-square error of ε to be $\mathcal{O}(\varepsilon^{-2}(\log\varepsilon)^2)$.

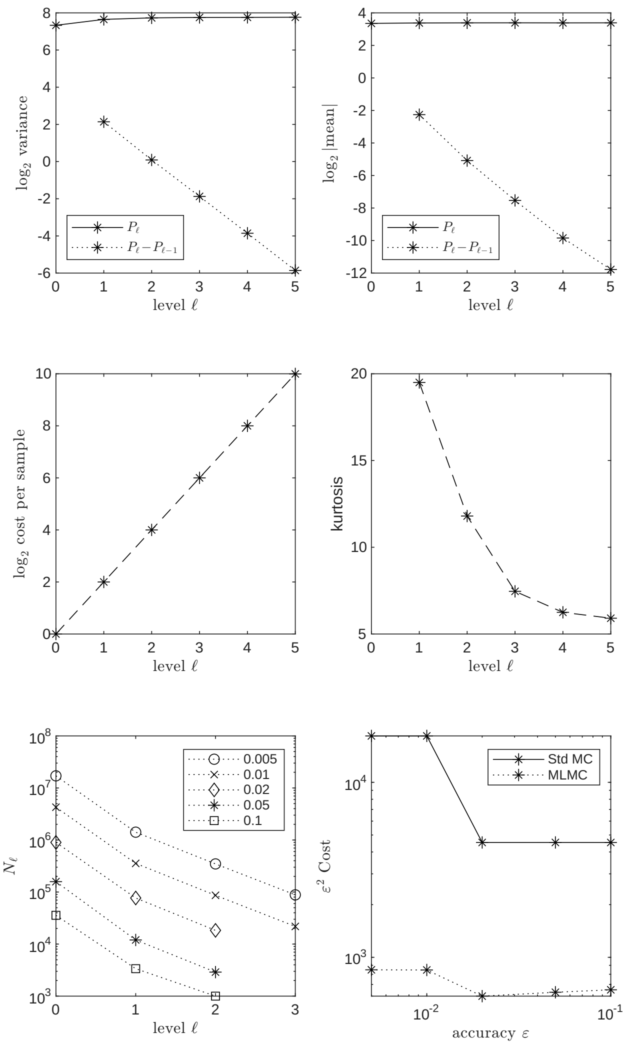

Figure 9: Numerical results for a European call option using the Euler-Maruyama discretisation of the GBM SDE.

Figure 9 shows results for a financial call option with a single underlying asset satisfying Geometric Brownian Motion SDE (332). From the weak error of the Euler-Maruyama discretization (323) and making a similar analysis as in (290) we obtain that

$$\mathbb{E}[P_\ell - P_{\ell-1}] = O(h_\ell)$$

Also we have from (328) thah

$$V_\ell = \mathbb{V}[\hat{P}_\ell - \hat{P}_{\ell-1}] = O(h_\ell)$$

(a) The top two plots show the variance and absolute mean value for both P_ℓ and $P_\ell - P_{\ell-1}$. The line for $\log_2 \mathbb{V}[P_\ell - P_{\ell-1}]$ in the top left plot has a slope of approximately -2 (If we take $\log_4$ the slope should be approx -1) corresponding to a variance proportional to h_ℓ, as expected. The line for $\log_2 |\mathbb{E}[P_\ell - P_{\ell-1}]|$ also has a slope of approximately -2, implying an $\mathcal{O}(h_\ell)$ weak convergence.

(b) The middle two plots show the cost per level. As we have did in the implementation $C_\ell = O(n_f) = O(M^\ell)$, plotting $\log_2 C_\ell$, the slope is approx 2. The other figure is the kurtosis, as discussed before. This indicate the kurtosis is actually improving slightly with level.

(c) The bottom two plots have the results from 5 separate MLMC calculations, each with a different specified accuracy ε to be achieved. The bottom left plot shows the number of samples which is used for each level of the MLMC computation. The number of levels increases as ε decreases, as described in the mlmc Algorithm to achieve the required weak error. The number of samples also varies with level, with many more on the coarsest level with just one timestep. On the *finer* levels, the optimal allocation of computational effort leads to

$$N_\ell \propto \sqrt{V_\ell / C_\ell} \approx \; .$$

The bottom right plot displays the total computational cost C, multiplied by ε^2. If C were approximately proportional to ε^{-2} then $\varepsilon^2 C$ would be approximately constant. Indeed this is what is seen, with the slight increase as ε decreases being due to the $|\log \varepsilon|^2$ term (287). For comparison, the bottom right plot also displays the cost using the standard Monte Carlo algorithm on the finest level used by the MLMC algorithm. The cost ratio is $5 - 12$, illustrating the benefits provided by the MLMC algorithm.

10 Appendix

Definition 11. Let $(\mathbf{X}_n)_{n\in\mathbb{N}}$ be a sequence of random vectors in $\mathbb{R}^m$ and let $\mathbf{X}$ be a random vector in $\mathbb{R}^m$. We say that $(\mathbf{X}_n)_{n\in\mathbb{N}}$ *converges in distribution* to $\mathbf{X}$ if

$$\lim_{n\to\infty} F_{\mathbf{X}_n}(\mathbf{t}) = F_{\mathbf{X}}(\mathbf{t})$$

for all $\mathbf{t} \in \mathbb{R}^m$ that are continuity points of $F_{\mathbf{X}}$. In this case, we write

$$X_n \xrightarrow{d} \mathbf{X} \quad \text{as } n \to \infty.$$

Remark 23. Note that convergence in distribution only depends on the distributions of $(X_n)_{n\in\mathbb{N}}$ and $\mathbf{X}$, whence they do not need to be defined on the same probability space. This is in contrast to almost sure convergence, convergence in probability and L^p-convergence, where all involved random variables must have the same underlying probability space. Sometimes convergence in distribution is also called *weak convergence*.

Theorem 13. *Let $(\mathbf{X}_n)_{n\in\mathbb{N}}$ be a sequence of random vectors in $\mathbb{R}^m$ and let $\mathbf{X}$ be a random vector in $\mathbb{R}^m$. Then, the following statements are equivalent:*

(a) $(\mathbf{X}_n)_{n\in\mathbb{N}}$ converges in distribution to $\mathbf{X}$.

(b) For each bounded continuous function $f : \mathbb{R}^m \to \mathbb{R}$,

$$\mathbb{E}[f(\mathbf{X}_n)] \to \mathbb{E}[f(\mathbf{X})] \quad \text{as} \quad n \to \infty. \tag{340}$$

Remark 24. From Remark 23 this means that weak convergence is not generally equivalent to say that

$$\mathbb{E}[f(X_n) - f(X)] \to 0 \text{ as } n \to \infty$$

but it means generally

$$\lim_{n\to\infty} \int f(x)\mu_n(dx) = \int f(x)\mu(dx) \tag{341}$$

Theorem 14 (Strong law of Larger Numbers)**.** *Let $(X_n)_{n\in\mathbb{N}}$ be i.i.d. random variables such that $\mathbb{E}[|X_1|] < \infty$ and let $S_n := \sum_{i=1}^n X_i$ for $n \in \mathbb{N}$. Then,*

$$\frac{S_n}{n} \xrightarrow{P\text{-}a.s.} \mathbb{E}[X_1] \quad \textit{as} \quad n \to \infty. \tag{342}$$

Theorem 15 (Central Limit Theorem)**.** *Let $(X_n)_{n\in\mathbb{N}}$ be a sequence of i.i.d. random variables such that $\mathbb{E}[X_1^2] < \infty$[3], define $\bar{X}_M := \frac{\sum_{i=1}^M X_i}{M}$ for $M \in \mathbb{N}$. Then,*

$$\sqrt{M}\frac{\overline{X}_M - m_X}{\sigma_X} \xrightarrow{\mathcal{L}} \mathcal{N}(0,1) \quad \textit{as} \quad M \to +\infty, \tag{343}$$

where $m_X = \mathbb{E}[X_1]$, and $\sigma_X^2 = Var(X) := \mathbb{E}\left((X - \mathbb{E}X)^2\right) = \mathbb{E}X^2 - (\mathbb{E}X)^2$ is the variance.

Proposition 5 (MSE and RMSE)**.** *Let $(X_n)_{n\in\mathbb{N}}$ be a sequence of i.i.d. random variables such that $X_1 \in L^2$ and $m_X = \mathbb{E}[X_1]$. Then for $\bar{X}_M := \frac{\sum_{i=1}^M X_i}{M}$, the error $m_X - \bar{X}_M$ in the L^2 norm, also denoted Root Mean Square Error (RMSE) is*

$$\|m_X - \bar{X}_M\|_{L^2} = \mathbb{E}[(m_X - \bar{X}_M)^2]^{1/2} = \frac{\sigma_X}{\sqrt{M}}\text{[4]} \tag{344}$$

Analogously, the Mean Square Error (MSE)is

$$\|m_X - \bar{X}_M\|_{L^2}^2 = \frac{\sigma_X^2}{M} \tag{345}$$

Proof. Let $\epsilon_M = X - \bar{X}_M$, then $\mathbb{E}[\epsilon_M] = 0$. Then

$$\|X - \bar{X}_M\|_{L^2}^2 = \mathbb{E}[\epsilon_M^2] = \mathrm{Var}\left(\epsilon_M\right) = \mathrm{Var}(m_X - \bar{X}_M) \tag{346}$$

$$= \mathrm{Var}(\bar{X}_M) = \mathrm{Var}\left(\frac{1}{M}\sum_{i=1}^M X_i\right) = \frac{M\sigma_X^2}{M^2} \tag{347}$$

$$= \frac{\sigma_X^2}{M} \tag{348}$$

$\square$

[3] $\mathbb{E}[X] < \infty$ by Cauchy-Schwarz inequality, so $\mathrm{Var}(X) = \mathbb{E}[X^2] + \mathbb{E}([X])^2 < \infty$

[4] Dividing the error by 10(one more significant digit) requires increase the simulations by 100

Proposition 6. *Let $(X_n)_{n\in\mathbb{N}}$ be a sequence of i.i.d. random variables such that $\mathbb{E}[X_1^2] < \infty$, define $\bar{X}_M := \frac{\sum_{i=1}^M X_i}{M}$ for $M \in \mathbb{N}$. Let $a, b \in \mathbb{R}^d$ with $a < b$, then*

$$\lim_{M\to\infty} \mathbb{P}\left(\sqrt{M}\frac{\overline{X}_M - m_X}{\sigma_X} \in [a,b]\right) = \mathbb{P}(\mathcal{N}(0,1) \in [a,b]) \quad as \quad M \to +\infty \tag{349}$$

$$= \Phi(b) - \Phi(a)^5 \tag{350}$$

where

$$\Phi(x) = \int_{-\infty}^{x} \frac{e^{-\frac{t^2}{2}}}{\sqrt{2\pi}} dt$$

Proof. Direct from CLT. □

Proposition 7 (Confidence Interval)**.** *Let $\alpha \in (0,1)$ and let q_α such that*

$$\mathbb{P}(|\mathcal{N}(0,1)| \leq q_\alpha) = \alpha \quad i.e., \quad 2\Phi(q_\alpha) - 1 = \alpha \tag{351}$$

Then the confidence Interval

$$J_M^\alpha := \left[\overline{X}_M - q_\alpha \frac{\sigma_X}{\sqrt{M}},\ \overline{X}_M + q_\alpha \frac{\sigma_X}{\sqrt{M}}\right] \tag{352}$$

satisfies

$$\mathbb{P}(m_X \in J_M^\alpha) = \mathbb{P}\left(\sqrt{M}\left|\frac{\overline{X}_M - m_X}{\sigma_X}\right| \leq q_\alpha\right) \xrightarrow[M\to+\infty]{} \mathbb{P}(|\mathcal{N}(0;1)| \leq q_\alpha) = \alpha. \tag{353}$$

Proof. From Proposition 6, letting $a = -q_\alpha$ and $b = q_\alpha$, we have

$$\mathbb{P}\left(-q_\alpha \leq \sqrt{M}\frac{\overline{X}_M - m_X}{\sigma_X} \leq q_\alpha\right) = \mathbb{P}\left(-\frac{\sigma_X q_\alpha}{\sqrt{M}} - \overline{X}_M \leq -m_X \leq \frac{\sigma_X q_\alpha}{\sqrt{M}} - \overline{X}_M\right) \tag{354}$$

$$= \mathbb{P}\left(\overline{X}_M - q_\alpha \frac{\sigma_X}{\sqrt{M}} \leq m_X \leq \overline{X}_M + q_\alpha \frac{\sigma_X}{\sqrt{M}}\right) \tag{355}$$

$$\xrightarrow[M\to\infty]{} \mathbb{P}(|\mathcal{N}(0;1)| \leq q_\alpha) = \alpha. \tag{356}$$

□

Remark 25. The interval J_M^α is theoretical since it involves σ_X. Monte Carlo can be used to estimate σ_X^2, namely

$$\overline{V}_M = \frac{1}{M-1}\sum_{k=1}^{M}\left(X_k - \overline{X}_M\right)^2 = \frac{1}{M-1}\sum_{k=1}^{M} X_k^2 - \frac{M}{M-1}\overline{X}_M^2 \xrightarrow[M\to\infty]{\mathbb{P}\text{-a.s.}} \sigma_X^2 \tag{357}$$

$$\mathbb{E}\left[\overline{V}_M\right] = \sigma_X^2, \quad \text{i.e., } \overline{V}_M \text{ is unbiased} \tag{358}$$

As a consequence, one derives by Slutsky Theorem that

$$\sqrt{M}\frac{\overline{X}_M - m_X}{\sqrt{\overline{V}_M}} = \sqrt{M}\frac{\overline{X}_M - m_X}{\sigma_X} \cdot \frac{\sigma_X}{\sqrt{\overline{V}_M}} \xrightarrow[M\to\infty]{\mathcal{L}} \mathcal{N}(0,1) \tag{359}$$

Finally, one derives the confidence interval at level α of the Monte Carlo Simulation by

$$I_M^\alpha := \left[\overline{X}_M - q_\alpha\sqrt{\frac{\overline{V}_M}{M}},\ \overline{X}_M + q_\alpha\sqrt{\frac{\overline{V}_M}{M}}\right] \tag{360}$$

which satisfy, for large M,

$$\mathbb{P}\left(m_X \in I_M^\alpha\right) \approx \mathbb{P}\left(|\mathcal{N}(0,1)| \leq q_\alpha\right) = \alpha \tag{361}$$

Usually $q_\alpha = 2$, which corresponds to $\alpha \approx 95\%$.

Remark 26 (Error Control)**.** We can define the error in Monte Carlo as $\epsilon_M = \bar{X}_M - m_X$. We choose M such that the probability of an error larger than a prescibed tolerance $\varepsilon > 0$ is smaller than $\delta > 0$, namely

$$\mathbb{P}\left(|\epsilon_M| > \varepsilon\right) < \delta \tag{362}$$

that is,

$$\mathbb{P}\left(|\epsilon_M| > \varepsilon\right) = 1 - \mathbb{P}\left(\varepsilon < \epsilon_M < \varepsilon\right) = 1 - \mathbb{P}\left(-\frac{\sigma_X\bar{\varepsilon}}{\sqrt{M}} < \epsilon_M < \frac{\sigma_X\bar{\varepsilon}}{\sqrt{M}}\right) \approx 1 - (2\Phi(\bar{\varepsilon}) - 1) \tag{363}$$

$$\approx 2 - 2\Phi(\bar{\varepsilon}) \tag{364}$$

where

$$\bar{\varepsilon} = \frac{\sqrt{M}\varepsilon}{\sigma_X} \tag{365}$$

Thus the right hand side holds for $M \to \infty$, hence solving for M we get,

$$\begin{aligned}
2 - 2\Phi\left(\frac{\sqrt{M}\varepsilon}{\sigma_X}\right) &< \delta \\
\Phi\left(\frac{\sqrt{M}\varepsilon}{\sigma_X}\right) &> \frac{2-\delta}{2} \\
\frac{\sqrt{M}\varepsilon}{\sigma_X} &> \Phi^{-1}\left(\frac{2-\delta}{2}\right) \\
M &\geq \left(\Phi^{-1}\left(\frac{2-\delta}{2}\right)\right)^2 \frac{\sigma_X^2}{\varepsilon^2}
\end{aligned}$$

1. For a fixed tolerance $\varepsilon > 0$, the size of of a Monte Carlo simulation grows linearly with the variance of X, σ_X^2.

2. For a fixed variance σ_X^2, the size of of a Monte Carlo simulation grows quadratically as the inverse of the prescribed accuracy $\varepsilon > 0$.

Remark 27 (Couplep Paths Simulation)**.**

$$\text{level } \ell: \quad h_\ell = \frac{T}{M^\ell} \tag{366}$$

$$\text{level } \ell-1: \quad h_{\ell-1} = \frac{T}{M^{\ell-1}} = Mh_\ell \tag{367}$$

We simulate Brownian increments

$$\Delta W_j^{(\ell)} = W(t_j) - W(t_{j-1}), \quad j = 1, \ldots, M^\ell \tag{368}$$

with

$$\Delta W_j^{(\ell)} \sim \mathcal{N}(0, h_\ell), \tag{369}$$

We need the increments

$$\Delta W_k^{(\ell-1)} = W(s_k) - W(s_{k-1}), \quad k = 1, \ldots, M^{\ell-1} \tag{370}$$

with

$$\Delta W_k^{(\ell-1)} \sim \mathcal{N}(0, h_{\ell-1}). \tag{371}$$

Assume that we start at time

$$t_{k-1}^{(\ell-1)} = (k-1)h_{\ell-1} = (k-1)Mh_\ell \tag{372}$$

until

$$t_k^{(\ell-1)} = kh_{\ell-1} = kMh_\ell \tag{373}$$

Observe that $t_k^{(\ell-1)} - t_{k-1}^{(\ell-1)} = h_{\ell-1}$. Hence the steps from level ℓ we need are from $j = M(k-1)+1$ until $j = Mk$, namely:

$$\sum_{j=M(k-1)+1}^{Mk} \Delta W_j^{(\ell)} \tag{374}$$

We compute the variance:

$$\begin{aligned} \mathbb{V}\left(\sum_{j=M(k-1)+1}^{Mk} \Delta W_j^{(\ell)}\right) &= \sum_{j=M(k-1)+1}^{Mk} \mathbb{V}[\Delta W_j^{(\ell)}] && (375)\\ &= (Mk - (M(k-1)+1)+1)h_\ell && (376)\\ &= Mh_\ell = h_{\ell-1} && (377) \end{aligned}$$

Hence we can make

$$\Delta W_k^{(\ell-1)} = \sum_{j=M(k-1)+1}^{Mk} \Delta W_j^{(\ell)} \tag{378}$$

Remark 28 (Cost MLMC)**.** At level ℓ:

- We generate N_ℓ independent coupled samples.

- Each sample uses an Euler discretization with timestep h_ℓ, so for a sample $P_\ell, P_{\ell-1}$ we have

$$h_\ell = \frac{T}{M^\ell}, \quad \text{and} \quad h_{\ell-1} = \frac{T}{M^{\ell-1}} \quad \text{time steps} \tag{379}$$

Therefore,

$$M^\ell = \frac{T}{h_\ell} = \mathcal{O}(h_\ell^{-1}), \quad M^{\ell-1} = \frac{T}{h_{\ell-1}} = \frac{T}{M h_\ell} = \mathcal{O}(h_\ell^{-1}) \tag{380}$$

Hence,

$$M^\ell + M^{\ell-1} = \mathcal{O}(h_\ell^{-1}) + \mathcal{O}(h_\ell^{-1}) = \mathcal{O}(h_\ell^{-1}) \tag{381}$$

Thus, the total cost at level ℓ is

$$N_\ell \cdot h_\ell^{-1}, \tag{382}$$

and the total computational cost is

$$C \propto \sum_{\ell=0}^{L} N_\ell \, h_\ell^{-1} \tag{383}$$

Remark 29.

$$X := \widehat{P}_\ell - P, \qquad Y := \widehat{P}_{\ell-1} - P \quad \Rightarrow \quad \widehat{P}_\ell - \widehat{P}_{\ell-1} = X - Y.$$

$$\begin{aligned}
\operatorname{Var}(\widehat{P}_\ell - \widehat{P}_{\ell-1}) &= \operatorname{Var}(X - Y) \\
&= \operatorname{Var}(X) + \operatorname{Var}(Y) - 2 \operatorname{Cov}(X, Y) \\
&\le \operatorname{Var}(X) + \operatorname{Var}(Y) + 2 \, |\operatorname{Cov}(X, Y)| \\
&\le \operatorname{Var}(X) + \operatorname{Var}(Y) + 2\sqrt{\operatorname{Var}(X) \operatorname{Var}(Y)} \\
&= \left(\sqrt{\operatorname{Var}(X)} + \sqrt{\operatorname{Var}(Y)}\right)^2 \\
&= \left(\sqrt{\operatorname{Var}(\widehat{P}_\ell - P)} + \sqrt{\operatorname{Var}(\widehat{P}_{\ell-1} - P)}\right)^2.
\end{aligned}$$

Remark 30 (Optimal L)**.** Solve $h_L = M^{-L} = O(\varepsilon)$ for L. We write

$$M^{-L} = c\varepsilon \quad \text{with some constant } c > 0. \tag{384}$$

Take natural logarithms:

$$- L \log M = \log c + \log \varepsilon. \tag{385}$$

Re-arranging,

$$L = -\frac{\log c}{\log M} + \frac{\log \varepsilon^{-1}}{\log M} \tag{386}$$

- The first term $-\log c / \log M$ does **not** depend on ε; it is a fixed number and is absorbed into the $O(1)$ term.

- The second term grows like $\log \varepsilon^{-1}$.

Therefore

$$L = \frac{\log \varepsilon^{-1}}{\log M} + O(1) \quad (\varepsilon \to 0). \tag{387}$$

Remark 31 (Complexity)**.** Insert N_ℓ into the cost sum (321)

$$\sum_{\ell=0}^{L} N_\ell h_\ell^{-1} = \sum_{\ell=0}^{L} \left(C\varepsilon^{-2} L h_\ell\right) h_\ell^{-1} = C\varepsilon^{-2} L \sum_{\ell=0}^{L} 1 \tag{388}$$

$$= C\varepsilon^{-2} L(L+1) \tag{389}$$

Because L is of order $\log \varepsilon^{-1}$, the last product behaves like

$$\text{Cost} = O\left(\varepsilon^{-2}(\log \varepsilon^{-1})^2\right) = O\left(\varepsilon^{-2}(-\log \varepsilon)^2\right) \tag{390}$$

which is exactly the complexity quoted:

$$O(\varepsilon^{-2} L^2) = O\left(\varepsilon^{-2}(\log \varepsilon)^2\right) \tag{391}$$

The authors absorb the factor $(L+1)$ into the asymptotic symbol, writing $O(\varepsilon^{-2}L^2)$ and then replacing L by its order $\log \varepsilon^{-1}$.

Remark 32. A good observation! We do not need to assume anything about the sign of v_i nor of $z_i = v_i^\top \lambda$. The bound

$$\delta \log \sum_i e^{z_i/\delta} - M \ \leq \ \delta \log K, \qquad M := \max_i z_i \tag{392}$$

follows only from the definition of M. Step by step: First, factor the largest exponent:

$$\sum_i e^{z_i/\delta} \ = \ e^{M/\delta} \sum_i e^{(z_i - M)/\delta}. \tag{393}$$

Since $M = \max_i z_i$, for all i it holds $z_i - M \leq 0$. Therefore,

$$e^{(z_i - M)/\delta} \leq 1 \quad \Longrightarrow \quad \sum_i e^{(z_i - M)/\delta} \ \leq \ \sum_i 1 = K. \tag{394}$$

Moreover, since there exists at least one index such that $z_i = M$, one of the summands is $e^0 = 1$, hence

$$1 \ \leq \ \sum_i e^{(z_i - M)/\delta} \ \leq \ K. \tag{395}$$

Now apply the logarithm and multiply by δ:

$$0 \ \leq \ \delta \log \sum_i e^{(z_i - M)/\delta} \ \leq \ \delta \log K. \tag{396}$$

Undo the common factor:

$$\delta \log \sum_i e^{z_i/\delta} = \delta \log \left(e^{M/\delta} \sum_i e^{(z_i - M)/\delta} \right) \tag{397}$$

$$= M + \delta \log \sum_i e^{(z_i - M)/\delta}. \tag{398}$$

Finally, subtracting M on both sides gives

$$0 \ \le \ \delta \log \sum_i e^{z_i/\delta} - M \ \le \ \delta \log K. \tag{399}$$

Theorem 16 (Transfer Theorem)**.**

References

[1] Barron, E., Jensen, R. (1986). *The Pontryagin Maximum Principle From Dynamic Programming and Viscosity Solutions to First-Order Partial Differential Equations*, *Transactions of the American Mathematical Society*, Vol. 298, No. 2, pp. 635–617.

[2] cannarsa, P., Siniestrari, C. (2004).*Semiconcave Functions,Hamilton–Jacobi Equations,and Optimal Control*, *Birkh¨auser*, Vol. 58.

[3] Evans, L. (2010). *Partial Differential Equations*, *American Mathematical Society*, Vol. 19